UNIVERSITÉ D'EBOLOWA
FACULTÉ DES SCIENCES
UNITÉ DE FORMATION DOCTORALE EN SCIENCES APPLIQUÉES ET TECHNOLOGIES INDUSTRIELLES

UNIVERSITY OF EBOLOWA
FACULTY OF SCIENCE
DOCTORAL TRAINING UNIT IN APPLIED SCIENCES AND INDUSTRIAL TECHNOLOGIES

DÉPARTEMENT DE PHYSIQUE APPLIQUÉE
Department of Applied Physics
LABORATOIRE DE PHYSIQUE ET APPLICATIONS TECHNOLOGIQUES INDUSTRIELLES
Laboratory of Physics and Industrial Technological Applications

ALGORITHME D'APPRENTISSAGE AUTOMATIQUE POUR UN SYSTÈME DE SURVEILLANCE EDGE AI

Mémoire présenté en vue de l'obtention du diplôme de Master 2 Recherche en Physique Appliquée

Spécialité : Systèmes numériques et automatiques des systèmes intelligents

Par

KENGNE TAKAM Ibrahim Eric

Matricule : 25A1005FS

PLETP en Électronique

Sous la direction de

Pr ATANGANA Jacques
(Professeur)

Dr TABI FOUDA Bernard Marie
(Chargé de Cours)

Année académique 2025–2026

UNIVERSITÉ D'EBOLOWA
FACULTÉ DES SCIENCES
UNITÉ DE FORMATION DOCTORALE EN SCIENCES APPLIQUÉES ET TECHNOLOGIES INDUSTRIELLES

UNIVERSITY OF EBOLOWA
FACULTY OF SCIENCE
DOCTORAL TRAINING UNIT IN APPLIED SCIENCES AND INDUSTRIAL TECHNOLOGIES

DÉPARTEMENT DE PHYSIQUE APPLIQUÉE
Department of Applied Physics
LABORATOIRE DE PHYSIQUE ET APPLICATIONS TECHNOLOGIQUES INDUSTRIELLES
Laboratory of Physics and Industrial Technological Applications

ALGORITHME D'APPRENTISSAGE AUTOMATIQUE POUR UN SYSTÈME DE SURVEILLANCE EDGE AI

Mémoire présenté en vue de l'obtention du diplôme de Master 2 Recherche en Physique Appliquée

Spécialité : Systèmes numériques et automatiques des systèmes intelligents

Par

KENGNE TAKAM Ibrahim Eric

Matricule : 25A1005FS

PLETP en Électronique

Sous la direction de

Pr ATANGANA Jacques
(Professeur)

Dr TABI FOUDA Bernard Marie
(Chargé de Cours)

Année académique 2025–2026

DÉDICACE

À ma famille,
Pour votre soutien indéfectible et vos sacrifices tout au long de ce parcours.

REMERCIEMENTS

Je tiens tout d'abord à exprimer ma profonde gratitude à mon directeur de mémoire, **Monsieur le Professeur ATANGANA Jacques**, ainsi qu'à mon co-directeur, **Monsieur le Docteur TABI FOUDA Bernard Marie** (Chargé de Cours), pour leur encadrement de qualité, leur patience, leurs conseils avisés et la confiance qu'ils m'ont accordée tout au long de ce travail de recherche.

Mes remerciements s'adressent également à Monsieur le Recteur de l'Université d'Ebolowa ainsi qu'à l'ensemble des responsables de l'École Doctorale pour les facilités administratives et académiques mises à notre disposition.

Je salue respectueusement tous les membres du corps enseignant et de l'Unité de Formation Doctorale des Sciences Appliquées et Technologies Industrielles **(UFD-SATI)**, en particulier ceux du Laboratoire de Physique et Applications Technologiques Industrielles, pour la rigueur scientifique transmise au cours de cette formation.

Enfin, je remercie chaleureusement mes collègues, mes amis et tous les membres de ma famille qui, de près ou de loin, ont contribué par leurs encouragements et leur soutien constant à la concrétisation de ce mémoire.

TABLE DES MATIÈRES

DÉDICACE i

REMERCIEMENTS ii

TABLE DES MATIÈRES iii

LISTE DES FIGURES vi

LISTE DES TABLEAUX vii

LISTE DES ABRÉVIATIONS, SIGLES ET ACRONYMES viii

LISTE DES DÉFINITIONS x

LISTE DES SYMBOLES ET UNITÉS xiii

RÉSUMÉ xv

ABSTRACT xvi

INTRODUCTION GÉNÉRALE 1

Contexte 2
Problématique 2
Objectif principal et objectifs spécifiques 3
Questions de recherche 3
Hypothèses de recherche 3
Intérêt de l'étude 4
Démarche méthodologique 5
Organisation du mémoire 5

CHAPITRE I : REVUE DE LA LITTÉRATURE 6

Introduction 7
I.1 Évolution des systèmes de vidéosurveillance 7
I.2 Fondements de l'apprentissage profond pour la vision 7
I.2.1 Réseaux de neurones convolutifs 7
I.2.2 Entraînement et transfert d'apprentissage 8
I.3 Méthodes classiques de détection d'objets 8

I.4 Détecteurs d'objets fondés sur l'apprentissage profond 9
I.4.1 Détecteurs en deux étapes 9
I.4.2 Détecteurs en une seule passe 9
I.4.3 Évolution de la famille YOLO 11
I.5 Architecture de YOLOv8 11
I.6 Métriques d'évaluation des détecteurs 12
I.7 Détection de personnes et de petites cibles 12
I.8 Suivi multi-objets 13
I.9 Intelligence en périphérie de réseau (*Edge AI*) 13
I.9.1 Du *cloud* à la périphérie 13
I.9.2 Modèles légers et compression 13
I.9.3 Microcontrôleurs et nœuds d'alerte 14
I.10 Vie privée et éthique de la vidéosurveillance 14
I.11 Synthèse et identification de l'écart de recherche (*Research Gap*) 14
Conclusion 15

CHAPITRE II : MATÉRIEL ET MÉTHODES 16

Introduction 17
II.1 Conception globale du système 17
II.2 Outils et architecture algorithmique 17
II.2.1 Modèle YOLOv8 Nano 17
II.2.2 Justification de l'approche Edge AI et contraintes matérielles . . . 18
II.3 Collecte et structuration des données 19
II.3.1 Présentation du jeu de données 19
II.3.2 Répartition des données (entraînement, validation, test) 19
II.4 Étapes de traitement et métriques d'évaluation 20
II.4.1 Processus d'apprentissage et hyperparamètres 20
II.4.2 Indicateurs de performance (précision, rappel, F1-score, mAP) . . 20
II.5 Inférence et alerte 21
II.6 Alimentation et interfaces du nœud d'alerte 23
II.7 Techniques éthiques et confidentialité des données vidéo 24
Conclusion 24

CHAPITRE III : RÉSULTATS ET DISCUSSION 25

Introduction 26
III.1 Présentation des résultats d'entraînement et d'inférence 26
III.1.1 Analyse de la convergence et des fonctions de perte 26
III.1.2 Performances de détection 26
III.1.3 Validation visuelle des inférences 27
III.2 Interprétation des performances du modèle YOLOv8n 27

III.2.1 Analyse des matrices de confusion 27
III.2.2 Choix du seuil de confiance 27
III.3 Performances temporelles 30
III.3.1 Modèle seul 30
III.3.2 Système complet 30
III.4 Validation expérimentale du nœud d'alerte ESP32 31
III.5 Implications opérationnelles pour la vidéosurveillance intelligente . . . 34
III.6 Contraintes et limites du système 35
Conclusion 35

CONCLUSION GÉNÉRALE, PERSPECTIVES ET RECOMMANDATIONS 36
Rappel de l'objectif et de la démarche 37
Réponses aux questions de recherche 37
Atteinte des objectifs et vérification des hypothèses 37
Limites de l'étude 38
Avantages et contributions du système 38
Perspectives 39
Recommandations 39

RÉFÉRENCES BIBLIOGRAPHIQUES 41

ANNEXES

LISTE DES FIGURES

1 Intersection sur l'union (IoU) 12

2 Architecture du système 18
3 Chaîne d'entraînement du détecteur 20
4 Organigrammes du logiciel 22
5 Alimentation et interfaces du nœud d'alerte ESP32 23

6 Évolution des pertes et des métriques au cours de l'entraînement 27
7 Lot de validation : vérité terrain et prédictions 28
8 Matrices de confusion de la classe `person` 28
9 Courbes de performance en fonction du seuil de confiance 29
10 Nœud d'alerte ESP32 : état « ZONE RAS » 31
11 Nœud d'alerte ESP32 : état « INTRUSION » 32
12 Nœud d'alerte ESP32 : fin de session 32
13 Nœud d'alerte ESP32 : perte de la connexion Wi-Fi 33
14 Seconde version de l'interface de l'afficheur OLED 34

15 Bloc 1 : charge et protection de la batterie XXVII
16 Bloc 2 : convertisseur élévateur MT3608 XXIX
17 Bloc 3 : régulateur ajustable AMS1117 XXXII
18 Bloc 4 : étage de calcul ESP32-WROOM-32 XXXIV
19 Schéma électrique complet du nœud d'alerte XXXVIII
20 Routage de la face avant de la carte V1.4 XLII
21 Routage de la face arrière de la carte V1.4 XLIII
22 Rendu 3D de la carte V1.4, vue de dessus XLIV
23 Rendu 3D de la carte V1.4, vue de dessous XLV
24 Rendu 3D de la carte V1.4 en perspective XLVI

LISTE DES TABLEAUX

1 Principales familles de détecteurs d'objets 10

2 Répartition du jeu de données *CCTV Person Detection* 19
3 Hyperparamètres d'entraînement (fichier `args.yaml`) 21

4 Métriques de validation (213 images, 1144 instances) 26
5 Durée des étapes de la boucle d'inférence 30

6 Affectation des broches de la carte de développement ESP32 XIII

7 Rails d'alimentation et classes de pistes XXVII
8 Tension de sortie du MT3608 selon la position du curseur de RV1. XXXI
9 Évolution du circuit imprimé du nœud d'alerte. XXXVII
10 Empilement des couches de la carte XXXIX
11 Distances caractéristiques du placement final XL
12 Résultats du routage de la version 1.4. XLI

LISTE DES ABRÉVIATIONS, SIGLES ET ACRONYMES

AdamW	Adaptive Moment Estimation with Weight Decay / Algorithme d'optimisation avec pondération de poids
API	Application Programming Interface / Interface de Programmation d'Application
BCE	Binary Cross-Entropy / Entropie Croisée Binaire
BoT-SORT	Robust Associations Multi-Pedestrian Tracking / Algorithme de suivi multi-objets (suivi par défaut d'Ultralytics)
ByteTRACK	Multi-Object Tracking by Associating Every Detection Box / Algorithme de suivi d'objets multiples
CCTV	Closed-Circuit Television / Télévision en Circuit Fermé (Vidéosurveillance)
CIoU	Complete Intersection over Union / Intersection sur Union Complète
CNN	Convolutional Neural Network / Réseau de Neurones Convolutif
COCO	Common Objects in Context / Jeu de données standardisé pour la vision par ordinateur
CP2102	Convertisseur USB-UART de Silicon Labs (liaison série PC-ESP32)
CPU	Central Processing Unit / Processeur Central
CUDA	Compute Unified Device Architecture / Architecture de calcul unifiée développée par NVIDIA
DFL	Distribution Focal Loss / Perte focale de distribution (régression des boîtes)
DL	Deep Learning / Apprentissage Profond
Edge AI	Edge Artificial Intelligence / Intelligence Artificielle à la Périphérie
ESP32	Microcontrôleur SoC (System on Chip) intégrant Wi-Fi et Bluetooth à basse consommation
FN	Faux Négatifs / False Negatives
FP	Faux Positifs / False Positives

FP32	Floating Point 32-bit / Précision en virgule flottante 32 bits (Pleine précision)
FPS	Frames Per Second / Images par Seconde (Fluidité temporelle)
GFLOP	Giga Floating Point Operations / Milliard d'opérations en virgule flottante (par image)
GPU	Graphics Processing Unit / Processeur Graphique
HDR	High Dynamic Range / Grande Plage Dynamique
I^{2}C	Inter-Integrated Circuit / Bus série à deux fils (écran OLED)
IA	Intelligence Artificielle
INT8	8-bit Integer / Précision entière sur 8 bits (Quantification post-entraînement)
IoT	Internet of Things / Internet des Objets
IoU	Intersection over Union / Intersection sur Union
JSON	JavaScript Object Notation / Format d'échange de données textuelles léger
LDO	Low-Dropout Regulator / Régulateur linéaire à faible chute de tension (AMS1117)
mAP	mean Average Precision / Précision Moyenne Générale
ML	Machine Learning / Apprentissage Automatique
MQTT	Message Queuing Telemetry Transport / Protocole léger de messagerie pour l'IoT
NMS	Non-Maximum Suppression / Suppression des Non-Maximums
OLED	Organic Light-Emitting Diode / Écran à diodes électroluminescentes organiques
PANet	Path Aggregation Network / Réseau d'Agrégation de Chemins
PTQ	Post-Training Quantization / Quantification post-entraînement
RGB	Red-Green-Blue / Rouge-Vert-Bleu
ROI	Region Of Interest / Région d'Intérêt
RTSP	Real-Time Streaming Protocol / Protocole de streaming en temps réel
SGD	Stochastic Gradient Descent / Descente de Gradient Stochastique
SORT	Simple Online and Realtime Tracking / Algorithme de suivi simple en temps réel
UART	Universal Asynchronous Receiver-Transmitter / Liaison série asynchrone (PC-ESP32)
UDP	User Datagram Protocol / Protocole de transport sans connexion (alerte Wi-Fi)
VN	Vrais Négatifs / True Negatives
VP	Vrais Positifs / Vrais Positifs
VRAM	Video Random Access Memory / Mémoire Vive Vidéo
Wi-Fi	Wireless Fidelity / Technologie de réseau local sans fil (norme IEEE 802.11)
YOLO	You Only Look Once / Algorithme de détection d'objets en un seul passage

LISTE DES DÉFINITIONS

Apprentissage supervisé Méthode d'apprentissage automatique où l'algorithme est entraîné sur des données préalablement étiquetées ou annotées par un opérateur humain.

Arrêt précoce (*Early Stopping*) Technique de régularisation consistant à interrompre l'entraînement dès que la perte sur l'ensemble de validation cesse de décroître, évitant ainsi le surapprentissage.

Boîte englobante (*Bounding Box*) Rectangle bidimensionnel défini par ses coordonnées spatiales, encadrant un objet détecté dans une image et servant de cible géométrique pour la régression.

BoT-SORT Algorithme de suivi multi-objets utilisé par défaut par la fonction `model.track()` d'Ultralytics; il associe les détections d'une trame à l'autre et attribue à chaque personne un identifiant de piste.

Bruit thermique Perturbation électronique aléatoire générée par l'agitation des porteurs de charge dans le capteur optique, particulièrement visible sous forme de grain dans les flux vidéos de nuit.

ByteTRACK Algorithme de suivi multi-objets (*Multi-Object Tracking*) basé sur l'association de données temporelles, exploitant l'ensemble des boîtes de détection, y compris celles à faible score de confiance, pour maintenir l'identité des cibles sous occlusion.

Convolution Opération mathématique combinant deux fonctions pour en créer une troisième. En traitement d'images, elle consiste à appliquer un filtre (noyau) sur une matrice de pixels.

Détection sans ancres (*Anchor-Free*) Approche algorithmique moderne (utilisée par YOLOv8) prédisant directement le centre et les dimensions d'un objet sans s'appuyer sur des boîtes de référence prédéfinies, réduisant ainsi la complexité de calcul.

Early Stopping Voir *Arrêt précoce*.

Edge AI Paradigme d'architecture réseau consistant à exécuter des modèles d'intelligence artificielle localement sur des équipements périphé-

riques embarqués, éliminant la dépendance au stockage ou traitement distant.

F1-Score Indicateur de performance statistique représentant la moyenne harmonique pondérée de la précision et du rappel, utile pour évaluer un modèle de détection sur une classe ciblée.

Flux vidéo Succession continue de trames ou d'images numériques transmises en temps réel depuis un capteur optique vers une unité de traitement.

Inférence Phase durant laquelle un modèle d'apprentissage automatique déjà entraîné est exécuté sur le terrain pour prédire ou classer de nouvelles données en temps réel.

Intersection over Union (IoU) Métrique évaluant le taux de superposition entre la boîte englobante prédite par l'algorithme et la boîte de vérité terrain dessinée lors de l'annotation.

Latence Délai temporel s'écoulant entre la capture d'un événement physique par un capteur optique et sa mise à disposition ou son traitement final par l'algorithme informatique.

Précision Proportion de détections correctes parmi toutes les alertes levées par le modèle. Elle mesure la capacité du système à limiter les fausses alarmes.

Quantification (*Quantization*) Processus de compression de modèles de réseaux de neurones consistant à convertir les poids et les activations d'une représentation continue haute précision (par exemple FP32) vers une représentation discrète de précision inférieure (par exemple INT8), réduisant l'empreinte mémoire et le temps de calcul.

Rappel Proportion d'objets réels correctement identifiés par le modèle parmi l'ensemble des cibles présentes. Elle mesure la sensibilité du système face aux intrusions.

Sémantique (Information sémantique) Niveau d'interprétation d'une donnée brute (pixels) qui lui confère une signification humaine ou logique métier (par exemple identifier qu'un groupe de pixels mouvants représente une « personne »).

Suppression des non-maximums (NMS) Algorithme de filtrage géométrique éliminant les boîtes englobantes redondantes superposées pour ne conserver que la prédiction ayant le score de confiance le plus élevé.

Surapprentissage (*Overfitting*) Phénomène par lequel un modèle mathématique s'adapte trop précisément aux bruits et spécificités des données d'entraînement, perdant ainsi sa capacité de généralisation sur le terrain.

Temps d'inférence Durée temporelle (exprimée en millisecondes) requise par un réseau de neurones pour traiter une trame d'entrée et restituer les tenseurs de prédiction.

Vérité terrain (*Ground Truth*) Information de référence collectée empiriquement ou annotée manuellement, considérée comme l'état réel et exact servant à l'évaluation du modèle.

Vision par ordinateur Branche de l'intelligence artificielle et du traitement du signal qui permet à un système informatique d'extraire, d'analyser et de comprendre des informations pertinentes à partir d'images.

LISTE DES SYMBOLES ET UNITÉS

Symboles

B_p, B_g	Boîte englobante prédite et boîte de vérité terrain
c	Indice de classe d'une annotation ($c = 0$: `person`)
F_1	F1-score, moyenne harmonique de la précision et du rappel
f	Cadence de traitement (images par seconde)
H, W	Hauteur et largeur de l'image (pixels)
h, w	Hauteur et largeur d'une boîte englobante (pixels)
I	Courant électrique consommé (A)
$\mathcal{L}$	Fonction de perte totale de YOLOv8
$\mathcal{L}_{\text{BCE}}$, $\mathcal{L}_{\text{CIoU}}$, $\mathcal{L}_{\text{DFL}}$	Pertes de classification, de localisation et de distribution
λ_{box}, λ_{cls}, λ_{dfl}	Poids des termes de la fonction de perte
N	Nombre de classes ($N = 1$ dans ce travail)
n	Nombre de personnes détectées, transmis au nœud d'alerte
P	Précision
P_{LDO}	Puissance dissipée par le régulateur linéaire (W)
R	Rappel
τ	Seuil de confiance de la détection
V_{in}, V_{out}	Tensions d'entrée et de sortie d'un régulateur (V)
$(x_{\text{c}}, y_{\text{c}})$	Coordonnées du centre d'une boîte englobante

Unités

A, mA, µA	Ampère, milliampère, microampère (courant)
bauds	Débit de la liaison série (symboles par seconde)
°	Degré (angle)
F, µF, nF, pF	Farad, microfarad, nanofarad, picofarad (capacité)

FPS	Images par seconde (*frames per second*)
GFLOP	Milliard d'opérations en virgule flottante
h, min, s	Heure, minute, seconde
ms, µs, ns	Milliseconde, microseconde, nanoseconde
Hz, kHz, MHz, GHz	Hertz et multiples (fréquence)
µH	Microhenry (inductance)
mAh, Wh	Milliampère-heure, wattheure (énergie d'une batterie)
mm, µm, mm^2	Millimètre, micromètre, millimètre carré
px	Pixel
V	Volt (tension)
W	Watt (puissance)
Ω, kΩ	Ohm, kilo-ohm (résistance)
‰	Pourcentage

RÉSUMÉ

La vidéosurveillance centralisée repose sur la transmission de flux vidéo vers des serveurs distants et sur la vigilance d'opérateurs humains; elle souffre d'une latence et d'une dépendance au réseau qui empêchent l'alerte en temps réel lorsque la connectivité est faible ou l'alimentation électrique instable. Ce travail développe un système de vidéosurveillance en périphérie de réseau (*Edge AI*) qui détecte localement les personnes et déclenche une alarme physique par l'intermédiaire d'un nœud d'alerte construit autour d'un microcontrôleur ESP32. Un modèle YOLOv8n, pré-entraîné sur MS COCO, est affiné pendant 100 époques sur des images de caméras de surveillance réelles. En exploitation, les trames sont traitées sur un poste local équipé d'un GPU; l'algorithme BoT-SORT suit les cibles d'une trame à l'autre, les résultats sont journalisés au format JSON et le nombre de personnes détectées est transmis par Wi-Fi (UDP) ou par liaison série au microcontrôleur, qui pilote un écran OLED et un voyant d'alarme. Sur un jeu de données de 1415 images (classe unique `person`), le modèle atteint sur l'ensemble de validation une précision de 91,9 %, un rappel de 89,9 %, une mAP@0.5 de 93,4 %, une mAP@0.5:0.95 de 50,8 % et un F1-score maximal de 0,91 pour un seuil de confiance de 0,459. Le détecteur seul traite une image en 7,1 ms (environ 140 images par seconde, FPS) sur un GPU NVIDIA RTX A500; la chaîne complète traite une trame en 47,9 ms en moyenne, soit 20,7 FPS effectifs, dont 25,8 ms pour la détection et le suivi, principal poste. Côté poste, l'alerte est émise 35,0 ms après le début de la capture de la trame. Ce modèle léger répond aux exigences de la détection d'intrusion sans transmettre d'image hors du site; une optimisation complémentaire de la chaîne de traitement reste nécessaire pour atteindre 25 FPS.

Mots-clés : Edge AI, YOLOv8, détection de personnes, temps réel, vidéosurveillance, microcontrôleur ESP32.

ABSTRACT

Centralized video surveillance relies on remote video streaming and human monitoring, and suffers from latency and network dependence that hinder real-time alerts under poor connectivity or power instability. This work develops an edge AI surveillance system that detects people locally and triggers a physical alarm through an alert node built around an ESP32 microcontroller. A YOLOv8n model, pre-trained on MS COCO, is fine-tuned over 100 epochs on real surveillance camera images. At run time, frames are processed on a local GPU workstation; the BoT-SORT algorithm tracks targets from frame to frame, results are logged in JSON, and person counts are transmitted via Wi-Fi (UDP) or a serial link to the microcontroller, which drives an OLED display and an alarm indicator. Using a 1415-image dataset (single class `person`), the model achieves on the validation set 91.9 % precision, 89.9 % recall, 93.4 % mAP@0.5, 50.8 % mAP@0.5:0.95 and a peak F1-score of 0.91 at a confidence threshold of 0.459. The detector alone processes an image in 7.1 ms (about 140 frames per second, FPS) on an NVIDIA RTX A500 GPU, while the full pipeline processes a frame in 47.9 ms on average, i.e., 20.7 effective FPS, of which 25.8 ms is spent on detection and tracking, the main contributor. On the workstation, the alert is issued 35.0 ms after the start of frame capture. This lightweight model meets intrusion detection requirements without sending any image off site; further pipeline optimization is needed to reach 25 FPS.

Keywords: Edge AI, YOLOv8, person detection, real-time, video surveillance, ESP32 microcontroller.

INTRODUCTION GÉNÉRALE

Contexte

Au cours de la dernière décennie, l'urbanisation rapide et la complexification des infrastructures publiques ont accru les défis liés à la sécurité des biens et des personnes. Face à ces enjeux, les systèmes de vidéosurveillance par circuit fermé (CCTV) se sont largement généralisés. Leur exploitation repose cependant sur des opérateurs humains qui doivent observer en continu de nombreux écrans, ce qui limite l'efficacité de la détection des événements et a motivé le développement de la vidéosurveillance automatisée (Elharrouss et al., 2021).

Pour pallier cette limite, l'intégration de la vision par ordinateur et de l'intelligence artificielle (IA) au traitement des flux vidéo a ouvert la voie à la vidéosurveillance intelligente. Historiquement adossée à des descripteurs conçus manuellement, comme la cascade de classifieurs de Viola et Jones (2004), la discipline a connu une rupture majeure avec l'apprentissage profond (*Deep Learning*) et les réseaux de neurones convolutifs (CNN) : d'abord avec les détecteurs en deux étapes comme Faster R-CNN (Ren et al., 2017), puis avec les détecteurs en une seule passe de la famille YOLO (Redmon et al., 2016 ; Terven et al., 2023), capables de localiser des silhouettes humaines en temps réel. Ces modèles restent toutefois coûteux en calcul et sont souvent exécutés sur des serveurs distants (*Cloud Computing*), ce qui impose de transmettre en continu des flux vidéo à haut débit.

Problématique

Le déploiement de modèles d'apprentissage automatique pour la vidéosurveillance intelligente se heurte à des verrous particulièrement aigus dans les pays en développement, et plus spécifiquement au Cameroun. Trois composantes de ce problème peuvent être identifiées :

1. **La dépendance au réseau :** l'approche fondée sur le *Cloud Computing* impose la transmission continue de flux vidéo vers des serveurs centralisés. Les réseaux locaux subissent des fluctuations de bande passante et des interruptions de service, qui induisent une latence incompatible avec une alerte immédiate (Shi et al., 2016).
2. **Les contraintes énergétiques et matérielles :** les serveurs de calcul à processeurs graphiques (GPU) sont coûteux et énergivores. Face à l'instabilité du réseau électrique, il faut des solutions sobres, capables de s'exécuter localement sur du matériel courant (processeur standard, mémoire vidéo VRAM limitée).
3. **La complexité des modèles :** les détecteurs les plus précis comportent un grand nombre de paramètres, difficiles à exécuter en temps réel sur des ressources limitées.

Le calcul en périphérie de réseau (*edge computing*) répond à ces contraintes en traitant les données au plus près du capteur (Shi et al., 2016 ; Zhou et al., 2019).

Objectif principal et objectifs spécifiques

L'objectif principal de ce travail est de concevoir et d'évaluer un système de vidéosurveillance en périphérie de réseau (*Edge AI*) qui détecte localement les personnes dans un flux vidéo et déclenche une alerte physique sans dépendre d'un serveur distant.

Les objectifs spécifiques sont les suivants :

— affiner un détecteur léger, YOLOv8 Nano (Jocher et al., 2023), sur un jeu de données public d'images de caméras de surveillance réelles (Konkuk University, 2024), confronté aux occlusions, aux distorsions optiques et aux variations d'éclairage;

— évaluer ses performances avec des indicateurs standardisés (précision, rappel, F1-score, mAP) et choisir un seuil de confiance opérationnel;

— intégrer le détecteur dans une chaîne complète : suivi des cibles, journalisation des détections et transmission de l'alerte à un microcontrôleur ESP32 (Espressif Systems, s. d.) ;

— mesurer les performances temporelles de cette chaîne et analyser ses implications pour la protection de la vie privée (Zuboff, 2019).

Questions de recherche

La démarche scientifique et expérimentale de ce travail est guidée par les questions de recherche suivantes :

Q1. Quelle performance de détection (précision, rappel, mAP) le détecteur léger YOLOv8n atteint-il après affinage sur des images de caméras de surveillance réelles ?

Q2. Quel seuil de confiance retenir en exploitation, compte tenu du compromis entre fausses alarmes et intrusions manquées ?

Q3. Le système complet (détection, suivi, journalisation et transmission de l'alerte à un microcontrôleur) respecte-t-il la contrainte de temps réel ($f \geq 25\,\text{images/s}$) sur un poste local ?

Hypothèses de recherche

À ces questions correspondent les hypothèses suivantes, vérifiées au chapitre III et discutées dans la conclusion générale :

H1. Affiné par transfert d'apprentissage sur des images de caméras de surveillance réelles, le détecteur léger YOLOv8n atteint une mAP@0.5 d'au moins 90 % sur la classe `person`, malgré les occlusions et les variations d'éclairage.

H2. Un seuil de confiance intermédiaire, voisin de celui qui maximise le F1-score, offre le meilleur compromis entre fausses alarmes et intrusions manquées.

H3. Le modèle seul respecte largement le budget de 40 ms par trame sur un GPU d'entrée de gamme; le respect du temps réel par la chaîne complète dépend surtout des étapes extérieures au modèle (suivi, annotation, enregistrement, affichage).

Intérêt de l'étude

Portée théorique

Cette étude contribue à documenter le comportement d'un détecteur compressé de l'état de l'art lorsqu'il est affiné sur des images de surveillance réelles. Elle analyse la convergence de la fonction de perte de YOLOv8, qui combine l'entropie croisée binaire pour la classification, la perte CIoU (Zheng et al., 2020) et la *Distribution Focal Loss* (Li et al., 2020) pour la régression des boîtes englobantes, ainsi que l'écart entre la mAP@0.5 et la mAP@0.5:0.95, qui renseigne sur la précision de localisation.

Portée pratique et applicative

D'un point de vue applicatif, ce mémoire présente un prototype complet et reproductible, de la caméra jusqu'à l'alerte physique. Il fournit le seuil de confiance qui maximise le F1-score, une analyse du compromis entre fausses alarmes et intrusions manquées, et une décomposition mesurée du temps de traitement qui identifie les étapes à optimiser pour atteindre le temps réel.

Pertinence socio-économique et infrastructurelle au Cameroun

Au Cameroun, et plus largement en Afrique centrale, la modernisation des infrastructures (sécurisation des campus universitaires, zones industrielles, villes intelligentes) se heurte au déficit énergétique et à la fracture numérique. L'approche développée dans ce travail répond à ces défis :

— **Souveraineté des données et réduction des coûts :** le traitement local évite les abonnements à des services cloud étrangers et ne transmet aucune image hors du site.

— **Résilience :** le système reste fonctionnel en cas de coupure des liaisons Internet ou de dégradation du réseau mobile.

— **Valorisation du matériel existant :** un détecteur léger peut s'exécuter sur un ordinateur portable équipé d'un GPU d'entrée de gamme, sans serveur dédié.

Démarche méthodologique

Pour répondre à ces questions et vérifier ces hypothèses, un système est conçu, associant un poste d'inférence local et un nœud d'alerte ESP32 (Espressif Systems, s. d.). Le détecteur YOLOv8n (Jocher et al., 2023), pré-entraîné sur MS COCO (Lin et al., 2014), est affiné pendant 100 époques par descente de gradient stochastique (SGD) sur un jeu de 1415 images de caméras de surveillance réelles annotées pour la seule classe `person` (dont 1057 pour l'entraînement) (Konkuk University, 2024). Ses performances sont évaluées sur l'ensemble de validation à l'aide de la précision, du rappel, du F1-score et de la mAP (Everingham et al., 2010 ; Padilla et al., 2020). En exploitation, chaque trame est analysée sur un GPU, les cibles sont suivies d'une trame à l'autre par BoT-SORT (Aharon et al., 2022), les détections sont journalisées au format JSON, et le nombre de personnes détectées est transmis par Wi-Fi (UDP (Postel, 1980)) ou par liaison série à l'ESP32, qui pilote un écran OLED et un voyant d'alarme. Le temps de traitement de chaque étape de la chaîne est enfin mesuré sur une séquence de 5 min. Le code source, les fichiers de configuration et la documentation de ce travail sont publiés par l'auteur dans un dépôt public (Kengne Takam, 2026b). Ce travail a également été présenté par l'auteur dans deux vidéos de vulgarisation diffusées sur sa chaîne YouTube (Kengne Takam, 2026c ; Kengne Takam, 2026a).

Organisation du mémoire

Outre la présente introduction générale, ce mémoire comporte trois chapitres, suivis des références bibliographiques et des annexes. Le chapitre I présente la revue de la littérature : évolution de la vidéosurveillance, fondements de l'apprentissage profond, détecteurs d'objets de la famille YOLO, métriques d'évaluation, suivi multi-objets et intelligence en périphérie de réseau. Le chapitre II décrit le matériel et les méthodes : architecture du système, jeu de données, entraînement de YOLOv8n, chaîne d'inférence et nœud d'alerte. Le chapitre III présente et discute les résultats de détection et les performances temporelles. Une conclusion générale fait le point sur l'atteinte des objectifs, répond aux questions de recherche, en dégage les limites et formule des perspectives et des recommandations. Les annexes regroupent le code source, les schémas et le micrologiciel du nœud d'alerte, ainsi que la conception détaillée de sa carte électronique.

CHAPITRE I

REVUE DE LA LITTÉRATURE

Introduction

Ce chapitre présente les fondements scientifiques sur lesquels repose ce travail et les principaux travaux antérieurs du domaine. Il retrace d'abord l'évolution de la vidéosurveillance, puis rappelle les bases de l'apprentissage profond appliqué à la vision par ordinateur. Il passe ensuite en revue les méthodes de détection d'objets, des descripteurs conçus manuellement aux détecteurs de la famille YOLO, ainsi que les métriques qui permettent de les comparer. Il aborde enfin la détection de personnes, le suivi multi-objets, l'intelligence en périphérie de réseau et les questions de vie privée, avant de situer l'écart de recherche auquel répond ce mémoire.

I.1 Évolution des systèmes de vidéosurveillance

Les premiers systèmes de vidéosurveillance par circuit fermé (CCTV) se limitaient à enregistrer et à afficher les images de caméras analogiques; la détection des événements reposait entièrement sur des opérateurs humains. Or l'attention d'un opérateur décroît rapidement lorsqu'il doit observer simultanément de nombreux écrans, ce qui a motivé, dès les années 1990, la recherche sur la surveillance visuelle automatisée (Hu et al., 2004). Hu et al. (2004) décrivent la chaîne de traitement de ces premiers systèmes : modélisation de l'environnement, détection du mouvement, classification des objets, suivi, puis compréhension du comportement.

La détection du mouvement s'appuyait principalement sur la soustraction de fond. Le modèle de mélange de gaussiennes adaptatif de Stauffer et Grimson représente chaque pixel par plusieurs distributions gaussiennes mises à jour en ligne, ce qui permet de distinguer les objets en mouvement d'un fond lentement variable (Stauffer et Grimson, 1999). Ces méthodes sont peu coûteuses, mais elles détectent tout ce qui bouge (feuillage, ombres, variations d'éclairage) sans reconnaître la nature de l'objet, ce qui engendre de nombreuses fausses alarmes.

La revue de Elharrouss et al. (2021) montre que les systèmes récents remplacent progressivement ces étapes par des réseaux de neurones profonds, capables de reconnaître directement les personnes et les véhicules, et que les principaux défis portent désormais sur le traitement en temps réel, le volume de données à transmettre et la protection de la vie privée.

I.2 Fondements de l'apprentissage profond pour la vision

I.2.1 Réseaux de neurones convolutifs

L'apprentissage profond désigne l'apprentissage de représentations hiérarchiques par des réseaux de neurones à nombreuses couches (LeCun et al., 2015 ; Goodfellow et al., 2016). En vision par ordinateur, l'architecture de référence est le réseau de neurones convolutif (CNN), popularisé par LeNet pour la reconnaissance de caractères manuscrits (LeCun et al., 1998). Une couche de convolution applique à l'image un ensemble de filtres appris; le partage des

poids entre toutes les positions de l'image réduit fortement le nombre de paramètres et rend la détection d'un motif indépendante de sa position. Les couches de sous-échantillonnage (*pooling*) ou les convolutions à pas supérieur à un réduisent ensuite la résolution, si bien que les couches profondes extraient des descripteurs de plus en plus abstraits sur des régions de plus en plus étendues.

La victoire d'AlexNet au concours ImageNet en 2012 a marqué le basculement du domaine vers ces modèles, grâce à l'entraînement sur GPU, à la fonction d'activation ReLU et à la régularisation par *dropout* (Krizhevsky et al., 2012). Les réseaux résiduels (ResNet) ont ensuite permis d'entraîner des réseaux de plus de cent couches, grâce à des connexions qui ajoutent l'entrée d'un bloc à sa sortie et facilitent la propagation du gradient (He et al., 2016). Ces architectures servent de *backbone* (extracteur de caractéristiques) à la plupart des détecteurs d'objets modernes.

I.2.2 Entraînement et transfert d'apprentissage

Les paramètres d'un réseau sont ajustés par rétropropagation du gradient d'une fonction de perte, à l'aide d'une variante de la descente de gradient stochastique (SGD) (Goodfellow et al., 2016). La SGD avec moment reste très utilisée en détection; les méthodes adaptatives comme Adam (Kingma et Ba, 2015) et AdamW, qui découple la pénalisation des poids de la mise à jour du gradient (Loshchilov et Hutter, 2019), accélèrent souvent la convergence. Le choix de l'optimiseur et du calendrier du taux d'apprentissage fait partie des hyperparamètres étudiés au chapitre II.

Entraîner un détecteur à partir de zéro exige des centaines de milliers d'images annotées. Le transfert d'apprentissage consiste à réutiliser un modèle pré-entraîné sur un grand jeu de données, puis à l'affiner sur le jeu de données cible (Pan et Yang, 2010). C'est la stratégie retenue dans ce travail : YOLOv8n, pré-entraîné sur MS COCO (Lin et al., 2014), est affiné sur environ un millier d'images de vidéosurveillance.

I.3 Méthodes classiques de détection d'objets

Avant l'apprentissage profond, la détection reposait sur des descripteurs conçus manuellement associés à un classifieur, appliqués selon une fenêtre glissante à plusieurs échelles (Zou et al., 2023). L'algorithme de Viola et Jones (Viola et Jones, 2004) combine des caractéristiques de type Haar, calculées rapidement grâce à l'image intégrale, et une cascade de classifieurs entraînée par AdaBoost qui rejette très tôt les fenêtres sans objet. Il a permis la première détection de visages en temps réel, mais reste sensible aux variations d'éclairage, de pose et aux occlusions partielles.

Pour la détection de personnes, Dalal et Triggs (2005) ont proposé les histogrammes de gradients orientés (HOG), qui décrivent la silhouette par la distribution locale des orientations de

contours, associés à une machine à vecteurs de support linéaire. Le modèle à parties déformables (DPM) de Felzenszwalb et al. (2010) représente ensuite un objet comme un ensemble de parties dont la position peut varier autour d'une configuration de référence, ce qui améliore la robustesse aux changements de posture. Ces méthodes ont dominé les campagnes d'évaluation PASCAL VOC (Everingham et al., 2010) jusqu'en 2012, mais leurs descripteurs fixes plafonnent en précision et leur coût augmente avec le nombre d'échelles explorées.

I.4 Détecteurs d'objets fondés sur l'apprentissage profond

Les détecteurs profonds se répartissent en deux familles (Zhao et al., 2019 ; Zou et al., 2023) : les détecteurs en deux étapes, qui proposent d'abord des régions candidates puis les classent, et les détecteurs en une seule passe, qui prédisent directement les boîtes et les classes sur une grille dense.

I.4.1 Détecteurs en deux étapes

R-CNN extrait environ deux mille régions candidates par recherche sélective, puis calcule pour chacune des descripteurs CNN classés par SVM (Girshick et al., 2014) ; il a nettement amélioré la précision sur PASCAL VOC, mais au prix de plusieurs secondes par image. Fast R-CNN mutualise le calcul des descripteurs sur toute l'image et extrait ceux de chaque région par *RoI pooling* (Girshick, 2015). Faster R-CNN remplace enfin la recherche sélective par un réseau de propositions de régions (RPN) qui partage les couches convolutives avec le détecteur (Ren et al., 2017). Les réseaux pyramidaux de caractéristiques (FPN) complètent ces architectures en combinant les cartes de plusieurs résolutions, ce qui améliore la détection des petits objets (Lin et al., 2017a). Ces détecteurs restent précis, mais leur vitesse, de l'ordre de quelques à une vingtaine d'images par seconde sur GPU, se prête mal à l'exécution sur un matériel modeste.

I.4.2 Détecteurs en une seule passe

YOLO (*You Only Look Once*) formule la détection comme un problème de régression unique : l'image est divisée en une grille dont chaque cellule prédit des boîtes englobantes et des probabilités de classe, en une seule évaluation du réseau (Redmon et al., 2016). SSD prédit des boîtes à partir de boîtes de référence (ancres) sur plusieurs cartes de caractéristiques de résolutions différentes (Liu et al., 2016). RetinaNet a montré que l'écart de précision entre les deux familles provenait surtout du déséquilibre entre les très nombreux exemples de fond et les rares objets, et l'a corrigé par la perte focale (*focal loss*) (Lin et al., 2017b). EfficientDet a ensuite proposé une famille de détecteurs dont la résolution, la profondeur et la largeur sont mises à l'échelle conjointement (Tan et al., 2020). Le tableau 1 résume ces familles.

TABLE 1 – Principales familles de détecteurs d'objets (synthèse de la littérature).

Méthode	**Année**	**Principe**	**Limite principale**
Viola-Jones (Viola et Jones, 2004)	2004	Caractéristiques de Haar, cascade AdaBoost	Sensible à l'éclairage et aux occlusions
HOG + SVM (Dalal et Triggs, 2005)	2005	Gradients orientés, fenêtre glissante	Descripteurs fixes, coût multi-échelle
DPM (Felzenszwalb et al., 2010)	2010	Modèle à parties déformables	Lent, précision plafonnée
R-CNN (Girshick et al., 2014)	2014	Régions candidates + CNN + SVM	Plusieurs secondes par image
Faster R-CNN (Ren et al., 2017)	2017	Réseau de propositions (RPN) partagé	Deux étapes, peu adapté à l'embarqué
SSD (Liu et al., 2016)	2016	Une passe, ancres multi-échelles	Moins précis sur les petits objets
RetinaNet (Lin et al., 2017b)	2017	Une passe, FPN et perte focale	Tête de détection lourde
YOLOv1–v3 (Redmon et al., 2016 ; Redmon et Farhadi, 2017 ; Redmon et Farhadi, 2018)	2016–2018	Régression sur une grille, puis ancres et trois échelles	Localisation moins fine que les deux étapes
YOLOv8 (Jocher et al., 2023)	2023	Une passe, sans ancres, tête découplée	mAP@0.5:0.95 limitée sur petits objets

I.4.3 Évolution de la famille YOLO

La famille YOLO a connu de nombreuses versions, analysées en détail par Terven et al. (2023). YOLOv2 (YOLO9000) introduit les ancres, la normalisation par lots et l'entraînement multi-échelle (Redmon et Farhadi, 2017). YOLOv3 adopte le *backbone* Darknet-53 et prédit à trois échelles, à la manière d'un FPN (Redmon et Farhadi, 2018). YOLOv4 rassemble un ensemble de techniques d'entraînement (augmentation mosaïque, *backbone* CSPDarknet, agrégation PANet) pour optimiser le compromis vitesse-précision sur un seul GPU (Bochkovskiy et al., 2020). YOLOv5, publié par Ultralytics sous la forme d'un logiciel PyTorch, se distingue par sa facilité d'entraînement et de déploiement et par ses variantes de tailles croissantes (n, s, m, l, x) (Jocher, 2020). YOLOX revient à une prédiction sans ancres avec une tête découplée (Ge et al., 2021) ; YOLOv6 (Li et al., 2022) et YOLOv7 (Wang et al., 2023) optimisent l'architecture pour l'inférence industrielle. YOLOv8 (Jocher et al., 2023) reprend ces acquis dans un cadre logiciel unifié; les versions ultérieures, comme YOLOv9 (Wang et al., 2024b) et YOLOv10, qui supprime la suppression des non-maxima en post-traitement (Wang et al., 2024a), poursuivent cette évolution. YOLOv8 a été retenu dans ce travail pour la maturité de son écosystème logiciel, la disponibilité d'une variante Nano très légère et son intégration native du suivi BoT-SORT.

I.5 Architecture de YOLOv8

YOLOv8, développé par Ultralytics (Jocher et al., 2023 ; Terven et al., 2023), reprend l'approche en une seule passe avec trois éléments principaux :

— **Un *backbone* dérivé de CSPNet (Wang et al., 2020)**, construit avec des blocs C2f qui concatènent les sorties de goulots d'étranglement successifs;

— **Un *neck* de type PANet (Liu et al., 2018)**, qui fusionne les cartes de caractéristiques de plusieurs résolutions;

— **Une tête découplée et sans ancres** (*anchor-free*), qui prédit séparément la classe et la boîte, celle-ci étant exprimée par les distances du point d'ancrage aux quatre bords (Terven et al., 2023).

La boîte est apprise avec la perte CIoU (Zheng et al., 2020), qui prolonge la perte GIoU (Rezatofighi et al., 2019) en tenant compte de la distance entre les centres et du rapport d'aspect, et avec la *Distribution Focal Loss* (Li et al., 2020), qui modélise chaque bord de la boîte par une distribution discrète. La classe est apprise avec une entropie croisée binaire. La variante Nano (YOLOv8n), la plus légère, compte environ 3,2 millions de paramètres; ses caractéristiques sont détaillées au chapitre II.

I.6 Métriques d'évaluation des détecteurs

La qualité d'une détection se mesure d'abord par l'intersection sur l'union (IoU) entre la boîte prédite B_p et la boîte de vérité terrain B_g (figure 1) :

$$\text{IoU} = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \tag{I.1}$$

Une prédiction est comptée comme vrai positif lorsque son IoU avec une boîte de vérité terrain dépasse un seuil, en général 0,5 (Everingham et al., 2010). On en déduit la précision (proportion des détections correctes) et le rappel (proportion des personnes réellement détectées), puis la précision moyenne (AP), aire sous la courbe précision-rappel (Padilla et al., 2020). La campagne PASCAL VOC utilise l'AP à IoU = 0,5 (Everingham et al., 2010) ; le jeu MS COCO moyenne l'AP sur dix seuils d'IoU de 0,5 à 0,95, ce qui récompense une localisation précise (Lin et al., 2014). Ces deux indicateurs, notés mAP@0.5 et mAP@0.5:0.95, sont utilisés au chapitre III. Padilla et al. (2020) soulignent que les implémentations de ces métriques diffèrent d'un outil à l'autre (interpolation de la courbe, traitement des doublons), ce qui impose de préciser l'outil d'évaluation employé; dans ce travail, toutes les métriques sont calculées par Ultralytics.

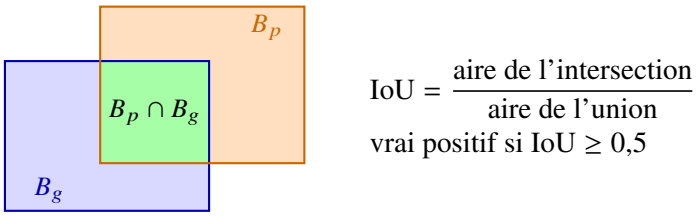


FIGURE 1 – Intersection sur l'union entre une boîte prédite B_p et une boîte de vérité terrain B_g.

I.7 Détection de personnes et de petites cibles

La détection de personnes est l'un des problèmes les plus étudiés de la vision par ordinateur. L'évaluation de Dollár et al. (2012) sur le jeu de données Caltech a montré que les performances chutent fortement pour les piétons de faible hauteur et pour les personnes partiellement occultées, deux situations fréquentes en vidéosurveillance. Le jeu de données CityPersons, constitué d'images urbaines plus variées, confirme que la diversité des scènes d'entraînement améliore la généralisation des détecteurs (Zhang et al., 2017). Ces travaux justifient l'affinage du détecteur sur des images de caméras de surveillance réelles plutôt que sur des images généralistes.

Dans les scènes de surveillance, les personnes éloignées n'occupent que quelques dizaines de pixels. Le jeu de données MS COCO (Lin et al., 2014) qualifie de « petits » les objets d'aire inférieure à 32×32 pixels; ce sont ceux pour lesquels les détecteurs obtiennent les performances les plus faibles. Le pas minimal de la tête de YOLOv8 étant de 8 pixels, une personne de 16×16 pixels n'est couverte que par 2×2 cellules de la carte de caractéristiques la plus fine. Les

architectures pyramidales (Lin et al., 2017a ; Liu et al., 2018) atténuent cette difficulté sans la supprimer.

I.8 Suivi multi-objets

Un détecteur traite chaque trame indépendamment. Pour compter des personnes, filtrer les détections isolées ou mesurer la durée de présence d'un intrus, il faut associer les détections d'une trame à l'autre, c'est-à-dire réaliser un suivi multi-objets selon le paradigme du « suivi par détection ». SORT prédit la position de chaque cible par un filtre de Kalman (Kalman, 1960) et associe prédictions et détections par l'algorithme hongrois sur un critère d'IoU (Bewley et al., 2016). DeepSORT ajoute un descripteur d'apparence appris, qui réduit les changements d'identité après une occlusion (Wojke et al., 2017). ByteTrack associe aussi les détections de faible confiance, souvent dues à une occlusion partielle, au lieu de les écarter (Zhang et al., 2022). BoT-SORT combine ces idées avec une compensation du mouvement de la caméra et un filtre de Kalman amélioré (Aharon et al., 2022) ; c'est le traqueur retenu dans ce travail, car il est intégré à Ultralytics.

I.9 Intelligence en périphérie de réseau (*Edge AI*)

I.9.1 Du *cloud* à la périphérie

Les approches fondées sur le *Cloud Computing* ont permis d'exploiter des CNN profonds, au prix d'une transmission continue de flux vidéo à haut débit. Le calcul en périphérie de réseau (*edge computing*) rapproche le traitement de la source des données afin de réduire la latence, le trafic réseau et l'exposition des données (Shi et al., 2016 ; Satyanarayanan, 2017). Satyanarayanan cite précisément l'analyse vidéo comme application type : une caméra produit un débit trop élevé pour être envoyé en continu vers un centre de données, alors qu'un traitement local ne transmet que le résultat (Satyanarayanan, 2017). L'intelligence en périphérie (*edge intelligence*) étend cette idée à l'inférence des réseaux de neurones (Zhou et al., 2019 ; Chen et Ran, 2019) : pour la vidéosurveillance, elle permet de ne transmettre que des métadonnées (présence, position, horodatage) au lieu du flux vidéo.

I.9.2 Modèles légers et compression

L'exécution de réseaux profonds sur un matériel modeste repose sur des architectures légères et des techniques de compression (Sze et al., 2017). Les MobileNets remplacent les convolutions classiques par des convolutions séparables en profondeur, qui divisent le coût de calcul par un facteur proche de huit à neuf pour un noyau 3×3 (Howard et al., 2017) ; MobileNetV2 y ajoute des blocs résiduels inversés (Sandler et al., 2018). La compression profonde combine élagage,

quantification et codage de Huffman (Han et al., 2016). La quantification en entiers sur 8 bits (INT8) divise par quatre la taille des poids et permet une inférence en arithmétique entière (Jacob et al., 2018 ; Krishnamoorthi, 2018). Les variantes Nano des détecteurs YOLO, dont YOLOv8n, réduisent quant à elles la profondeur et la largeur du réseau (Jocher et al., 2023).

I.9.3 Microcontrôleurs et nœuds d'alerte

À l'extrémité de la chaîne, les microcontrôleurs à faible coût permettent de matérialiser une alerte (voyant, sirène, afficheur) et de communiquer sans fil. L'ESP32 intègre deux cœurs de calcul, des liaisons Wi-Fi et Bluetooth et de nombreux périphériques (Espressif Systems, s. d.) ; son environnement Arduino s'appuie sur le noyau temps réel FreeRTOS (Barry, 2016). Le domaine du *TinyML* cherche à exécuter des modèles directement sur de tels microcontrôleurs (Warden et Situnayake, 2019), mais leur mémoire de quelques centaines de kilo-octets reste insuffisante pour un détecteur d'objets précis à la résolution de 640×640 pixels. C'est pourquoi ce travail répartit les rôles : l'inférence s'exécute sur un poste local à GPU, et l'ESP32 joue le rôle de nœud d'alerte qui reçoit uniquement le nombre de personnes détectées par le protocole UDP (Postel, 1980).

I.10 Vie privée et éthique de la vidéosurveillance

La centralisation des flux vidéo soulève des questions de vie privée : la collecte massive d'images de personnes peut être détournée à des fins de profilage commercial ou de contrôle (Zuboff, 2019). Senior et al. (2005) ont montré que la vision par ordinateur peut elle-même protéger la vie privée, en ne conservant que les informations utiles à la surveillance (positions, trajectoires) et en masquant l'identité des personnes. Le traitement en périphérie va dans ce sens, puisqu'aucune image ne quitte le site (Shi et al., 2016). Il ne dispense pas pour autant d'une politique de conservation et d'accès aux vidéos enregistrées localement, discutée au chapitre II.

I.11 Synthèse et identification de l'écart de recherche (*Research Gap*)

La littérature montre trois évolutions convergentes : le passage des descripteurs conçus manuellement aux détecteurs profonds en une seule passe (Zou et al., 2023 ; Terven et al., 2023), l'allègement de ces détecteurs pour une exécution locale (Sze et al., 2017 ; Howard et al., 2017), et le déplacement du traitement du *cloud* vers la périphérie du réseau (Shi et al., 2016 ; Zhou et al., 2019 ; Chen et Ran, 2019). Les travaux d'intelligence en périphérie évaluent cependant souvent les détecteurs sur des jeux de données généralistes et sur des plateformes embarquées dédiées (Zhou et al., 2019), et s'arrêtent au modèle. Les performances d'un détecteur léger affiné

sur des images de surveillance réelles, puis intégré dans une chaîne complète allant de la caméra jusqu'à une alerte physique, restent moins documentées, en particulier la répartition du temps de traitement entre le modèle et le reste de la chaîne (suivi, journalisation, affichage, transmission de l'alerte).

Ce travail se positionne dans cet écart : évaluer YOLOv8n affiné sur un jeu de données CCTV (Konkuk University, 2024), puis mesurer les performances de la chaîne complète associant un poste d'inférence local et un nœud d'alerte ESP32.

Conclusion

Ce chapitre a retracé l'évolution de la vision par ordinateur appliquée à la surveillance, des descripteurs conçus manuellement aux détecteurs profonds de la famille YOLO, et rappelé les métriques qui permettent de les évaluer. Il a présenté les méthodes de suivi multi-objets, les techniques qui rendent possible l'inférence en périphérie de réseau, et les enjeux de vie privée. Il a enfin mis en évidence la nécessité de solutions légères et résilientes face aux contraintes d'infrastructure. Ces bases justifient les choix détaillés au chapitre suivant : architecture du système, jeu de données, entraînement de YOLOv8n et chaîne d'alerte.

CHAPITRE II

MATÉRIEL ET MÉTHODES

Introduction

Ce chapitre décrit le matériel et les méthodes mis en œuvre pour atteindre les objectifs fixés dans l'introduction générale. Il présente d'abord la conception globale du système et les outils retenus, en particulier le détecteur YOLOv8 Nano et la justification de l'approche *Edge AI*. Il décrit ensuite le jeu de données et sa répartition, le processus d'apprentissage et ses hyperparamètres, puis les indicateurs utilisés pour évaluer le modèle. Il détaille enfin la chaîne d'inférence et d'alerte, l'alimentation et les interfaces du nœud d'alerte ESP32, ainsi que les précautions prises pour la confidentialité des données vidéo.

II.1 Conception globale du système

Le système de vidéosurveillance développé dans ce travail transforme un flux vidéo en une information exploitable : la présence de personnes dans la zone surveillée, suivie d'une alerte physique. La figure 2 présente son architecture.

Une caméra USB fournit des trames de 1280×720 pixels à un poste d'inférence local équipé d'un GPU NVIDIA RTX A500. Sur ce poste, chaque trame est analysée par YOLOv8n, les détections sont associées d'une trame à l'autre par le suivi BoT-SORT (Aharon et al., 2022) intégré à Ultralytics, puis :

— chaque détection est écrite dans un journal local au format JSON Lines (horodatage, identifiant de piste, confiance, boîte en pixels);

— la trame annotée est affichée et enregistrée dans un fichier vidéo;

— le nombre de personnes détectées n est envoyé au nœud d'alerte.

Le nœud d'alerte est une carte ESP32 (Espressif Systems, s. d.) qui affiche l'état de la zone sur un écran OLED SSD1306 et allume un voyant lorsque $n > 0$. Deux liaisons ont été réalisées : une liaison série USB à 115 200 bauds et une liaison Wi-Fi par datagrammes UDP (Postel, 1980) sur le port 4210. Seul l'entier n (en texte ASCII) transite vers l'ESP32; aucune image ne quitte le poste local.

II.2 Outils et architecture algorithmique

II.2.1 Modèle YOLOv8 Nano

Ce travail utilise la variante Nano de YOLOv8 (Jocher et al., 2023), la plus légère de la famille : environ 3,2 millions de paramètres et 8,7 GFLOP par image de 640×640 pixels (nombre d'opérations par image, et non débit par seconde). Son architecture, décrite au chapitre I, se compose de trois parties :

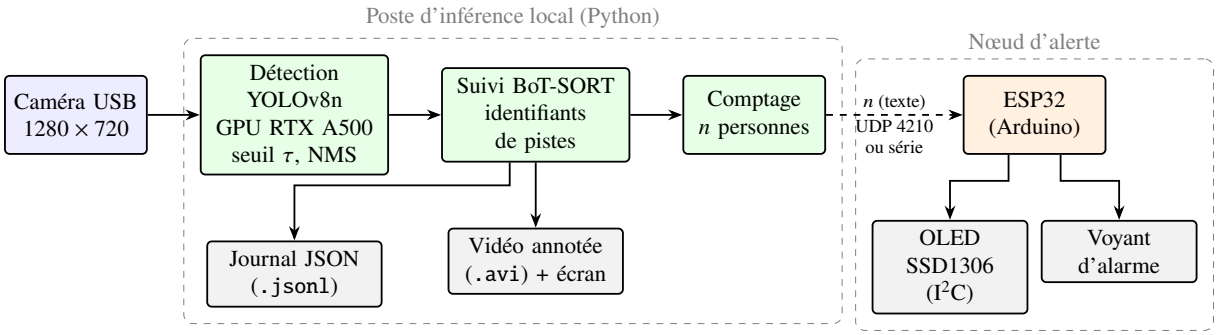


FIGURE 2 – Architecture du système. Le poste local exécute la détection, le suivi et la journalisation; seul le nombre n de personnes détectées est transmis au nœud d'alerte ESP32, par Wi-Fi (UDP) ou par liaison série.

— **Le *backbone* (dorsale) :** dérivé de CSPNet (Wang et al., 2020), il est construit avec des blocs C2f qui concatènent les sorties de goulots d'étranglement successifs :

$$Y = \mathrm{Concat}\big(F_1(X), F_2(F_1(X)), \ldots, F_n(F_{n-1}(X))\big), \tag{II.1}$$

ce qui préserve le flux de gradients tout en limitant le nombre de paramètres.

— **Le *neck* (cou) :** de type PANet (Liu et al., 2018), il fusionne les caractéristiques de plusieurs résolutions, afin d'associer l'information sémantique des couches profondes aux repères géométriques précis des premières couches.

— **La tête (*head*) :** découplée et sans ancres, elle sépare les branches de classification (classe `person`) et de régression de la boîte englobante, exprimée par les distances du point d'ancrage aux quatre bords (Terven et al., 2023).

Le modèle est initialisé avec les poids pré-entraînés sur MS COCO (Lin et al., 2014) (`yolov8n.pt`) puis affiné sur une classe unique, `person`.

II.2.2 Justification de l'approche Edge AI et contraintes matérielles

Le déploiement de modèles d'apprentissage profond dans des infrastructures de surveillance locales au Cameroun se heurte aux limites de bande passante et à la latence des liaisons vers des serveurs distants (*Cloud Computing*). Le calcul en périphérie de réseau (Shi et al., 2016 ; Zhou et al., 2019) relocalise l'inférence au plus près du capteur. Le choix de la variante Nano répond aux contraintes du matériel disponible : un ordinateur portable équipé d'un GPU NVIDIA RTX A500 de 4 Go de mémoire vidéo (VRAM).

Déclaration d'utilisation d'outils d'intelligence artificielle

Conformément à la note n° 26-00015/CR/UEb/ED du 19 février 2026 de l'École Doctorale de l'Université d'Ebolowa, il est déclaré que des assistants d'intelligence artificielle générative

(Claude, Anthropic) (Anthropic, 2026) ont été utilisés pour la mise en forme LaTeX du document, la relecture et la reformulation de textes en français et en anglais, la traduction du résumé, ainsi que pour l'aide à la relecture et au débogage du code. Le contenu scientifique, les expériences, les mesures, les données et les conclusions sont ceux de l'auteur, qui les a vérifiés. Aucun texte produit par ces outils n'a été intégré au mémoire sans avoir été relu et validé par l'auteur.

II.3 Collecte et structuration des données

II.3.1 Présentation du jeu de données

Le jeu de données *CCTV Person Detection* (version 3, licence CC BY 4.0) de l'Université de Konkuk (Konkuk University, 2024) contient des images issues de caméras de surveillance réelles. Contrairement aux bases généralistes comme MS COCO (Lin et al., 2014), il reflète les conditions d'un environnement de surveillance : vues plongeantes, objectifs grand angle, occlusions partielles et éclairages variés. Le modèle est configuré pour une classe unique ($nc = 1$), `person`.

II.3.2 Répartition des données (entraînement, validation, test)

Le jeu de données est divisé en trois sous-ensembles indépendants, déclarés dans le fichier `data.yaml` (tableau 2) :

— **l'ensemble d'entraînement**, utilisé pour ajuster les poids par descente de gradient;

— **l'ensemble de validation**, utilisé à la fin de chaque époque pour mesurer les performances et sélectionner le meilleur modèle;

— **l'ensemble de test**, réservé à une évaluation finale indépendante.

TABLE 2 – Répartition du jeu de données *CCTV Person Detection*

Sous-ensemble	**Images**	**Part**
Entraînement	1057	74,7 %
Validation	213	15,1 %
Test	145	10,2 %
Total	1415	100 %

Chaque personne est annotée au format YOLO, par le vecteur normalisé :

$$V_{\text{label}} = \left[c,\ \frac{x_c}{W},\ \frac{y_c}{H},\ \frac{w}{W},\ \frac{h}{H}\right] \tag{II.2}$$

où $c = 0$ est l'indice de la classe `person`, (x_c, y_c) le centre de la boîte, w et h sa largeur et sa hauteur, et W, H les dimensions de l'image.

II.4 Étapes de traitement et métriques d'évaluation

II.4.1 Processus d'apprentissage et hyperparamètres

La figure 3 résume la chaîne d'entraînement et le tableau 3 ses hyperparamètres, relevés dans le fichier `args.yaml` produit par Ultralytics. L'optimisation utilise la descente de gradient stochastique (*Stochastic Gradient Descent*, SGD) avec moment, sur 100 époques. La fonction de perte totale combine trois termes, pondérés par les gains par défaut d'Ultralytics :

$$\mathcal{L} = \lambda_{\text{box}}\,\mathcal{L}_{\text{CIoU}} + \lambda_{\text{cls}}\,\mathcal{L}_{\text{BCE}} + \lambda_{\text{dfl}}\,\mathcal{L}_{\text{DFL}}, \qquad (\lambda_{\text{box}}, \lambda_{\text{cls}}, \lambda_{\text{dfl}}) = (7{,}5;\ 0{,}5;\ 1{,}5) \tag{II.3}$$

où $\mathcal{L}_{\text{CIoU}}$ est la perte *Complete IoU* (Zheng et al., 2020), $\mathcal{L}_{\text{BCE}}$ l'entropie croisée binaire de classification et $\mathcal{L}_{\text{DFL}}$ la *Distribution Focal Loss* (Li et al., 2020).

Après chaque époque, le modèle est évalué sur l'ensemble de validation. La patience d'arrêt précoce étant fixée à 100 époques, l'entraînement s'est déroulé sur les 100 époques; Ultralytics conserve les poids de l'époque ayant le meilleur score de validation (`best.pt`, époque 74), qui sont ensuite utilisés pour l'inférence.

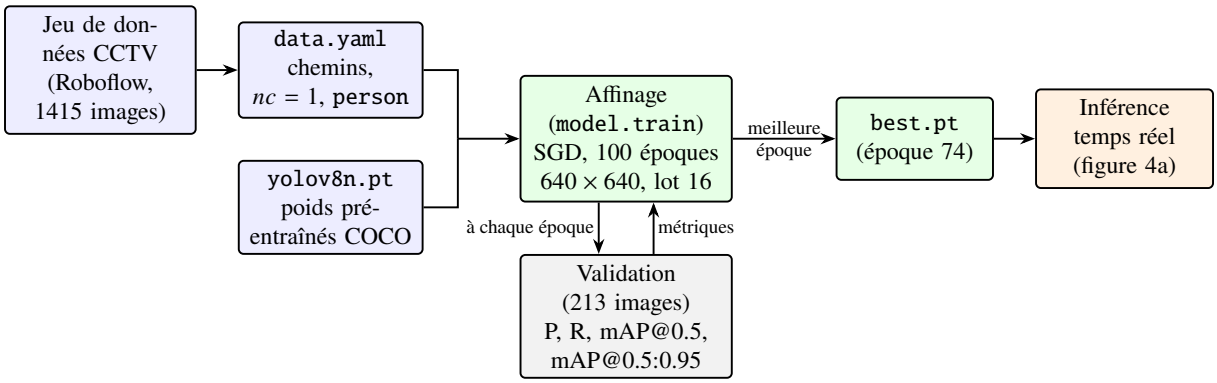


FIGURE 3 – Chaîne d'entraînement du détecteur : le fichier `data.yaml` et les poids pré-entraînés alimentent l'affinage; le modèle est validé à chaque époque et les poids de la meilleure époque sont conservés pour l'inférence.

II.4.2 Indicateurs de performance (précision, rappel, F1-score, mAP)

Soient VP, FP et FN les nombres de vrais positifs, faux positifs et faux négatifs, une détection étant un vrai positif si son IoU avec une boîte de vérité terrain dépasse 0,5.

La **précision** (P) mesure la capacité du système à éviter les fausses alarmes :

$$P = \frac{VP}{VP + FP} \tag{II.4}$$

TABLE 3 – Hyperparamètres d'entraînement (fichier `args.yaml`)

Paramètre	**Valeur**
Modèle initial	`yolov8n.pt` (MS COCO)
Époques / patience	100 / 100
Taille d'image / taille de lot	640×640 / 16
Optimiseur	SGD, moment 0,937
Taux d'apprentissage initial / final	0,01 / $0{,}01 \times 0{,}01$
Décroissance des poids	0,0005
Échauffement	3 époques
Gains de perte (box, cls, dfl)	7,5, 0,5, 1,5
Augmentation mosaïque	oui, désactivée les 10 dernières époques
Précision mixte (AMP)	oui
GPU	NVIDIA RTX A500

Le **rappel** (R) mesure sa capacité à ne manquer aucune intrusion :

$$R = \frac{VP}{VP + FN} \tag{II.5}$$

Le **F1-score** est la moyenne harmonique de ces deux indicateurs :

$$F_1 = \frac{2PR}{P + R} \tag{II.6}$$

La précision moyenne (*Average Precision*, AP) d'une classe est l'aire sous sa courbe précision-rappel, et la mAP sa moyenne sur les N classes :

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^{N} \int_0^1 P_i(R)\, dR \tag{II.7}$$

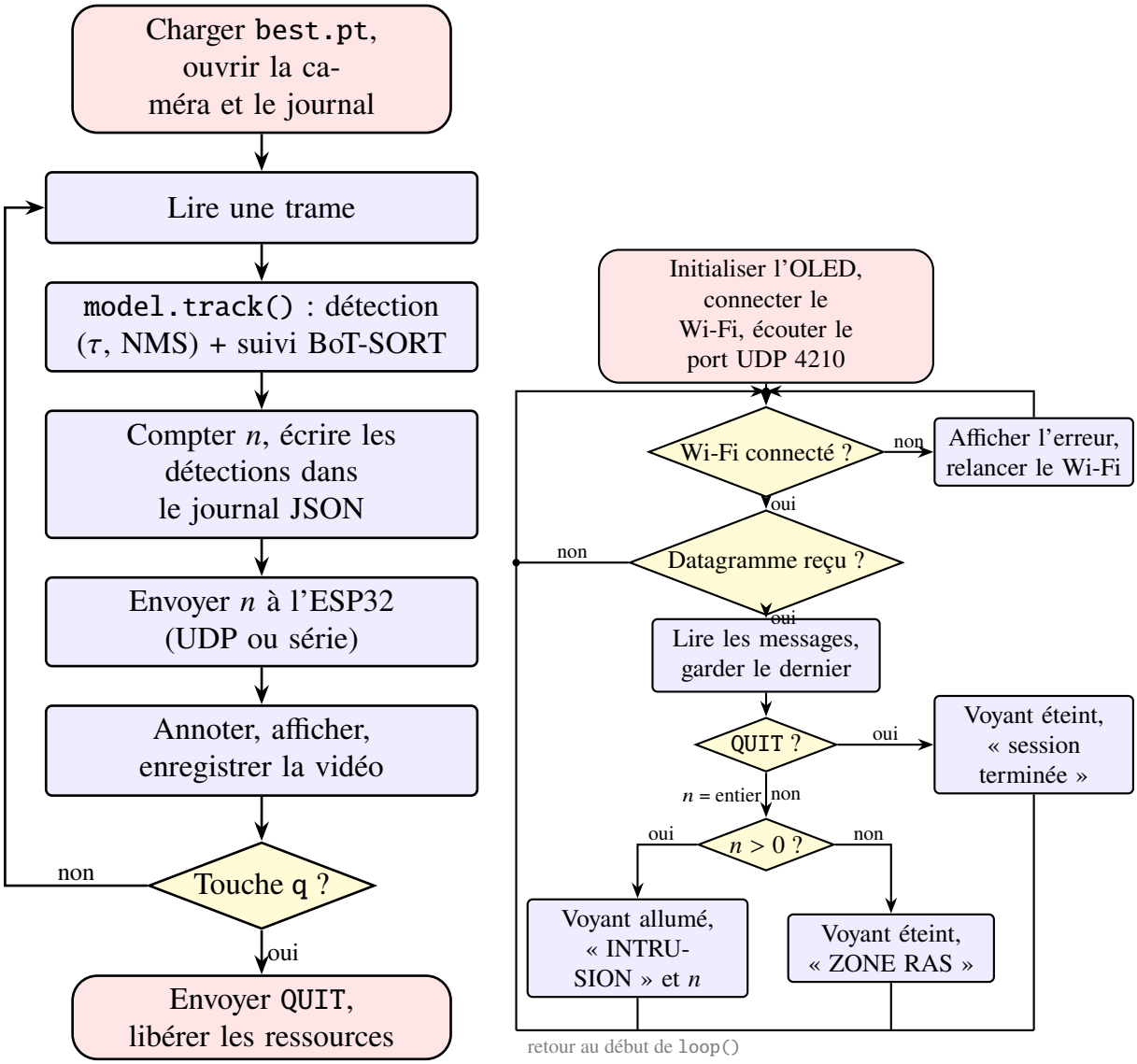
avec $N = 1$ ici. Sont rapportées la mAP@0.5 (seuil d'IoU de 0,5) et la mAP@0.5:0.95 (moyenne sur les seuils d'IoU de 0,5 à 0,95), plus exigeante sur la précision de localisation (Lin et al., 2014). Toutes les métriques sont calculées sur l'ensemble de validation.

II.5 Inférence et alerte

La boucle d'inférence (figure 4a) est écrite en Python avec OpenCV (OpenCV Team, 2024) et Ultralytics (code en annexe A et dans le dépôt public de l'auteur : Kengne Takam, 2026). Pour chaque trame, la fonction `model.track()` applique la détection puis le suivi; les détections de confiance inférieure au seuil τ sont rejetées et les boîtes redondantes sont supprimées par NMS. Deux réglages ont été utilisés : $\tau = 0{,}35$ (IoU de NMS 0,50) pour la liaison série, et $\tau = 0{,}55$ (IoU 0,45) pour la liaison Wi-Fi, ce seuil plus élevé ayant été adopté pour supprimer de fausses détections provoquées par un ventilateur dans le champ de la caméra. À l'arrêt (touche `q`), le

script envoie le message `QUIT` à l'ESP32.

Le micrologiciel de l'ESP32 (figure 4b, code en annexe B et dans le dépôt public de l'auteur : Kengne Takam, 2026) se connecte au réseau Wi-Fi puis, à chaque passage dans sa boucle principale, vérifie la connexion, lit les datagrammes UDP reçus sur le port 4210 en ne gardant que le plus récent, le convertit en entier n et met à jour l'écran et le voyant; en cas de perte du Wi-Fi, il affiche un message et relance la connexion. Dans sa seconde version, il est organisé en machine à états, chaque état ayant son propre écran, en partie animé (figure 14).



(a) Poste d'inférence (Python). (b) Nœud d'alerte ESP32 (fonction `loop()`).

FIGURE 4 – Organigrammes du logiciel : (a) boucle d'inférence sur le poste local; (b) boucle principale du micrologiciel de l'ESP32, exécutée en continu après l'initialisation.

II.6 Alimentation et interfaces du nœud d'alerte

Le nœud d'alerte est une carte de développement ESP32-WROOM-32 munie d'un connecteur USB-C, d'un convertisseur USB-UART CP2102 et d'un régulateur linéaire AMS1117-3.3 (Advanced Monolithic Systems, s. d.). La figure 5 présente sa chaîne d'alimentation et ses interfaces. La tension de 5 V du bus USB (port du poste d'inférence ou bloc secteur 5 V/1 A) alimente le rail 5 V de la carte (broche V5). Le régulateur AMS1117 en dérive le rail 3,3 V qui alimente le module ESP32 et, par la broche 3V3, l'écran OLED. Le CP2102 relie les lignes de données USB à l'UART0 de l'ESP32 (liaison série et programmation); ses signaux DTR et RTS commandent, par deux transistors, les broches EN et IO0 pour le redémarrage automatique en mode programmation.

Le régulateur étant linéaire, il dissipe

$$P_{\mathrm{LDO}} = (V_{\mathrm{in}} - V_{\mathrm{out}})\, I = (5\,\mathrm{V} - 3{,}3\,\mathrm{V})\, I = 1{,}7\,\mathrm{V} \times I, \tag{II.8}$$

soit environ 0,43 W pour un courant de 250 mA, ordre de grandeur des pointes de consommation de l'ESP32 en émission Wi-Fi (Espressif Systems, s. d.). La tension de déchet de l'AMS1117 (environ 1,1 V à pleine charge (Advanced Monolithic Systems, s. d.)) impose une tension d'entrée d'au moins 4,4 V environ : une alimentation directe par une batterie Li-ion de 3,7 V sur la broche V5 ne conviendrait donc pas. C'est pourquoi la carte dédiée conçue pour un fonctionnement sur batterie (annexe C) intercale un convertisseur élévateur MT3608 entre la batterie et l'AMS1117. Cette carte à quatre couches de 58 × 50 mm a été conçue et vérifiée, mais n'a pas encore été fabriquée : les essais du chapitre III ont été menés avec la carte de développement.

L'écran OLED est relié au bus I²C (adresse `0x3C`), sur les broches par défaut de la bibliothèque `Wire` (SDA : GPIO21, SCL : GPIO22). Le voyant est commandé par GPIO1, qui est aussi la sortie TX0 de l'UART0 utilisée par le CP2102 : en liaison série, le voyant et la communication se perturbent mutuellement, et une autre broche (par exemple GPIO2) doit être choisie.

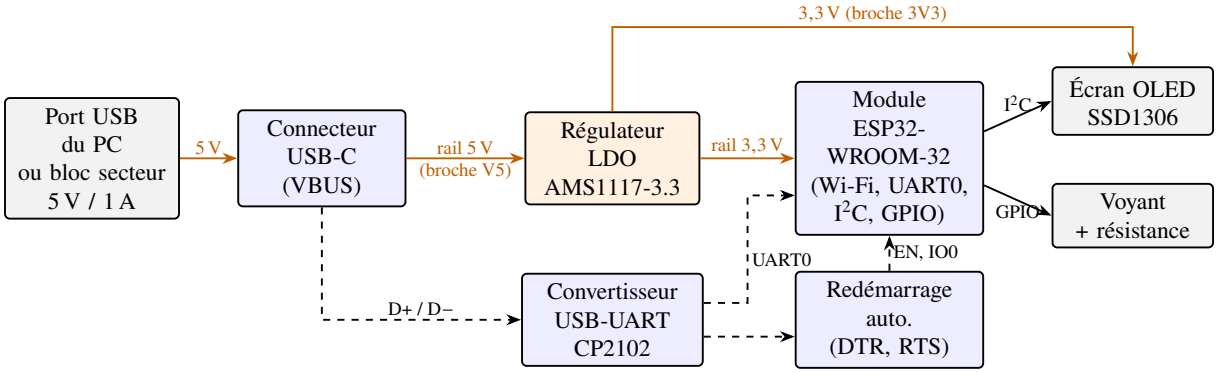


FIGURE 5 – Alimentation et interfaces du nœud d'alerte ESP32 : le 5 V du connecteur USB-C est abaissé à 3,3 V par le régulateur AMS1117, qui alimente l'ESP32 et l'écran OLED (traits pleins orange). Les traits pointillés représentent les données : liaison USB-série par le CP2102 et redémarrage automatique pour la programmation.

II.7 Techniques éthiques et confidentialité des données vidéo

L'application de l'apprentissage automatique à des flux vidéo de sécurité soulève des questions juridiques et déontologiques (Zuboff, 2019). Le système détecte la silhouette générique `person` et n'effectue aucune reconnaissance faciale ni identification biométrique. Le traitement étant local, aucun flux vidéo n'est transmis à un serveur distant : seul le nombre de personnes est envoyé à l'ESP32.

En revanche, le prototype enregistre localement la vidéo annotée et un journal des boîtes détectées. Un déploiement réel devra donc définir une durée de conservation, un accès restreint à ces fichiers et, si nécessaire, un floutage des personnes avant archivage, dans le respect de la vie privée.

Conclusion

Ce chapitre a présenté la démarche suivie, de la caméra jusqu'à l'alerte physique. Le détecteur léger YOLOv8n, pré-entraîné sur MS COCO, est affiné sur un jeu de 1415 images de caméras de surveillance réelles, puis évalué par la précision, le rappel, le F1-score et la mAP. En exploitation, il est associé au suivi BoT-SORT, à la journalisation JSON et à la transmission du nombre de personnes détectées à un nœud d'alerte ESP32 par Wi-Fi (UDP) ou par liaison série. Le chapitre suivant présente et discute les résultats obtenus avec ce matériel et ces méthodes.

CHAPITRE III

RÉSULTATS ET DISCUSSION

Introduction

Ce chapitre présente et discute les résultats obtenus avec le matériel et les méthodes décrits au chapitre II. Il examine successivement la convergence de l'entraînement et les performances de détection, l'interprétation de ces performances (matrices de confusion et choix du seuil de confiance), les performances temporelles du modèle seul et du système complet, puis la validation expérimentale du nœud d'alerte ESP32. Il se termine par les implications opérationnelles et les limites du système.

Les résultats proviennent du fichier `results.csv` de l'entraînement et des graphiques produits par Ultralytics lors de la validation du meilleur modèle (dossier `runs/detect/run_-production_yolov8n`).

III.1 Présentation des résultats d'entraînement et d'inférence

III.1.1 Analyse de la convergence et des fonctions de perte

L'entraînement de 100 époques a duré 2520 s (0,70 h) sur le GPU RTX A500. La figure 6 montre l'évolution des pertes et des métriques. Les trois pertes d'entraînement (boîte, classification et DFL) décroissent continûment; en fin d'entraînement, la perte de boîte passe de 1,29 (époque 74) à 1,12 (époque 100). Les pertes de validation se stabilisent (perte de boîte 1,52 à l'époque 74 et 1,54 à l'époque 100), tandis que la mAP@0.5 plafonne autour de 0,93.

Le meilleur score de validation est atteint à l'époque 74. Au-delà, l'écart croissant entre les pertes d'entraînement et de validation indique le début d'un surapprentissage léger, sans dégradation notable des métriques. La sélection automatique des poids de l'époque 74 (`best.pt`) en protège le modèle déployé.

III.1.2 Performances de détection

Le tableau 4 compare le meilleur modèle (époque 74) au modèle de la dernière époque. Le meilleur modèle atteint une précision de 91,9 %, un rappel de 89,9 % et une mAP@0.5 de 93,4 % sur les 1144 instances de l'ensemble de validation. La mAP@0.5:0.95, de 50,8 %, est nettement plus faible : les personnes sont bien détectées, mais la position exacte des boîtes reste imprécise, notamment pour les cibles petites ou partiellement masquées.

TABLE 4 – Métriques de validation (213 images, 1144 instances)

Modèle	*P*	*R*	**mAP@0.5**	**mAP@0.5:0.95**
Meilleur (époque 74)	0,919	0,899	0,934	0,508
Dernier (époque 100)	0,917	0,893	0,932	0,507

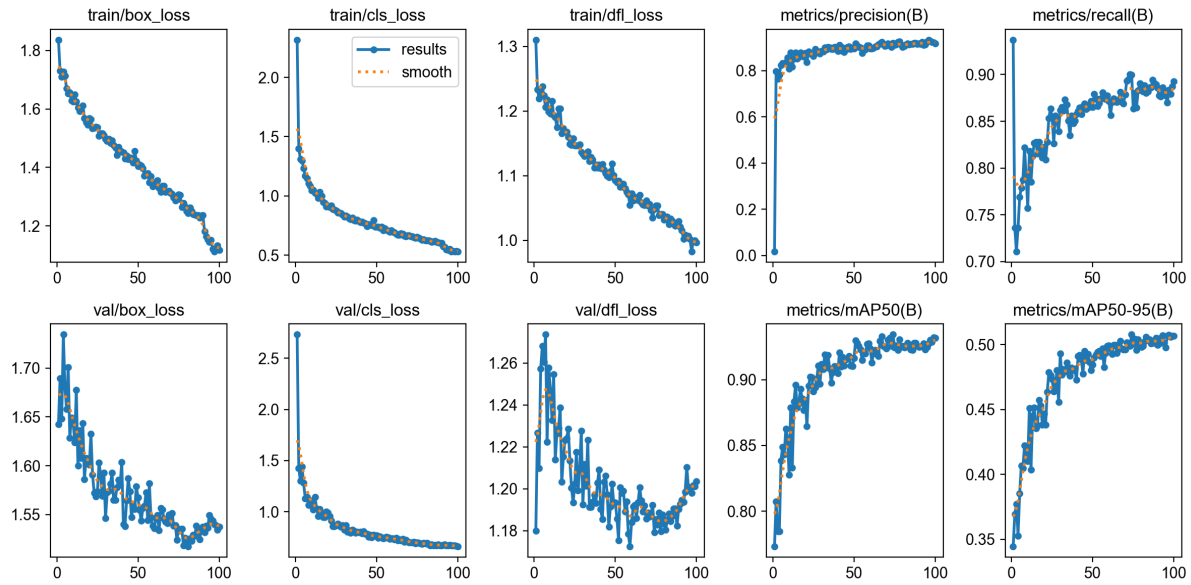


FIGURE 6 – Évolution des pertes d'entraînement et de validation (boîte, classification, DFL) et des métriques de validation au cours des 100 époques.

III.1.3 Validation visuelle des inférences

La figure 7 confronte les annotations d'un lot de validation (vérité terrain) aux prédictions du modèle. Les personnes sont localisées dans des postures variées (debout, assises, de dos) et en cas de recouvrement partiel.

III.2 Interprétation des performances du modèle YOLOv8n

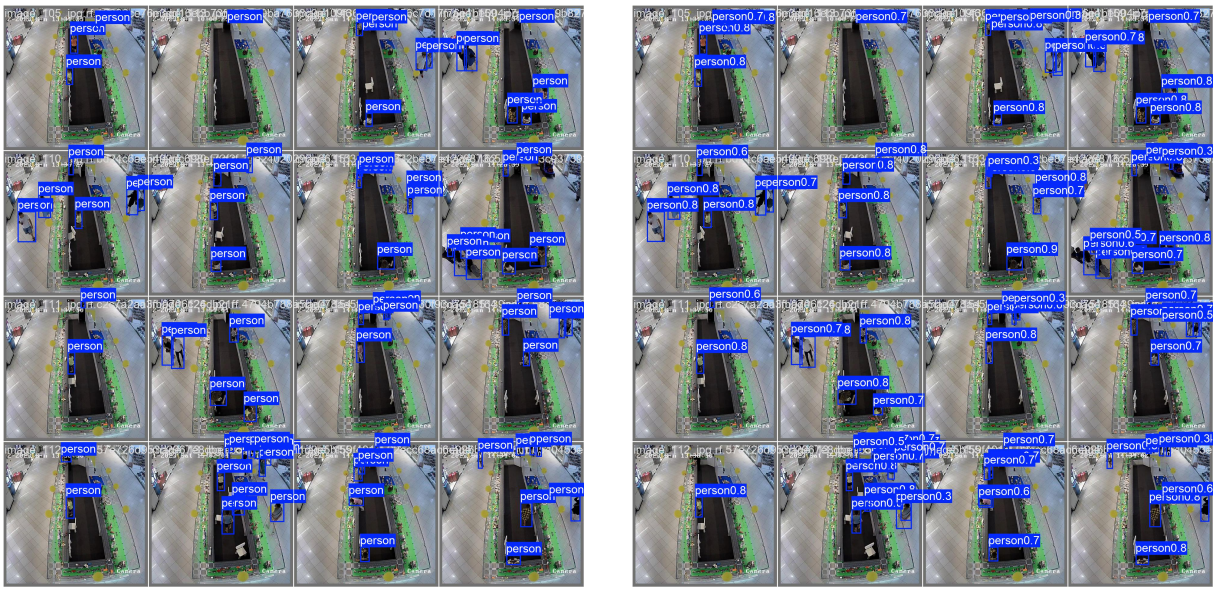
III.2.1 Analyse des matrices de confusion

La figure 8 présente la matrice de confusion brute et normalisée. Pour un détecteur à classe unique, la matrice oppose la classe `person` à l'arrière-plan : les personnes manquées apparaissent comme des faux négatifs (personne prédite comme arrière-plan) et les fausses alarmes comme des faux positifs (arrière-plan prédit comme personne).

Au seuil de confiance de 0,25 utilisé par Ultralytics pour construire la matrice, le décompte donne $VP = 1071$, $FN = 73$ et $FP = 188$, soit un rappel de 0,936 et une précision de 0,851. À ce seuil bas, le modèle manque peu de personnes, mais produit davantage de fausses alarmes que le nombre d'intrusions manquées; relever le seuil (section III.2.2) inverse ce compromis.

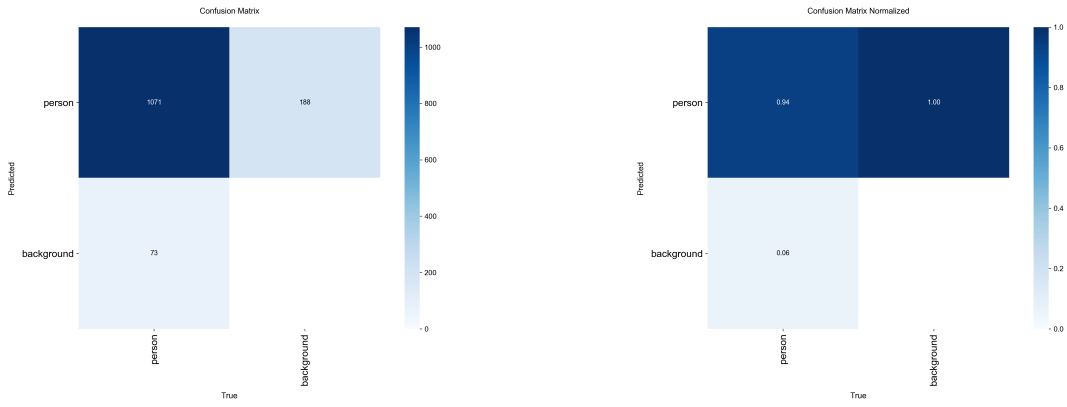
III.2.2 Choix du seuil de confiance

La figure 9 présente les courbes précision-confiance, rappel-confiance, F1-confiance et précision-rappel. La précision atteint 1,00 pour $\tau \geq 0{,}871$, au prix d'un rappel fortement réduit. Le F1-score atteint son maximum, 0,91, pour $\tau = 0{,}459$: c'est le meilleur compromis



(a) Vérité terrain. (b) Prédictions de YOLOv8n.

FIGURE 7 – Lot de validation : (a) annotations de référence; (b) boîtes prédites par le modèle.



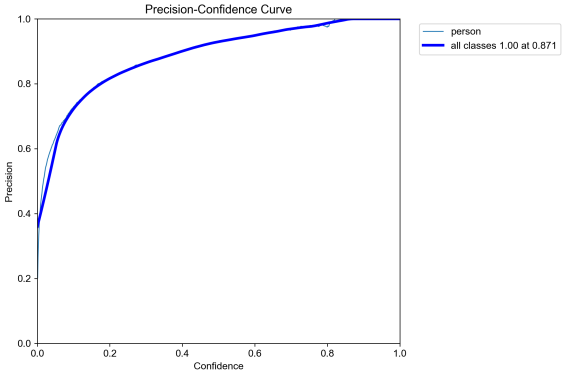
(a) Effectifs. (b) Normalisée.

FIGURE 8 – Matrices de confusion pour la classe `person` : (a) effectifs; (b) version normalisée.

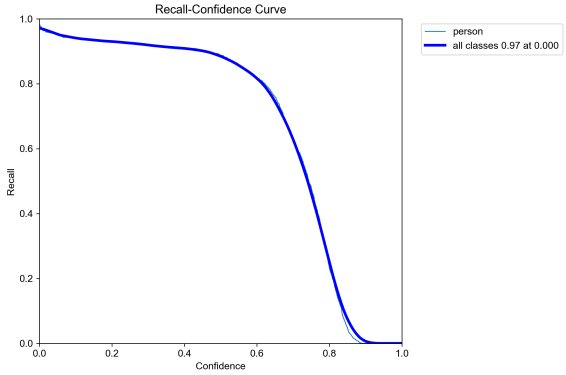
entre fausses alarmes et intrusions manquées sur l'ensemble de validation. L'aire sous la courbe précision-rappel donne la mAP@0.5 :

$$\text{mAP@0.5} = \int_0^1 P(R)\, dR \approx 0{,}934 \tag{III.1}$$

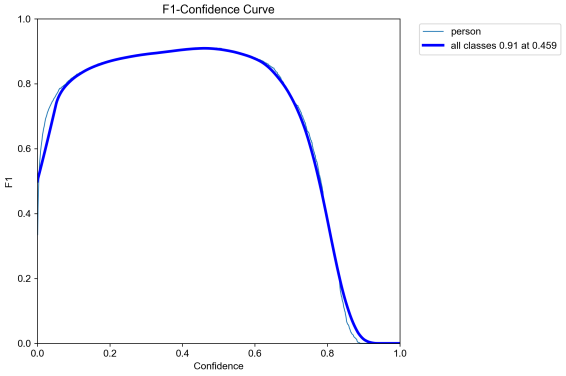
Les deux seuils utilisés en exploitation encadrent la valeur optimale. Avec $\tau = 0{,}35$ (liaison série), le système privilégie le rappel : moins d'intrusions manquées, plus de fausses alarmes. Avec $\tau = 0{,}55$ (liaison Wi-Fi), il privilégie la précision, ce qui a permis d'éliminer les fausses détections dues au ventilateur. Pour une application de sécurité, où manquer une intrusion est plus grave qu'une fausse alarme, un seuil légèrement inférieur à 0,459 est cohérent, à condition de filtrer les fausses alarmes isolées grâce au suivi temporel.



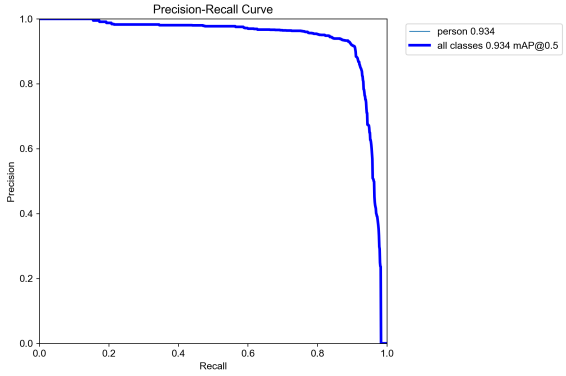
(a) Précision-confiance.



(b) Rappel-confiance.



(c) F1-confiance.



(d) Précision-rappel (IoU 0,5).

FIGURE 9 – Courbes de performance en fonction du seuil de confiance τ, et courbe précision-rappel. Le F1-score est maximal (0,91) pour $\tau = 0{,}459$.

III.3 Performances temporelles

III.3.1 Modèle seul

Lors de la validation, Ultralytics mesure en moyenne 0,4 ms de prétraitement, 4,3 ms d'inférence et 2,4 ms de post-traitement (NMS) par image de 640×640 pixels sur le GPU RTX A500 avec CUDA, soit 7,1 ms et environ 140 images par seconde. Cette valeur concerne le modèle seul.

III.3.2 Système complet

La cadence du système complet est plus faible, car chaque itération comprend aussi la capture de la trame, le suivi, l'annotation, l'affichage, l'écriture de la vidéo et du journal, et l'envoi du message. Pour la mesurer, chaque étape de la boucle a été chronométrée (`time.perf_counter`) pendant une session de 300 s en liaison Wi-Fi, avec la caméra intégrée du poste en 1280×720 pixels et un seuil $\tau = 0{,}55$. Les 30 premières trames (initialisation de CUDA) ont été écartées. Sur les 6206 trames mesurées, 86 % contenaient au moins une personne (deux au maximum). Le tableau 5 détaille les durées obtenues.

TABLE 5 – Durée des étapes de la boucle d'inférence (liaison Wi-Fi, 6206 trames, session de 300 s)

Étape	Moyenne (ms)	Médiane (ms)	P95 (ms)
Capture de la trame	8,6	8,5	10,3
Détection + suivi	25,8	25,2	30,7
Envoi UDP + journal JSON	0,6	0,5	0,7
Annotation, vidéo, affichage	12,9	12,8	20,9
Boucle complète	47,9	46,7	61,7

La boucle complète dure en moyenne 47,9 ms, soit un débit théorique de 20,9 images par seconde; le débit effectif est de 20,7 images par seconde (6206 trames en 300,1 s), la durée de la session incluant aussi les 30 trames d'échauffement écartées. La boucle reste stable : le 5e percentile de la cadence vaut 16,2 images par seconde, et seules 34 trames (0,5 %) descendent sous 15 images par seconde. La durée de la boucle varie peu selon que la scène contient au moins une personne (48,1 ms, soit 20,8 images par seconde, sur 5335 trames) ou aucune (46,2 ms, soit 21,7 images par seconde, sur 871 trames).

Le système complet n'atteint donc pas l'objectif de 25 images/s. L'étape de détection et de suivi (25,8 ms) est près de quatre fois plus longue que le modèle seul (7,1 ms) : elle comprend aussi la mise à l'échelle de la trame 1280×720, le transfert entre la mémoire centrale et le GPU, le suivi BoT-SORT et la conversion des résultats. L'annotation, l'écriture de la vidéo et l'affichage ajoutent 12,9 ms. L'envoi du message à l'ESP32 est, lui, négligeable (0,6 ms).

Côté poste, l'alerte est émise en moyenne 35,0 ms (41,0 ms au 95e percentile) après le début de la capture de la trame, soit la somme de la capture, de la détection et du suivi, et de l'envoi UDP; mesurée à partir de la fin de la capture, elle est de 26,3 ms en moyenne. Le temps de transmission Wi-Fi et de mise à jour de l'ESP32 n'a pas été mesuré séparément.

III.4 Validation expérimentale du nœud d'alerte ESP32

Le nœud d'alerte a été validé sur un prototype monté sur plaque d'essai (*breadboard*) : une carte de développement ESP32-WROOM-32 munie d'un connecteur USB-C (convertisseur CP2102, régulateur AMS1117-3.3, figure 5) et un module OLED SSD1306 de 128×64 pixels relié au bus I^{2}C (broches GND, VCC, SCL, SDA). La carte est alimentée par le câble USB-C et reçoit les messages du poste d'inférence en liaison Wi-Fi (UDP, port 4210). Les figures 10 à 13 montrent les quatre états affichés par le micrologiciel (annexe B) au cours d'une session réelle; la première ligne, « `EDGE AI via WI-FI` », rappelle le mode de liaison.

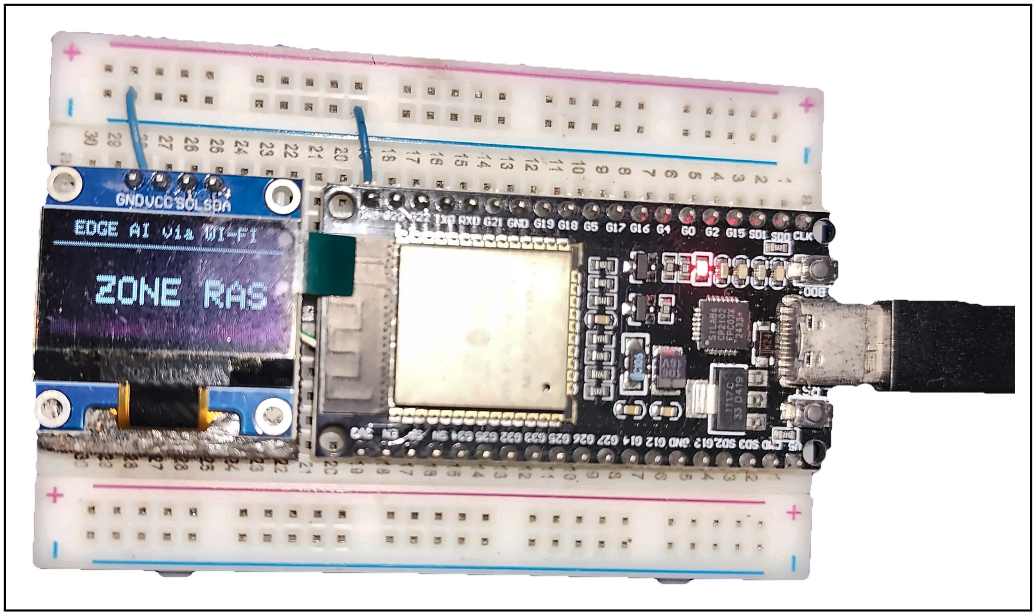


Figure 10 – Nœud d'alerte ESP32 : état « ZONE RAS ». Prototype sur plaque d'essai : écran OLED SSD1306 (à gauche) et carte ESP32-WROOM-32 alimentée par USB-C (à droite). Le poste d'inférence a envoyé $n = 0$: aucune personne n'est détectée dans le champ de la caméra, l'écran affiche « `ZONE RAS` » (rien à signaler) et le voyant d'alerte reste éteint.

Ces quatre états, aucune personne ($n = 0$), intrusion ($n > 0$), message `QUIT` et perte du Wi-Fi, correspondent aux branches de l'organigramme du micrologiciel (figure 4b). Pendant la session, l'affichage suit le nombre de personnes détectées à chaque trame reçue, ce qui confirme le fonctionnement de bout en bout de la chaîne caméra, YOLOv8n, BoT-SORT, UDP et ESP32. Ces photographies valident le comportement fonctionnel; la latence entre la détection et la mise à jour de l'écran n'a pas été mesurée (section III.6).

Seconde version de l'interface de l'afficheur. Après ces essais, l'interface a été retravaillée

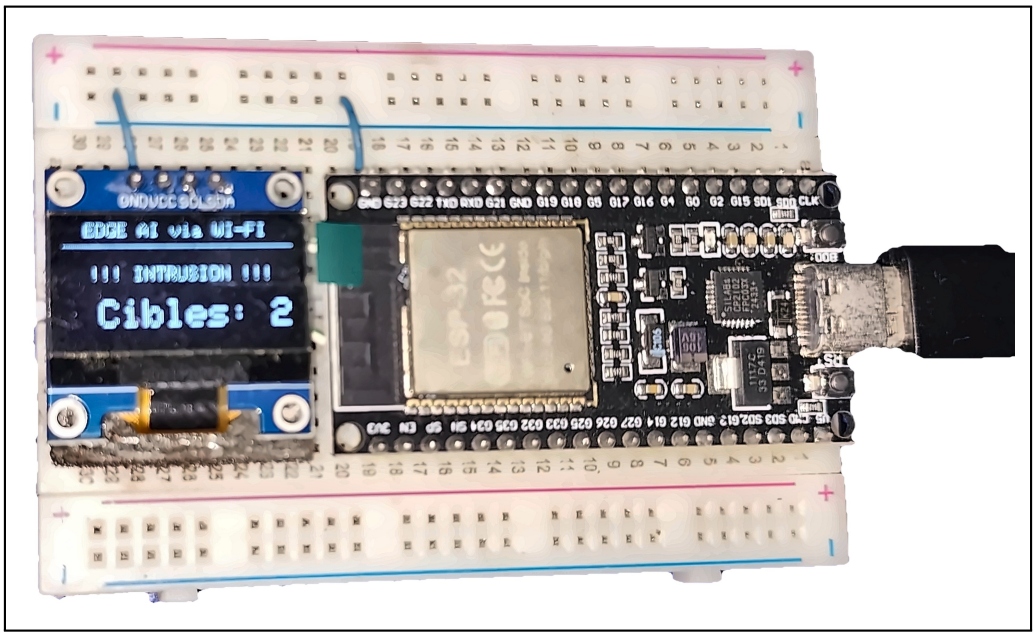


FIGURE 11 – Nœud d'alerte ESP32 : état « INTRUSION ». Le poste d'inférence a détecté deux personnes et envoyé $n = 2$ par UDP. L'écran affiche « `!!! INTRUSION !!!` » et le nombre de cibles, « `Cibles: 2` », en grands caractères; le micrologiciel commande en même temps le voyant d'alerte. Seul cet entier transite sur le réseau : aucune image ne quitte le poste local.

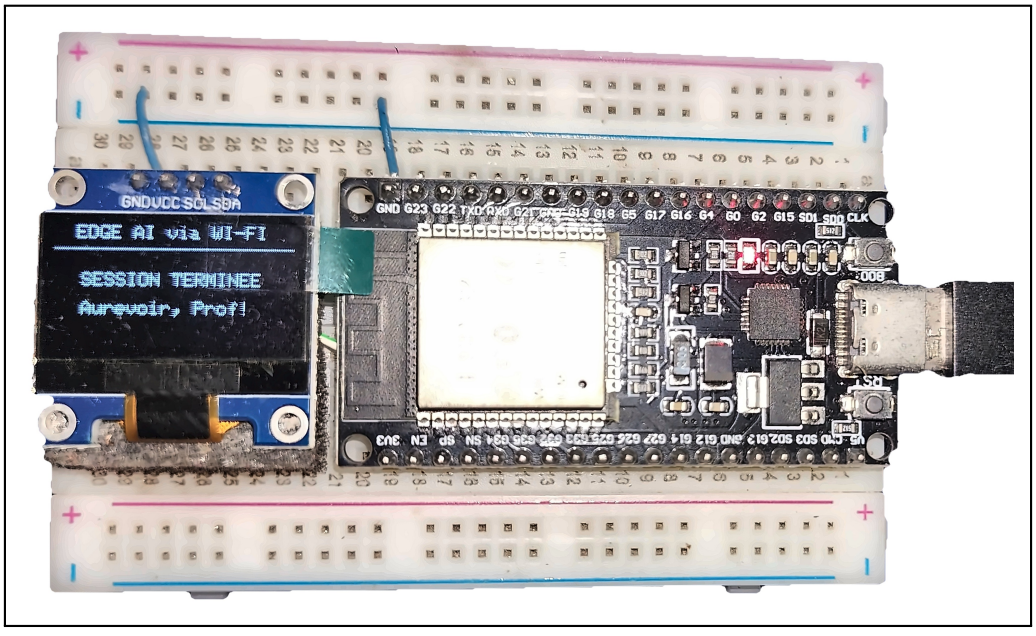


FIGURE 12 – Nœud d'alerte ESP32 : fin de session. À l'arrêt du script d'inférence (touche `q`), le poste envoie le message `QUIT`; l'ESP32 éteint le voyant et affiche « `SESSION TERMINEE` » et « `Aurevoir, Prof!` ». Les libellés des broches de la carte (GND, G23, G22, TXD, RXD, G21…) sont lisibles sur cette vue.

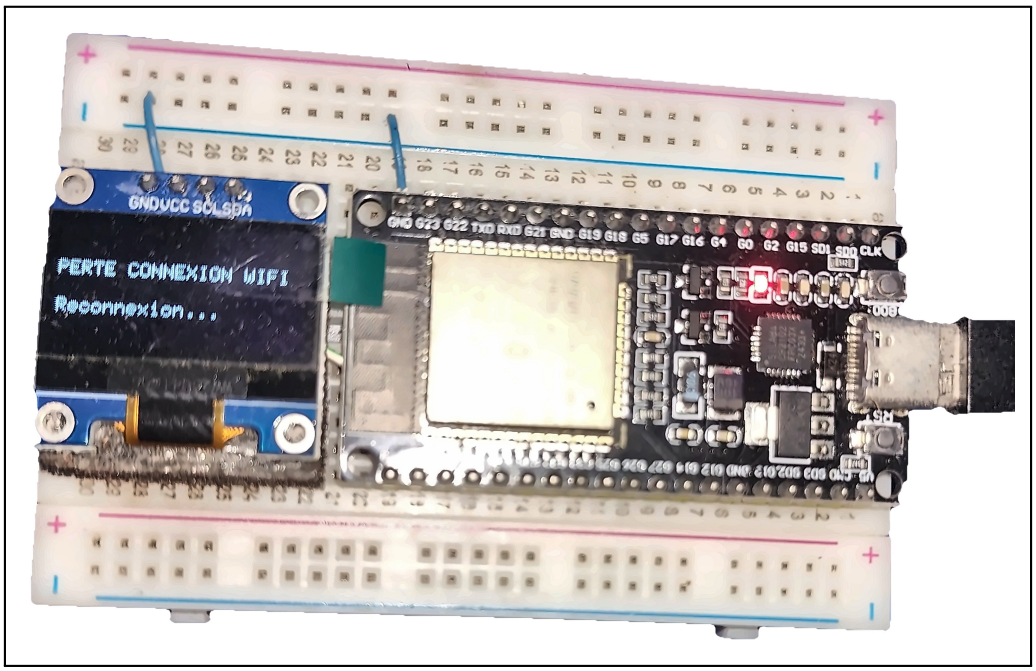


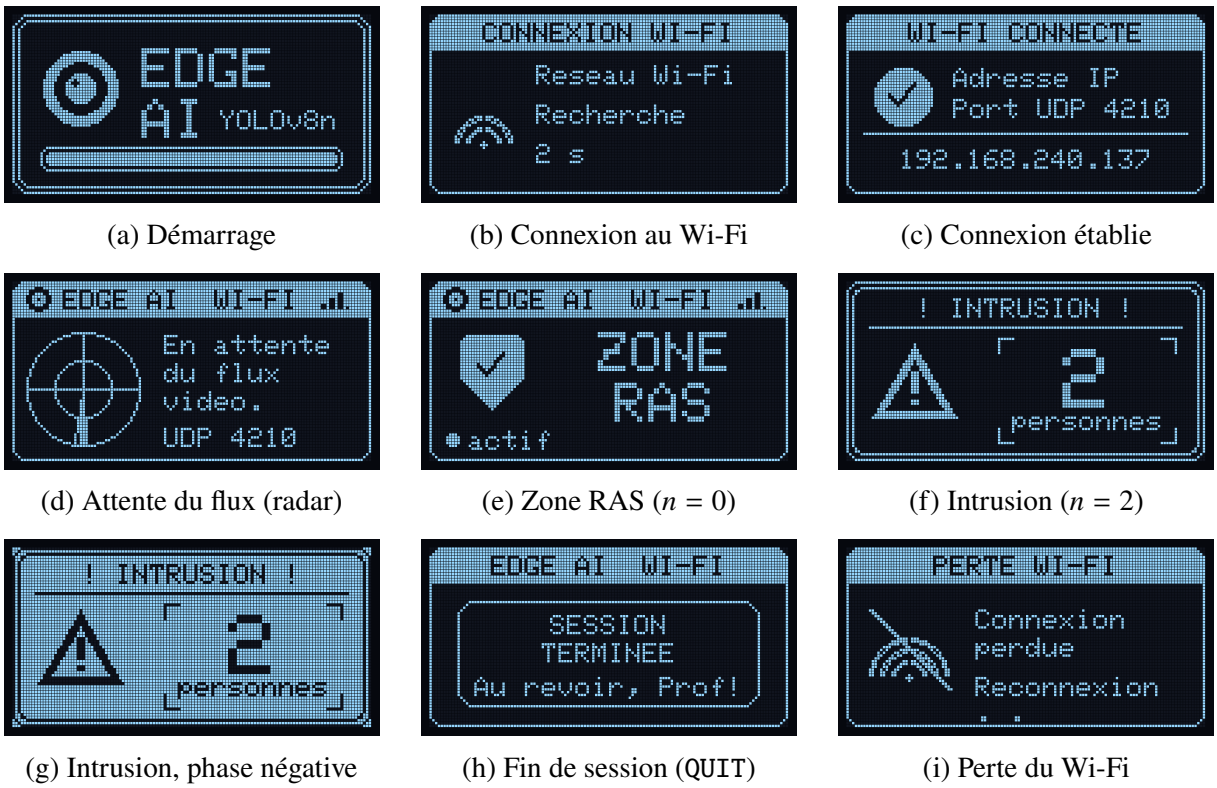
FIGURE 13 – Nœud d'alerte ESP32 : perte de la connexion Wi-Fi. Lorsque l'ESP32 n'est plus connecté au point d'accès, le micrologiciel efface l'écran, affiche « `PERTE CONNEXION WIFI` » et « `Reconnexion...` », relance la connexion (`WiFi.begin()`) puis attend 2 s avant de vérifier à nouveau. Tant que la liaison n'est pas rétablie, aucun message du poste d'inférence n'est traité; l'utilisateur sait donc que l'alerte est momentanément indisponible.

pour que l'état du système se lise d'un coup d'œil, en particulier lors d'une démonstration (code en annexe B). Le micrologiciel est désormais organisé en machine à états (attente du flux, zone RAS, intrusion, fin de session, perte du Wi-Fi), et chaque état dispose d'un écran dessiné avec les primitives de la bibliothèque Adafruit GFX : cadre à coins arrondis, bandeau de titre en négatif, pictogrammes (objectif de caméra, bouclier, triangle d'avertissement, icône Wi-Fi) et indicateur de la puissance du signal reçu (RSSI). Les écrans d'attente, de zone RAS et d'intrusion sont animés à environ 11 images par seconde : balayage d'un radar, point de vie qui pulse et clignotement en négatif de l'écran d'intrusion toutes les 270 ms environ (figure 14).

Deux modifications touchent aussi le comportement du nœud. D'une part, à chaque passage dans la boucle, le micrologiciel lit tous les datagrammes en attente et ne garde que le plus récent : le rafraîchissement des animations ne crée pas de file d'attente, et l'écran est redessiné dès que l'état ou le nombre de personnes change. D'autre part, si aucun datagramme n'arrive pendant 3 s, le nœud revient à l'écran d'attente et éteint le voyant, ce qui signale l'arrêt du poste d'inférence. Pendant la connexion, l'écran indique enfin la cause d'un échec (réseau introuvable, par exemple un point d'accès en 5 GHz, ou authentification refusée), ce qui a facilité l'emploi du partage de connexion d'un téléphone comme point d'accès.

Les photographies des figures 10 à 13 ont été prises avec la première version du micrologiciel. Les écrans de la figure 14 sont des rendus produits par la bibliothèque Adafruit GFX à partir du code de la seconde version, et non des photographies. Les mesures de la section III.3 portent

sur le poste d'inférence et ne dépendent pas du micrologiciel; l'effet des animations sur le délai d'affichage de l'ESP32 n'a pas été mesuré.



(a) Démarrage (b) Connexion au Wi-Fi (c) Connexion établie

(d) Attente du flux (radar) (e) Zone RAS ($n = 0$) (f) Intrusion ($n = 2$)

(g) Intrusion, phase négative (h) Fin de session (`QUIT`) (i) Perte du Wi-Fi

FIGURE 14 – Seconde version de l'interface de l'afficheur OLED (128×64 pixels) : rendus des écrans produits par la bibliothèque Adafruit GFX à partir du micrologiciel. Les écrans (d), (e) et (f)–(g) sont animés (balayage du radar, point de vie, clignotement de l'alerte); le bandeau supérieur indique la puissance du signal Wi-Fi reçu.

III.5 Implications opérationnelles pour la vidéosurveillance intelligente

— **Fiabilité de la détection :** avec une mAP@0.5 de 93,4 % et un F1-score maximal de 0,91, le détecteur est compatible avec la détection d'intrusion dans des scènes de surveillance réelles.

— **Marge de calcul du modèle :** le modèle seul n'occupe que 7,1 ms par trame; c'est le reste de la chaîne qu'il faut optimiser pour atteindre 25 images/s.

— **Autonomie réseau (Edge AI) :** le traitement étant local, seul l'entier n est transmis à l'ESP32; la bande passante reste disponible et le système fonctionne sans connexion Internet (Shi et al., 2016).

III.6 Contraintes et limites du système

1. **Évaluation :** les métriques sont calculées sur l'ensemble de validation, qui a aussi servi à sélectionner le meilleur modèle; l'ensemble de test (145 images) n'a pas encore été évalué.
2. **Petites cibles :** la mAP@0.5:0.95 de 50,8 % traduit une localisation imprécise, attendue pour les personnes éloignées, dont la taille approche le pas minimal du détecteur (chapitre I).
3. **Éclairage :** le jeu de données contient peu d'images nocturnes en infrarouge; la robustesse dans ces conditions n'a pas été mesurée.
4. **Matériel :** l'inférence s'exécute sur un poste équipé d'un GPU; l'ESP32 ne fait qu'afficher l'alerte. Le système n'est donc pas encore une caméra intelligente autonome.
5. **Liaison :** UDP ne garantit pas la livraison des messages; un message perdu n'est corrigé qu'à la trame suivante.
6. **Mesures temporelles :** la cadence a été mesurée en liaison Wi-Fi uniquement, sur une session de 5 min; la latence jusqu'à l'allumage du voyant et la liaison série restent à caractériser.

Conclusion

Les résultats confirment que le détecteur léger YOLOv8n, affiné sur des images de surveillance réelles, détecte les personnes de manière fiable (précision de 91,9 %, rappel de 89,9 %, mAP@0.5 de 93,4 %), avec un F1-score maximal de 0,91 pour un seuil de confiance de 0,459. Le modèle seul traite une image en 7,1 ms, mais la chaîne complète plafonne à 20,7 images par seconde : ce sont les étapes extérieures au modèle qui limitent le temps réel. Le nœud d'alerte ESP32 affiche correctement les différents états prévus. Ces résultats, avec les limites relevées, permettent de répondre aux questions de recherche dans la conclusion générale.

CONCLUSION GÉNÉRALE, PERSPECTIVES ET RECOMMANDATIONS

Rappel de l'objectif et de la démarche

L'objectif de ce travail était de concevoir et d'évaluer un système de vidéosurveillance en périphérie de réseau (*Edge AI*) capable de détecter automatiquement les personnes et de déclencher une alerte physique. Le détecteur YOLOv8 Nano (Jocher et al., 2023), pré-entraîné sur MS COCO, a été affiné sur 1057 images de caméras de surveillance réelles (Konkuk University, 2024). Il a été intégré dans une chaîne complète : suivi BoT-SORT (Aharon et al., 2022), journalisation JSON et transmission du nombre de personnes détectées à un nœud d'alerte ESP32 par Wi-Fi (UDP) ou liaison série. Les performances ont été évaluées par les courbes d'apprentissage, les matrices de confusion, les courbes de confiance et une mesure du temps de traitement de chaque étape de la chaîne.

Réponses aux questions de recherche

Q1. **Performance de détection :** sur l'ensemble de validation, le modèle atteint une précision de 0,919, un rappel de 0,899, une mAP@0.5 de 0,934 et une mAP@0.5:0.95 de 0,508. La détection des personnes est fiable; la précision de localisation reste à améliorer, notamment pour les cibles éloignées.

Q2. **Seuil de confiance :** le F1-score est maximal (0,91) pour $\tau = 0{,}459$. Les seuils de 0,35 et 0,55 utilisés en exploitation correspondent respectivement à une priorité au rappel et à une priorité à la précision.

Q3. **Temps réel :** le modèle seul traite une image en 7,1 ms sur un GPU RTX A500, bien en deçà des 40 ms correspondant à 25 images/s. En revanche, la chaîne complète atteint 20,7 images par seconde (47,9 ms par trame en moyenne) : l'objectif de 25 images/s n'est pas encore atteint. Le temps se répartit entre la détection et le suivi (25,8 ms), l'annotation, l'enregistrement et l'affichage (12,9 ms) et la capture (8,6 ms), l'envoi de l'alerte restant négligeable.

Atteinte des objectifs et vérification des hypothèses

Les quatre objectifs spécifiques fixés dans l'introduction générale sont atteints, le dernier en partie seulement :

1. **Affinage du détecteur :** atteint. YOLOv8n a été affiné pendant 100 époques sur 1057 images de caméras de surveillance réelles; le léger surapprentissage observé en fin d'entraînement ne dégrade pas les métriques du meilleur modèle retenu.
2. **Évaluation et choix du seuil :** atteint. Les indicateurs standardisés ont été calculés et le seuil qui maximise le F1-score ($\tau = 0{,}459$) a été déterminé.

3. **Intégration dans une chaîne complète :** atteint. Le prototype associe détection, suivi BoT-SORT, journalisation JSON et alerte sur le nœud ESP32, dont les quatre états d'affichage ont été validés expérimentalement. La carte dédiée du nœud d'alerte est conçue mais pas encore fabriquée.
4. **Mesure des performances temporelles :** partiellement atteint. La cadence de la chaîne complète a été mesurée en liaison Wi-Fi; la latence jusqu'à l'allumage du voyant et la liaison série restent à caractériser.

Les hypothèses H1 (mAP@0.5 d'au moins 90 %) et H2 (seuil intermédiaire optimal) sont vérifiées. L'hypothèse H3 l'est aussi : le modèle seul respecte largement le budget de 40 ms par trame, et ce sont les étapes extérieures au modèle qui empêchent la chaîne complète d'atteindre 25 images/s.

L'objectif principal est donc atteint pour l'essentiel : le système détecte localement les personnes et déclenche une alerte physique sans serveur distant. Le temps réel strict (25 images/s) n'est pas encore atteint par la chaîne complète (20,7 images par seconde), ce qui fixe la priorité des travaux futurs.

Limites de l'étude

Plusieurs points n'ont pas été traités et auraient mérité de l'être (section III.6) : l'évaluation sur l'ensemble de test, distinct de l'ensemble de validation qui a servi à choisir le meilleur modèle; la robustesse sur des séquences nocturnes en infrarouge; la précision de localisation des personnes éloignées (mAP@0.5:0.95 de 50,8 %); la fiabilité de la liaison UDP, sans accusé de réception; et l'exécution de l'inférence sur une plateforme embarquée plutôt que sur un poste équipé d'un GPU.

Avantages et contributions du système

La contribution de ce mémoire est un prototype complet et mesuré, de la caméra jusqu'à l'alerte physique, qui fonctionne sans serveur distant (Shi et al., 2016). Ses avantages sont les suivants :

— **Autonomie et résilience :** l'inférence étant locale, le dispositif reste fonctionnel en cas de connectivité instable, une contrainte fréquente au Cameroun.

— **Sobriété matérielle :** un ordinateur portable muni d'un GPU d'entrée de gamme et une carte ESP32 à bas coût suffisent, sans serveur de calcul dédié.

— **Limitation de l'exposition des données :** aucune image n'est transmise hors du poste local; seul le nombre de personnes est envoyé à l'ESP32. Les vidéos et journaux enregistrés localement doivent toutefois faire l'objet d'une politique de conservation et d'accès (Zuboff, 2019).

Perspectives

— **Optimisation de la chaîne d'inférence :** pour dépasser 25 images/s, utiliser l'inférence en demi-précision (FP16), déplacer l'écriture de la vidéo et l'affichage dans un fil d'exécution séparé, ou désactiver l'affichage en exploitation.

— **Compléments de mesure :** évaluer le modèle sur l'ensemble de test et sur des séquences nocturnes, et mesurer la latence jusqu'à l'allumage du voyant en liaison Wi-Fi et série.

— **Quantification post-entraînement :** convertir les poids du format FP32 au format INT8 (Krishnamoorthi, 2018), ce qui divise par quatre l'empreinte mémoire des poids et ouvre la voie à une exécution sur une plateforme embarquée plutôt que sur un poste à GPU.

— **Fabrication de la carte dédiée :** fabriquer la carte V1.4 du nœud d'alerte (annexe C) et mesurer son rendement, son autonomie et la latence jusqu'au voyant.

— **Fiabilité de l'alerte :** remplacer l'envoi UDP simple par un protocole avec accusé de réception (ou MQTT), et filtrer temporellement les alarmes grâce aux identifiants de suivi.

Recommandations

Pour les gestionnaires de sites à surveiller

— **Adapter le seuil à la zone :** retenir un seuil bas ($\tau \approx 0{,}35$) pour les zones sensibles, où une intrusion manquée coûte cher, et un seuil plus élevé ($\tau \approx 0{,}55$) pour les zones de passage, afin de limiter les fausses alarmes.

— **Affiner le modèle sur les images du site :** avant tout déploiement, compléter le jeu d'entraînement par quelques centaines d'images annotées des caméras réellement installées, de jour comme de nuit.

— **Encadrer les données enregistrées :** fixer une durée de conservation des vidéos et des journaux, en restreindre l'accès et flouter les personnes avant tout archivage.

— **Sécuriser l'alimentation :** associer le poste d'inférence et le nœud d'alerte à une alimentation secourue (onduleur ou batterie) pour conserver l'alerte pendant les coupures d'électricité.

Recommandations plus générales

— **Mesurer la chaîne complète :** les travaux sur la détection embarquée gagneraient à rapporter le temps de traitement de toute la chaîne (capture, suivi, affichage, transmission) et pas seulement celui du modèle, qui n'en représente ici qu'une faible part.

— **Privilégier le traitement local :** dans les contextes de connectivité instable comme au Cameroun, le traitement en périphérie de réseau assure à la fois la résilience du système et la limitation de l'exposition des images.

— **Évaluer sur un ensemble de test indépendant :** les performances annoncées doivent être confirmées sur des images qui n'ont servi ni à l'entraînement ni à la sélection du modèle.

Enseignements tirés de l'étude

Sur le plan théorique et conceptuel, ce travail montre qu'un détecteur léger de l'état de l'art, affiné par transfert d'apprentissage sur un jeu de données modeste, suffit à détecter les personnes de manière fiable dans des scènes de surveillance réelles; l'écart entre la mAP@0.5 et la mAP@0.5:0.95 rappelle toutefois que détecter une personne et la localiser précisément sont deux exigences distinctes. Sur le plan pratique, il montre que le temps réel d'un système de vidéosurveillance intelligent se joue autant dans l'organisation de la chaîne de traitement que dans le choix du modèle.

RÉFÉRENCES BIBLIOGRAPHIQUES

Advanced Monolithic Systems, s. d. AMS1117 : 1A Low Dropout Voltage Regulator. Data sheet.

Aerosemi Technology, s. d. MT3608 : High Efficiency 1.2 MHz 2A Step Up Converter. Data sheet.

Aharon N, Orfaig R, Bobrovsky BZ, 2022. BoT-SORT : Robust Associations Multi-Pedestrian Tracking. arXiv :2206.14651.

Anthropic, 2026. Claude : assistant d'intelligence artificielle générative. URL : `https://claude.ai` (visité le 13/11/2026).

Barry R, 2016. Mastering the FreeRTOS Real Time Kernel : A Hands-On Tutorial Guide. Real Time Engineers Ltd.

Bewley A, Ge Z, Ott L, Ramos F, Upcroft B, 2016. Simple Online and Realtime Tracking. In : *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 3464-3468.

Bochkovskiy A, Wang CY, Liao HYM, 2020. YOLOv4 : Optimal Speed and Accuracy of Object Detection. arXiv :2004.10934.

Chen J, Ran X, 2019. Deep Learning With Edge Computing : A Review. Proceedings of the IEEE 107(8), 1655-1674.

Dalal N, Triggs B, 2005. Histograms of Oriented Gradients for Human Detection. In : *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*. T. 1, 886-893.

Dollár P, Wojek C, Schiele B, Perona P, 2012. Pedestrian Detection : An Evaluation of the State of the Art. IEEE Transactions on Pattern Analysis and Machine Intelligence 34(4), 743-761.

Elharrouss O, Almaadeed N, Al-Maadeed S, 2021. A Review of Video Surveillance Systems. Journal of Visual Communication and Image Representation 77, 103116. `https://doi.org/10.1016/j.jvcir.2021.103116`.

Espressif Systems, s. d. ESP32 Series Datasheet. URL : `https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf`.

Everingham M, Van Gool L, Williams CKI, Winn J, Zisserman A, 2010. The PASCAL Visual Object Classes (VOC) Challenge. International Journal of Computer Vision 88(2), 303-338.

Felzenszwalb PF, Girshick RB, McAllester D, Ramanan D, 2010. Object Detection with Discriminatively Trained Part-Based Models. IEEE Transactions on Pattern Analysis and Machine Intelligence 32(9), 1627-1645.

Fortune Semiconductor, s. d.(a). DW01A : One Cell Lithium-ion/Polymer Battery Protection IC. Data sheet.

Fortune Semiconductor, s. d.(b). FS8205A : Dual N-Channel Enhancement Mode Power MOSFET. Data sheet.

Ge Z, Liu S, Wang F, Li Z, Sun J, 2021. YOLOX : Exceeding YOLO Series in 2021. arXiv :2107.08430.

Girshick R, 2015. Fast R-CNN. In : *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 1440-1448.

Girshick R, Donahue J, Darrell T, Malik J, 2014. Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 580-587.

Goodfellow I, Bengio Y, Courville A, 2016. Deep Learning. Cambridge, MA : MIT Press.

Han S, Mao H, Dally WJ, 2016. Deep Compression : Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In : *International Conference on Learning Representations (ICLR)*.

He K, Zhang X, Ren S, Sun J, 2016. Deep Residual Learning for Image Recognition. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770-778.

Howard AG, Zhu M, Chen B, Kalenichenko D, Wang W, Weyand T, Andreetto M, Adam H, 2017. MobileNets : Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv :1704.04861.

Hu W, Tan T, Wang L, Maybank S, 2004. A Survey on Visual Surveillance of Object Motion and Behaviors. IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews) 34(3), 334-352.

Jacob B, Kligys S, Chen B, Zhu M, Tang M, Howard A, Adam H, Kalenichenko D, 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2704-2713.

Jocher G, 2020. Ultralytics YOLOv5. Licence AGPL-3.0. URL : `https://github.com/ultralytics/yolov5`.

Jocher G, Chaurasia A, Qiu J, 2023. Ultralytics YOLOv8. Version 8.0.0. Licence AGPL-3.0. URL : `https://github.com/ultralytics/ultralytics`.

Kalman RE, 1960. A New Approach to Linear Filtering and Prediction Problems. Journal of Basic Engineering 82(1), 35-45.

Kengne Takam IE, 2026a. Caméras de surveillance inutiles ? Comment l'IA peut stopper les cambrioleurs en direct ! Vidéo, 4 min 30 s. YouTube, chaîne Ibrahim Eric. URL : https://www.youtube.com/watch?v=Blm4Lbpq0t4 (visité le 13/11/2026).

Kengne Takam IE, 2026b. Memoire-M2R-EdgeAI : code source et documentation du système de surveillance Edge AI (YOLOv8n, ESP32). URL : https://github.com/ProfKengne/Memoire-M2R-EdgeAI (visité le 13/11/2026).

Kengne Takam IE, 2026c. Vidéosurveillance intelligente : quand l'IA prend des décisions en temps réel ! Vidéo, 8 min 15 s. YouTube, chaîne Ibrahim Eric. URL : https://www.youtube.com/watch?v=3qP1NFRnskk (visité le 13/11/2026).

KiCad Developers, s. d. KiCad EDA. Version 10.0.5. Licence GPL-3.0. URL : https://www.kicad.org.

Kingma DP, Ba J, 2015. Adam : A Method for Stochastic Optimization. In : *International Conference on Learning Representations (ICLR)*.

Konkuk University, 2024. CCTV Person Detection Dataset, version 3. Licence CC BY 4.0. Roboflow Universe. URL : https://universe.roboflow.com/konkuk-university-9whom/cctv-person-detection/dataset/3 (visité le 19/06/2026).

Krishnamoorthi R, 2018. Quantizing Deep Convolutional Networks for Efficient Inference : A Whitepaper. arXiv :1806.08342.

Krizhevsky A, Sutskever I, Hinton GE, 2012. ImageNet Classification with Deep Convolutional Neural Networks. In : *Advances in Neural Information Processing Systems (NeurIPS)*. T. 25, 1097-1105.

LeCun Y, Bengio Y, Hinton G, 2015. Deep Learning. Nature 521(7553), 436-444.

LeCun Y, Bottou L, Bengio Y, Haffner P, 1998. Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE 86(11), 2278-2324.

Li C, Li L, Jiang H, Weng K, Geng Y, Li L, Ke Z, Li Q, Cheng M, Nie W et al., 2022. YOLOv6 : A Single-Stage Object Detection Framework for Industrial Applications. arXiv :2209.02976.

Li X, Wang W, Wu L, Chen S, Hu X, Li J, Tang J, Yang J, 2020. Generalized Focal Loss : Learning Qualified and Distributed Bounding Boxes for Dense Object Detection. In : *Advances in Neural Information Processing Systems (NeurIPS)*. T. 33, 21002-21012.

Lin TY, Dollár P, Girshick R, He K, Hariharan B, Belongie S, 2017a. Feature Pyramid Networks for Object Detection. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2117-2125.

Lin TY, Goyal P, Girshick R, He K, Dollár P, 2017b. Focal Loss for Dense Object Detection. In : *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2980-2988.

Lin TY, Maire M, Belongie S, Hays J, Perona P, Ramanan D, Dollár P, Zitnick CL, 2014. Microsoft COCO : Common Objects in Context. In : *Proceedings of the European Conference on Computer Vision (ECCV)*, 740-755.

Liu S, Qi L, Qin H, Shi J, Jia J, 2018. Path Aggregation Network for Instance Segmentation. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 8759-8768.

Liu W, Anguelov D, Erhan D, Szegedy C, Reed S, Fu CY, Berg AC, 2016. SSD : Single Shot MultiBox Detector. In : *Proceedings of the European Conference on Computer Vision (ECCV)*, 21-37.

Loshchilov I, Hutter F, 2019. Decoupled Weight Decay Regularization. In : *International Conference on Learning Representations (ICLR)*.

Microchip Technology, s. d. MCP73831/2 : Miniature Single-Cell, Fully Integrated Li-Ion, Li-Polymer Charge Management Controllers. Data sheet.

NXP Semiconductors, s. d. UM10204 : I^2C-bus specification and user manual. Manuel utilisateur.

OpenCV Team, 2024. OpenCV : Open Source Computer Vision Library. Licence Apache 2.0. URL : `https://opencv.org` (visité le 19/10/2026).

Padilla R, Netto SL, Silva EAB da, 2020. A Survey on Performance Metrics for Object-Detection Algorithms. In : *Proceedings of the International Conference on Systems, Signals and Image Processing (IWSSIP)*, 237-242.

Pan SJ, Yang Q, 2010. A Survey on Transfer Learning. IEEE Transactions on Knowledge and Data Engineering 22(10), 1345-1359.

PlatformIO, 2026. platformio.ini Configuration File Documentation. URL : `https://docs.platformio.org/page/projectconf.html` (visité le 30/08/2026).

Postel J, 1980. User Datagram Protocol. RFC 768, Internet Engineering Task Force.

Redmon J, Divvala S, Girshick R, Farhadi A, 2016. You Only Look Once : Unified, Real-Time Object Detection. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779-788.

Redmon J, Farhadi A, 2017. YOLO9000 : Better, Faster, Stronger. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 7263-7271.

Redmon J, Farhadi A, 2018. YOLOv3 : An Incremental Improvement. arXiv :1804.02767.

Ren S, He K, Girshick R, Sun J, 2017. Faster R-CNN : Towards Real-Time Object Detection with Region Proposal Networks. IEEE Transactions on Pattern Analysis and Machine Intelligence 39(6), 1137-1149.

Rezatofighi H, Tsoi N, Gwak J, Sadeghian A, Reid I, Savarese S, 2019. Generalized Intersection over Union : A Metric and a Loss for Bounding Box Regression. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 658-666.

Sandler M, Howard A, Zhu M, Zhmoginov A, Chen LC, 2018. MobileNetV2 : Inverted Residuals and Linear Bottlenecks. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 4510-4520.

Satyanarayanan M, 2017. The Emergence of Edge Computing. Computer 50(1), 30-39.

Senior A, Pankanti S, Hampapur A, Brown L, Tian YL, Ekin A, Connell J, Shu CF, Lu M, 2005. Enabling Video Privacy through Computer Vision. IEEE Security & Privacy 3(3), 50-57.

Shi W, Cao J, Zhang Q, Li Y, Xu L, 2016. Edge Computing : Vision and Challenges. IEEE Internet of Things Journal 3(5), 637-646.

Solomon Systech, s. d. SSD1306 : 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller. Data sheet.

Stauffer C, Grimson WEL, 1999. Adaptive Background Mixture Models for Real-Time Tracking. In : *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*. T. 2, 246-252.

Sze V, Chen YH, Yang TJ, Emer JS, 2017. Efficient Processing of Deep Neural Networks : A Tutorial and Survey. Proceedings of the IEEE 105(12), 2295-2329.

Tan M, Pang R, Le QV, 2020. EfficientDet : Scalable and Efficient Object Detection. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10781-10790.

Terven J, Córdova-Esparza DM, Romero-González JA, 2023. A Comprehensive Review of YOLO Architectures in Computer Vision : From YOLOv1 to YOLOv8 and YOLO-NAS. Machine Learning and Knowledge Extraction 5(4), 1680-1716.

USB Implementers Forum, s. d. Universal Serial Bus Type-C Cable and Connector Specification. Spécification.

Viola P, Jones MJ, 2004. Robust Real-Time Face Detection. International Journal of Computer Vision 57(2), 137-154.

Wang A, Chen H, Liu L, Chen K, Lin Z, Han J, Ding G, 2024a. YOLOv10 : Real-Time End-to-End Object Detection. arXiv :2405.14458.

Wang CY, Bochkovskiy A, Liao HYM, 2023. YOLOv7 : Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 7464-7475.

Wang CY, Liao HYM, Wu YH, Chen PY, Hsieh JW, Yeh IH, 2020. CSPNet : A New Backbone That Can Enhance Learning Capability of CNN. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 390-391.

Wang CY, Yeh IH, Liao HYM, 2024b. YOLOv9 : Learning What You Want to Learn Using Programmable Gradient Information. arXiv :2402.13616.

Warden P, Situnayake D, 2019. TinyML : Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media.

Wojke N, Bewley A, Paulus D, 2017. Simple Online and Realtime Tracking with a Deep Association Metric. In : *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 3645-3649.

Zhang S, Benenson R, Schiele B, 2017. CityPersons : A Diverse Dataset for Pedestrian Detection. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 3213-3221.

Zhang Y, Sun P, Jiang Y, Yu D, Weng F, Yuan Z, Luo P, Liu W, Wang X, 2022. ByteTrack : Multi-Object Tracking by Associating Every Detection Box. In : *Proceedings of the European Conference on Computer Vision (ECCV)*, 1-21.

Zhao ZQ, Zheng P, Xu ST, Wu X, 2019. Object Detection With Deep Learning : A Review. IEEE Transactions on Neural Networks and Learning Systems 30(11), 3212-3232.

Zheng Z, Wang P, Liu W, Li J, Ye R, Ren D, 2020. Distance-IoU Loss : Faster and Better Learning for Bounding Box Regression. In : *Proceedings of the AAAI Conference on Artificial Intelligence*. T. 34. 7, 12993-13000.

Zhou Z, Chen X, Li E, Zeng L, Luo K, Zhang J, 2019. Edge Intelligence : Paving the Last Mile of Artificial Intelligence With Edge Computing. Proceedings of the IEEE 107(8), 1738-1762.

Zou Z, Chen K, Shi Z, Guo Y, Ye J, 2023. Object Detection in 20 Years : A Survey. Proceedings of the IEEE 111(3), 257-276.

Zuboff S, 2019. The Age of Surveillance Capitalism : The Fight for a Human Future at the New Frontier of Power. PublicAffairs.

ANNEXES

ANNEXE A : CODE SOURCE DE L'ALGORITHME **II**

ANNEXE B : SCHÉMAS TECHNIQUES DU SYSTÈME DE SURVEILLANCE **XII**

ANNEXE C : CONCEPTION ÉLECTRONIQUE DÉTAILLÉE DU NŒUD D'ALERTE **XXV**

ANNEXE A
CODE SOURCE DE L'ALGORITHME

Les scripts de cette annexe sont également disponibles dans le dépôt public de l'auteur (Kengne Takam, 2026b).

A.1 Configuration de l'environnement d'apprentissage (Ultralytics)

Le script suivant configure le pipeline d'entraînement supervisé : il charge les poids pré-entraînés de YOLOv8 Nano, associe le fichier `data.yaml` du jeu de données Roboflow et lance l'entraînement sur 100 époques avec l'optimiseur SGD. Les hyperparamètres non précisés prennent les valeurs par défaut d'Ultralytics (tableau 3).

Listing A.1 – Script d'entraînement et de validation du modèle YOLOv8n (`train.py`)

```
# Script d'entraînement et de validation du modèle YOLOv8n
#import torch #import torch travaille désormais dans l'ombre d'Ultralytics
import os # import os sécurise la structure de vos dossiers locaux !
from ultralytics import YOLO

def main():
    # 1. Initialisation et chargement du modele de base (Pre-entraine)
    # Le suffixe 'n' designe l'architecture Nano (3.2 millions de parametres)
    model = YOLO('yolov8n.pt')

    # 2. Definition des chemins d'acces du projet
    dataset_config = os.path.abspath("data.yaml")
    project_dir = "runs/detect"
    experiment_name = "run_production_yolov8n"

    print(f"[INFO] Configuration du jeu de donnees : {dataset_config}")
    print("[INFO] Debut du cycle complet d'entrainement (100 epoques)...")

    # 3. Lancement de l'entrainement avec les hyperparametres optimises
    results = model.train(
        data=dataset_config,  # Fichier de description des classes et chemins
        epochs=100,           # Nombre de cycles complets
        imgsz=640,            # Resolution matricielle des trames d'entree
        batch=16,             # Taille du lot (adaptee aux ressources VRAM)
        workers=4,            # Nombre de threads pour le chargement des donnees
        optimizer='SGD',      # Optimiseur stochastique selectionne
        lr0=0.01,             # Taux d'apprentissage initial
        momentum=0.937,       # Facteur de quantite de mouvement
```

```
        weight_decay=0.0005,  # Regularisation L2 pour eviter le surapprentissage
        device=0,             # Index de l'unite graphique (GPU CUDA)
        project=project_dir,  # Dossier racine de supervision
        name=experiment_name, # Nom specifique de la simulation
        plots=True            # Generation automatique des courbes de performance
    )

    print(f"[SUCCESS] Entrainement termine. Les poids sont sauvegardes dans {project_dir}/{experiment_name}/weights/best.pt")

if __name__ == '__main__':
    main()
```

A.2 Script d'inférence, d'alerte et de mesure en temps réel

Ce script constitue le cœur opérationnel du dispositif (figure 4a). Il ouvre la caméra en vérifiant qu'elle livre de vraies images, applique le modèle entraîné (`best.pt`) avec le suivi BoT-SORT, journalise chaque détection au format JSON, envoie le nombre de personnes à l'ESP32 par UDP, enregistre la vidéo annotée et chronomètre chaque étape de la boucle. Les mesures du tableau 5 ont été obtenues avec ce script.

Listing A.2 – Inférence temps réel, alerte UDP et mesure des performances (`detect_rt_INTRUSION_save_JSON_WIFI.py`)

```
# Script d'inférence temps réel Edge AI, Suivi de cibles, Enregistrement Vidéo, Logs JSON & Wi-Fi UDP
# + Mesure des performances temporelles de bout en bout (pour l'article)
import os
import cv2
import time
import json
import csv
import socket # <-- Remplacera 'requests' pour une communication ultra-rapide
import statistics
from ultralytics import YOLO

#  CONFIGURATION RÉSEAU UDP
ESP32_IP = "192.168.1.197"
UDP_PORT = 4210 # Port arbitraire identique àcelui de l'ESP32
```

```
15
16  #  CONFIGURATION DES MESURES
17  TRAMES_ECHAUFFEMENT = 30 # Trames ignorées au début (chargement CUDA,
        ouverture caméra)
18  # Durée de la session de mesure en secondes (0 = illimitée, arrêt par la
        touche 'q')
19  DUREE_MESURE_S = float(os.environ.get("DUREE_MESURE_S", "300"))
20
21  # Initialisation du socket UDP
22  sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
23
24
25  def resume_statistique(valeurs):
26      """Moyenne, écart-type, minimum, maximum et percentiles d'une liste de
            mesures."""
27      v = sorted(valeurs)
28      n = len(v)
29      return {
30          "n": n,
31          "moyenne": round(statistics.mean(v), 2),
32          "ecart_type": round(statistics.stdev(v), 2) if n > 1 else 0.0,
33          "min": round(v[0], 2),
34          "p5": round(v[int(0.05 * (n - 1))], 2),
35          "mediane": round(statistics.median(v), 2),
36          "p95": round(v[int(0.95 * (n - 1))], 2),
37          "max": round(v[-1], 2),
38      }
39
40
41  def ouvrir_camera(source_video):
42      """Ouvre la caméra en vérifiant qu'elle livre de vraies images (ni échec,
            ni image noire).
43      Essaie d'abord le pilote Windows par défaut (MSMF), puis DirectShow."""
44      for nom, api in (("MSMF", cv2.CAP_MSMF), ("DirectShow", cv2.CAP_DSHOW)):
45          cap = cv2.VideoCapture(source_video, api)
46          if not cap.isOpened():
47              continue
48          cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
49          cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
50          for _ in range(15):
51              ok, trame = cap.read()
```

```
52              if ok and trame is not None and trame.mean() > 2.0:
53                  print(f"Caméra ouverte avec le pilote {nom}.")
54                  return cap
55          cap.release()
56      print("La caméra ne livre aucune image exploitable. Vérifiez qu'aucune autre "
57            "application (ou une session précédente de ce script) ne l'utilise, et que "
58            "l'obturateur de confidentialité est ouvert.")
59      return None
60
61  def executer_inference_suivi_enregistrement(chemin_poids, source_video):
62      print("Chargement du modèle YOLOv8 personnalisé...")
63      model = YOLO(chemin_poids)
64      print("Modèle chargé avec succès !")
65
66      cap = ouvrir_camera(source_video)
67      if cap is None:
68          return
69
70      frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
71      frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
72      fps_webcam = int(cap.get(cv2.CAP_PROP_FPS))
73
74      if fps_webcam <= 0 or fps_webcam > 60: # DirectShow renvoie -1
75          fps_webcam = 20
76
77      horodatage_base = time.strftime("%Y%m%d_%H%M%S", time.localtime())
78
79      nom_fichier_sortie = f"Video_surveillance_Ibrahim_{horodatage_base}.avi"
80      fourcc = cv2.VideoWriter_fourcc(*'XVID')
81      out = cv2.VideoWriter(nom_fichier_sortie, fourcc, fps_webcam, (frame_width, frame_height))
82
83      nom_fichier_log = f"Journal_Alertes_Ibrahim_{horodatage_base}.jsonl"
84      nom_fichier_mesures = f"Mesures_temps_Ibrahim_{horodatage_base}.csv"
85      nom_fichier_resume = f"Resume_mesures_Ibrahim_{horodatage_base}.json"
86      print(f"Les métadonnées Edge AI seront enregistrées dans : {nom_fichier_log}")
87      print(f"Vidéo enregistrée sous : {nom_fichier_sortie}")
88      print(f"Mesures temporelles enregistrées dans : {nom_fichier_mesures}")
```

```
89      print(f"Résolution réelle de la caméra : {frame_width}x{frame_height}, "
90            f"FPS annoncé : {int(cap.get(cv2.CAP_PROP_FPS))}")
91      print("\n Système actif en mode Wi-Fi UDP (Latence minimale). Appuyez sur
            'q' pour arrêter.")
92
93      # Mesures par trame (après échauffement)
94      mesures = []
95      index_trame = 0
96      debut_session = time.perf_counter()
97
98      with open(nom_fichier_log, mode='a', encoding='utf-8') as fichier_log, \
99           open(nom_fichier_mesures, mode='w', newline='', encoding='utf-8') as
                fichier_mesures:
100
101         ecrivain = csv.writer(fichier_mesures, delimiter=';')
102         ecrivain.writerow(["trame", "t_session_s", "nb_personnes",
103                            "capture_ms", "track_ms", "envoi_log_ms",
104                            "affichage_ms", "total_ms", "fps_bout_en_bout"])
105
106         while True:
107             t0 = time.perf_counter()
108             ret, frame = cap.read()
109             if not ret:
110                 print("Lecture de trame impossible : arrêt.")
111                 break
112             t1 = time.perf_counter() # fin capture
113
114             results = model.track(
115                 source=frame,
116                 persist=True,
117                 conf=0.55, # <-- Seuil augmenté à0.55 pour éliminer les
                        détections du ventilateur
118                 iou=0.45, # <-- Plus sélectif sur les chevauchements
119                 classes=[0],
120                 device=0,
121                 verbose=False,
122                 imgsz=640
123             )
124             t2 = time.perf_counter() # fin détection + suivi
125
126             nb_personnes = 0
```

```
127             liste_alertes = []
128             horodatage_actuel = time.strftime("%Y-%m-%d %H:%M:%S",
                    time.localtime())
129
130             for r in results:
131                 boxes = r.boxes
132                 nb_personnes = len(boxes)
133                 for box in boxes:
134                     coords = box.xyxy[0].tolist()
135                     confiance = float(box.conf[0])
136                     track_id = int(box.id[0].item()) if box.id is not None else -1
137
138                     alerte_metadonnees = {
139                         "timestamp": horodatage_actuel,
140                         "tracking_id": track_id,
141                         "classe": "person",
142                         "confiance": round(confiance, 4),
143                         "bbox_pixels": [round(c, 1) for c in coords]
144                     }
145                     liste_alertes.append(alerte_metadonnees)
146
147             #  ENVOI DE LA DONNÉE ÀL'ESP32 VIA PAQUET UDP BRUT
148             try:
149                 # Payload minimal : conversion directe de l'entier en octets
150                 payload = str(nb_personnes).encode()
151                 sock.sendto(payload, (ESP32_IP, UDP_PORT))
152             except Exception as e:
153                 # L'UDP n'étant pas connecté, les erreurs réseau ne bloquent pas
                        la boucle IA
154                 pass
155
156             #  Écriture dans le fichier de log local
157             if liste_alertes:
158                 for alerte in liste_alertes:
159                     fichier_log.write(json.dumps(alerte) + "\n")
160                 fichier_log.flush()
161                 print(f"[EDGE ALERT] Cibles : {nb_personnes} | Logs
                        synchronisés.")
162             t3 = time.perf_counter() # fin envoi UDP + journal
163
164             # FPS de la trame précédente (le calcul de la trame courante n'est
```

```
            complet qu'après affichage)
165         fps_affiche = mesures[-1]["fps"] if mesures else 0.0
166
167         annotated_frame = results[0].plot()
168
169         cv2.putText(annotated_frame, f"Format: 720p HD (16:9)", (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
170         cv2.putText(annotated_frame, f"Cibles: {nb_personnes}", (20, 70),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
171         cv2.putText(annotated_frame, f"System FPS: {fps_affiche:.1f}", (20,
                100), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
172         # Horloge en millisecondes : utile pour mesurer la latence jusqu'au
                voyant
173         # en filmant au ralenti l'écran du PC et l'ESP32 dans le même plan.
174         horloge_ms = int((time.perf_counter() - debut_session) * 1000)
175         cv2.putText(annotated_frame, f"t = {horloge_ms} ms", (20, 130),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
176
177         out.write(annotated_frame)
178         cv2.imshow("Systeme de Surveillance M2R - Edge AI", annotated_frame)
179         touche = cv2.waitKey(1) & 0xFF
180         t4 = time.perf_counter() # fin annotation + écriture vidéo +
                affichage
181
182         #  Enregistrement des mesures de cette trame (boucle complète t0 ->
                t4)
183         total_ms = (t4 - t0) * 1000.0
184         fps_trame = 1000.0 / total_ms if total_ms > 0 else 0.0
185         index_trame += 1
186         if index_trame > TRAMES_ECHAUFFEMENT:
187             m = {
188                 "capture": (t1 - t0) * 1000.0,
189                 "track": (t2 - t1) * 1000.0,
190                 "envoi_log": (t3 - t2) * 1000.0,
191                 "affichage": (t4 - t3) * 1000.0,
192                 "total": total_ms,
193                 "fps": fps_trame,
194             }
195             mesures.append(m)
196             ecrivain.writerow([index_trame, round(t4 - debut_session, 3),
                    nb_personnes,
```

```
                                round(m["capture"], 2), round(m["track"], 2),
                                round(m["envoi_log"], 2), round(m["affichage"],
                                    2),
                                round(total_ms, 2), round(fps_trame, 2)])

        # Gestion de la touche 'q'
        if DUREE_MESURE_S > 0 and (t4 - debut_session) >= DUREE_MESURE_S:
            print(f"Durée de mesure atteinte ({DUREE_MESURE_S:.0f} s).")
            touche = ord('q')
        if touche == ord('q'):
            # Envoi du signal de fermeture àl'ESP32
            try:
                sock.sendto(b"QUIT", (ESP32_IP, UDP_PORT))
                print("Signal 'QUIT' envoyé àl'ESP32.")
            except Exception as e:
                print(f"Erreur lors de l'envoi du signal : {e}")
            break

duree_session = time.perf_counter() - debut_session

# Libération des ressources
cap.release()
out.release()
sock.close()
cv2.destroyAllWindows()

#  Résumé statistique des mesures
if len(mesures) < 2:
    print("\n Pas assez de trames mesurées pour calculer des statistiques "
          f"(il faut plus de {TRAMES_ECHAUFFEMENT} trames).")
else:
    resume = {
        "date": horodatage_base,
        "duree_session_s": round(duree_session, 1),
        "trames_mesurees": len(mesures),
        "trames_echauffement_ignorees": TRAMES_ECHAUFFEMENT,
        "resolution": f"{frame_width}x{frame_height}",
        "seuil_confiance": 0.55,
        "liaison": "Wi-Fi UDP",
        "fps_bout_en_bout": resume_statistique([m["fps"] for m in mesures]),
        "temps_total_ms": resume_statistique([m["total"] for m in mesures]),
```

```
237             "capture_ms": resume_statistique([m["capture"] for m in mesures]),
238             "detection_suivi_ms": resume_statistique([m["track"] for m in
                    mesures]),
239             "envoi_udp_journal_ms": resume_statistique([m["envoi_log"] for m in
                    mesures]),
240             "annotation_video_affichage_ms": resume_statistique([m["affichage"]
                    for m in mesures]),
241         }
242         with open(nom_fichier_resume, mode='w', encoding='utf-8') as f:
243             json.dump(resume, f, indent=2, ensure_ascii=False)
244
245         fps = resume["fps_bout_en_bout"]
246         print("\n RÉSUMÉ DES MESURES (à reporter dans l'article)")
247         print(f"Durée : {resume['duree_session_s']} s, trames mesurées :
                {resume['trames_mesurees']}")
248         print(f"FPS bout en bout : moyenne {fps['moyenne']}, écart-type
                {fps['ecart_type']}, "
249               f"min {fps['min']}, 5e percentile {fps['p5']}")
250         for cle, nom in [("capture_ms", "Capture"), ("detection_suivi_ms",
                "Détection + suivi"),
251                          ("envoi_udp_journal_ms", "Envoi UDP + journal"),
252                          ("annotation_video_affichage_ms", "Annotation + vidéo +
                             affichage"),
253                          ("temps_total_ms", "TOTAL par trame")]:
254             s = resume[cle]
255             print(f"{nom:<32}: moyenne {s['moyenne']} ms, médiane
                    {s['mediane']} ms, p95 {s['p95']} ms")
256         print(f"Résumé complet enregistré dans : {nom_fichier_resume}")
257
258     print(f"\n Session terminée. Fichiers sauvegardés.")
259
260
261 if __name__ == '__main__':
262     CHEMIN_POIDS = "runs/detect/run_production_yolov8n/weights/best.pt"
263     SOURCE_VIDEO = 0
264     executer_inference_suivi_enregistrement(CHEMIN_POIDS, SOURCE_VIDEO)
```

ANNEXE B

SCHÉMAS TECHNIQUES DU SYSTÈME DE SURVEILLANCE

B.1 Architecture matérielle du nœud d'alerte

L'architecture du système est présentée à la figure 2 et la chaîne d'alimentation du nœud d'alerte à la figure 5. Le tableau 6 récapitule les connexions de la carte ESP32-WROOM-32.

TABLE 6 – Affectation des broches de la carte de développement ESP32 utilisée pour les essais

Broche	Signal	Fonction
V5	5 V (USB-C)	entrée du régulateur AMS1117
3V3	3,3 V	alimentation de l'écran OLED
GPIO21	SDA	données I^2C de l'écran OLED (adresse `0x3C`)
GPIO22	SCL	horloge I^2C de l'écran OLED
GPIO1	voyant d'alarme	aussi TX0 de l'UART0 : à déplacer (par ex. GPIO2)
GPIO1/3	TX0 / RX0	liaison série USB par le CP2102 (115 200 bauds)
EN, IO0	redémarrage automatique	commandés par DTR et RTS du CP2102

B.2 Carte dédiée du nœud d'alerte

Les essais du chapitre III ont été réalisés avec la carte de développement décrite ci-dessus, alimentée par le bus USB. Pour rendre le nœud d'alerte autonome, une carte dédiée a ensuite été conçue avec KiCad (projet `Surveillance-EdgeAI-YOLOv8n-ESP32`). Elle reprend l'affectation des broches du tableau 6 et ajoute la charge et la protection d'une batterie LiPo, un convertisseur élévateur MT3608 et un régulateur AMS1117 ajustable. Dans sa version finale (V1.4), c'est une carte à quatre couches de 58×50 mm, avec un plan de masse et un plan 3,3 V. Elle a été conçue et vérifiée, mais pas encore fabriquée. Son schéma complet (figure 19), le rôle de chaque bloc et le circuit imprimé (figures 20 à 24) sont détaillés à l'annexe C.

B.3 Diagramme des flux de données et micrologiciel

Le micrologiciel de l'ESP32 (figure 4b) est développé avec PlatformIO (PlatformIO, 2026) et le cadre Arduino, qui s'exécute sur le noyau temps réel FreeRTOS (Barry, 2016), et utilise les bibliothèques Adafruit SSD1306 et Adafruit GFX. Il reçoit sur le port UDP 4210 le nombre n de personnes détectées par le poste d'inférence et met à jour l'écran et le voyant. Le listing ci-dessous correspond à la seconde version, organisée en machine à états avec une interface graphique animée (figure 14). Les identifiants Wi-Fi y ont été masqués. Ce code est également disponible dans le dépôt public de l'auteur (Kengne Takam, 2026b).

Listing B.1 – Micrologiciel du nœud d'alerte ESP32, liaison Wi-Fi (`main_WIFI.cpp`)

```
1 // ============================================================
2 // Noeud d'alerte Edge AI - ESP32 + OLED SSD1306 128x64 - liaison Wi-Fi UDP
```

```
// Interface graphique animée : écran d'accueil, connexion, attente (radar),
// zone RAS (bouclier), intrusion (alerte clignotante), perte Wi-Fi, fin.
// ========================================================
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <WiFi.h>
#include <WiFiUdp.h>
#include <math.h>

// VOS IDENTIFIANTS WI-FI
const char* ssid = "VOTRE_SSID";
const char* password = "VOTRE_MOT_DE_PASSE";

// Port UDP identique àcelui configuré sous Python
unsigned int localUdpPort = 4210;
char incomingPacket[255]; // Buffer pour stocker le paquet entrant

WiFiUDP udp;

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define LED_PIN 1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// --- États de l'interface
    ----------------------------------------------------
enum Etat { ETAT_ATTENTE, ETAT_RAS, ETAT_INTRUSION, ETAT_FIN, ETAT_PERTE_WIFI
    };
Etat etat = ETAT_ATTENTE;
int nbCibles = 0;
unsigned long dernierPaquetMs = 0; // réception du dernier paquet UDP
unsigned long dernierRenduMs = 0; // dernier rafraîchissement de l'OLED
unsigned long dernieretentativeMs = 0; // dernière tentative de reconnexion
uint16_t image = 0;                // compteur d'animation

const unsigned long PERIODE_RENDU_MS = 90; // ~11 images/s pour les animations
const unsigned long DELAI_SANS_FLUX_MS = 3000; // plus de paquet -> écran
    d'attente
```

```
41
42 // ============================================================
43 //  Petits outils de dessin
44 // ============================================================
45
46 // Texte centré horizontalement (police 6x8 de base, multipliée par la taille)
47 void texteCentre(const char* txt, int16_t y, uint8_t taille, uint16_t couleur
       = SSD1306_WHITE) {
48   int16_t w = strlen(txt) * 6 * taille;
49   display.setTextSize(taille);
50   display.setTextColor(couleur);
51   display.setCursor((SCREEN_WIDTH - w) / 2, y);
52   display.print(txt);
53 }
54
55 // Texte centré dans une zone [x0, x0+largeur]
56 void texteCentreZone(const char* txt, int16_t x0, int16_t largeur, int16_t y,
       uint8_t taille,
57                      uint16_t couleur = SSD1306_WHITE) {
58   int16_t w = strlen(txt) * 6 * taille;
59   display.setTextSize(taille);
60   display.setTextColor(couleur);
61   display.setCursor(x0 + (largeur - w) / 2, y);
62   display.print(txt);
63 }
64
65 // Icône Wi-Fi : trois arcs + point (niveau = nombre d'arcs affichés, 0 à3)
66 void iconeWifi(int16_t cx, int16_t cy, uint8_t niveau, uint16_t couleur) {
67   display.fillCircle(cx, cy, 1, couleur);
68   for (uint8_t i = 1; i <= 3; i++) {
69     if (i <= niveau) {
70       display.drawCircleHelper(cx, cy, 4 * i + 2, 0x1 | 0x2, couleur);
71       display.drawCircleHelper(cx, cy, 4 * i + 1, 0x1 | 0x2, couleur);
72     }
73   }
74 }
75
76 // Barres de réception (RSSI) : 4 barres de hauteur croissante
77 void barresSignal(int16_t x, int16_t y, uint16_t couleur) {
78   long rssi = WiFi.RSSI();
79   uint8_t n = rssi > -55 ? 4 : rssi > -65 ? 3 : rssi > -75 ? 2 : rssi > -85 ?
```

```
      1 : 0;
  for (uint8_t i = 0; i < 4; i++) {
    int16_t h = 2 + 2 * i;
    if (i < n) display.fillRect(x + i * 3, y + 8 - h, 2, h, couleur);
    else       display.drawPixel(x + i * 3, y + 7, couleur);
  }
}

// Cadre général : bord arrondi + bandeau de titre en négatif
void cadreAvecTitre(const char* titre) {
  display.drawRoundRect(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT, 5, SSD1306_WHITE);
  display.fillRoundRect(0, 0, SCREEN_WIDTH, 13, 5, SSD1306_WHITE);
  display.fillRect(0, 8, SCREEN_WIDTH, 5, SSD1306_WHITE);
  texteCentre(titre, 3, 1, SSD1306_BLACK);
}

// Bandeau "EDGE AI" avec indicateur de signal àdroite
void cadreEdgeAI() {
  display.drawRoundRect(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT, 5, SSD1306_WHITE);
  display.fillRoundRect(0, 0, SCREEN_WIDTH, 13, 5, SSD1306_WHITE);
  display.fillRect(0, 8, SCREEN_WIDTH, 5, SSD1306_WHITE);
  // Petit oeil (caméra) àgauche
  display.fillCircle(9, 6, 4, SSD1306_BLACK);
  display.fillCircle(9, 6, 2, SSD1306_WHITE);
  display.drawPixel(9, 6, SSD1306_BLACK);
  display.setTextSize(1);
  display.setTextColor(SSD1306_BLACK);
  display.setCursor(18, 3);
  display.print("EDGE AI WI-FI");
  barresSignal(110, 2, SSD1306_BLACK);
}

// Coins décoratifs (style viseur de caméra) dans un rectangle
void coinsViseur(int16_t x, int16_t y, int16_t w, int16_t h, int16_t l,
    uint16_t c) {
  display.drawFastHLine(x, y, l, c);       display.drawFastVLine(x, y, l, c);
  display.drawFastHLine(x + w - l, y, l, c); display.drawFastVLine(x + w - 1,
      y, l, c);
  display.drawFastHLine(x, y + h - 1, l, c); display.drawFastVLine(x, y + h -
      l, l, c);
  display.drawFastHLine(x + w - l, y + h - 1, l, c); display.drawFastVLine(x +
```

```
      w - 1, y + h - l, l, c);
}

// Coche (validation)
void coche(int16_t x, int16_t y, uint16_t c) {
  for (int8_t e = 0; e < 2; e++) {
    display.drawLine(x, y + 4 + e, x + 3, y + 7 + e, c);
    display.drawLine(x + 3, y + 7 + e, x + 9, y + e, c);
  }
}

// Bouclier avec coche : symbole "zone sécurisée"
void bouclier(int16_t cx, int16_t y, uint16_t c) {
  display.fillRoundRect(cx - 11, y, 23, 16, 3, c);
  display.fillTriangle(cx - 11, y + 14, cx + 11, y + 14, cx, y + 26, c);
  coche(cx - 5, y + 6, SSD1306_BLACK);
}

// Triangle d'avertissement avec point d'exclamation
void triangleAlerte(int16_t cx, int16_t y, uint16_t c, uint16_t fond) {
  display.fillTriangle(cx, y, cx - 15, y + 26, cx + 15, y + 26, c);
  display.fillTriangle(cx, y + 6, cx - 10, y + 23, cx + 10, y + 23, fond);
  display.fillRect(cx - 1, y + 10, 3, 8, c);
  display.fillRect(cx - 1, y + 20, 3, 2, c);
}

// ============================================================
//  Écrans
// ============================================================

// Écran d'accueil animé (au démarrage)
void ecranAccueil() {
  for (int p = 0; p <= 100; p += 4) {
    display.clearDisplay();
    display.drawRoundRect(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT, 6, SSD1306_WHITE);
    display.drawRoundRect(2, 2, SCREEN_WIDTH - 4, SCREEN_HEIGHT - 4, 5,
        SSD1306_WHITE);
    // Oeil / objectif de caméra
    display.fillCircle(24, 26, 13, SSD1306_WHITE);
    display.fillCircle(24, 26, 9, SSD1306_BLACK);
    display.fillCircle(24, 26, 5, SSD1306_WHITE);
```

```
    display.fillCircle(22, 24, 1, SSD1306_BLACK);
    // Titre
    display.setTextColor(SSD1306_WHITE);
    display.setTextSize(2);
    display.setCursor(46, 12);
    display.print("EDGE");
    display.setCursor(46, 30);
    display.print("AI");
    display.setTextSize(1);
    display.setCursor(74, 34);
    display.print("YOLOv8n");
    // Barre de progression
    display.drawRoundRect(10, 47, 108, 9, 3, SSD1306_WHITE);
    display.fillRoundRect(12, 49, (104 * p) / 100, 5, 2, SSD1306_WHITE);
    display.display();
    delay(30);
  }
  delay(400);
}

// Connexion en cours (avec diagnostic)
void ecranConnexion(unsigned long ecoule, wl_status_t st) {
  display.clearDisplay();
  cadreAvecTitre("CONNEXION WI-FI");
  iconeWifi(19, 46, (ecoule / 400) % 4, SSD1306_WHITE);
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(38, 18);
  display.print("Reseau Wi-Fi");
  if (st == WL_NO_SSID_AVAIL) {
    display.setCursor(38, 30); display.print("Introuvable:");
    display.setCursor(38, 40); display.print("activer la");
    display.setCursor(38, 50); display.print("bande 2,4 GHz");
  } else if (st == WL_CONNECT_FAILED) {
    display.setCursor(38, 30); display.print("Echec : mot");
    display.setCursor(38, 40); display.print("de passe ou");
    display.setCursor(38, 50); display.print("WPA3 -> WPA2");
  } else {
    display.setCursor(38, 32);
    display.print("Recherche");
    for (uint8_t i = 0; i < (ecoule / 300) % 4; i++) display.print('.');
```

```
197     display.setCursor(38, 46);
198     display.print(ecoule / 1000);
199     display.print(" s");
200   }
201   display.display();
202 }
203
204 // Connexion réussie
205 void ecranConnecte() {
206   display.clearDisplay();
207   cadreAvecTitre("WI-FI CONNECTE");
208   display.fillCircle(20, 29, 10, SSD1306_WHITE);
209   coche(15, 24, SSD1306_BLACK);
210   display.setTextSize(1);
211   display.setTextColor(SSD1306_WHITE);
212   display.setCursor(38, 19);
213   display.print("Adresse IP");
214   display.setCursor(38, 30);
215   display.print("Port UDP ");
216   display.print(localUdpPort);
217   display.drawFastHLine(6, 42, SCREEN_WIDTH - 12, SSD1306_WHITE);
218   texteCentre(WiFi.localIP().toString().c_str(), 48, 1);
219   display.display();
220 }
221
222 // Attente du flux vidéo : radar animé
223 void ecranAttente() {
224   display.clearDisplay();
225   cadreEdgeAI();
226   const int16_t cx = 25, cy = 38, r = 20;
227   display.drawCircle(cx, cy, r, SSD1306_WHITE);
228   display.drawCircle(cx, cy, r / 2, SSD1306_WHITE);
229   display.drawFastHLine(cx - r, cy, 2 * r + 1, SSD1306_WHITE);
230   display.drawFastVLine(cx, cy - r, 2 * r + 1, SSD1306_WHITE);
231   float a = image * 0.35f;
232   for (int8_t k = 0; k < 3; k++) { // faisceau avec une petite traînée
233     float b = a - k * 0.12f;
234     display.drawLine(cx, cy, cx + (int16_t)(r * cosf(b)), cy + (int16_t)(r *
          sinf(b)), SSD1306_WHITE);
235   }
236   display.setTextSize(1);
```

```
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(54, 19); display.print("En attente");
  display.setCursor(54, 29); display.print("du flux");
  display.setCursor(54, 39); display.print("video");
  for (uint8_t i = 0; i < (image / 4) % 4; i++) display.print('.');
  display.setCursor(54, 52); display.print("UDP ");
  display.print(localUdpPort);
  display.display();
}

// Zone RAS : bouclier + pulsation "système actif"
void ecranRAS() {
  display.clearDisplay();
  cadreEdgeAI();
  bouclier(22, 19, SSD1306_WHITE);
  texteCentreZone("ZONE", 42, 84, 18, 2);
  texteCentreZone("RAS", 42, 84, 36, 2);
  // Point "vivant" qui pulse dans le coin bas gauche
  uint8_t r = (image / 4) % 2 ? 2 : 1;
  display.fillCircle(8, 56, r, SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(14, 53);
  display.print("actif");
  display.display();
}

// Intrusion : alerte clignotante en négatif
void ecranIntrusion() {
  bool negatif = (image / 3) % 2; // alterne toutes les ~270 ms
  uint16_t fg = negatif ? SSD1306_BLACK : SSD1306_WHITE;
  uint16_t bg = negatif ? SSD1306_WHITE : SSD1306_BLACK;
  display.fillScreen(bg);
  display.drawRoundRect(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT, 5, fg);
  display.drawRoundRect(2, 2, SCREEN_WIDTH - 4, SCREEN_HEIGHT - 4, 4, fg);
  texteCentre("! INTRUSION !", 6, 1, fg);
  display.drawFastHLine(8, 16, SCREEN_WIDTH - 16, fg);
  triangleAlerte(24, 22, fg, bg);
  // Nombre de personnes, dans un viseur
  coinsViseur(54, 20, 64, 38, 6, fg);
  char nb[6];
  snprintf(nb, sizeof(nb), "%d", nbCibles);
```

```
  texteCentreZone(nb, 54, 64, 24, 3, fg);
  texteCentreZone(nbCibles > 1 ? "personnes" : "personne", 54, 64, 46, 1, fg);
  display.display();
}

// Fin de session
void ecranFin() {
  display.clearDisplay();
  cadreAvecTitre("EDGE AI WI-FI");
  display.drawRoundRect(10, 19, 108, 38, 6, SSD1306_WHITE);
  texteCentre("SESSION", 24, 1);
  texteCentre("TERMINEE", 34, 1);
  texteCentre("Au revoir, Prof!", 46, 1);
  display.display();
}

// Perte du Wi-Fi : icône barrée
void ecranPerteWifi() {
  display.clearDisplay();
  cadreAvecTitre("PERTE WI-FI");
  iconeWifi(22, 46, 3, SSD1306_WHITE);
  display.drawLine(8, 22, 36, 50, SSD1306_WHITE);
  display.drawLine(9, 22, 37, 50, SSD1306_WHITE);
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(46, 22); display.print("Connexion");
  display.setCursor(46, 32); display.print("perdue");
  display.setCursor(46, 46); display.print("Reconnexion");
  display.setCursor(46, 55);
  for (uint8_t i = 0; i < (image / 3) % 4; i++) display.print(". ");
  display.display();
}

void dessiner() {
  switch (etat) {
    case ETAT_ATTENTE: ecranAttente(); break;
    case ETAT_RAS:     ecranRAS();     break;
    case ETAT_INTRUSION: ecranIntrusion(); break;
    case ETAT_FIN:     ecranFin();     break;
    case ETAT_PERTE_WIFI: ecranPerteWifi(); break;
  }
```

```
319   dernierRenduMs = millis();
320 }
321
322 // ========================================================
323 void setup() {
324   Serial.begin(115200);
325   pinMode(LED_PIN, OUTPUT);
326   display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
327
328   ecranAccueil();
329
330   // Mode station seul + pas de mise en veille radio (plus stable sur un point
          d'accès de téléphone)
331   WiFi.mode(WIFI_STA);
332   WiFi.setSleep(false);
333   WiFi.begin(ssid, password);
334   unsigned long debut = millis();
335   unsigned long debutAffiche = millis();
336   while (WiFi.status() != WL_CONNECTED) {
337     delay(150);
338     ecranConnexion(millis() - debutAffiche, WiFi.status());
339     // Nouvelle tentative toutes les 15 s
340     if (millis() - debut > 15000) {
341       WiFi.disconnect();
342       delay(200);
343       WiFi.begin(ssid, password);
344       debut = millis();
345     }
346   }
347
348   Serial.print("Connecté ! Adresse IP : ");
349   Serial.println(WiFi.localIP());
350   ecranConnecte();
351   delay(2500);
352
353   // Démarrer l'écoute du port UDP
354   udp.begin(localUdpPort);
355   Serial.printf("Écoute active sur le port UDP %d\n", localUdpPort);
356   etat = ETAT_ATTENTE;
357   dessiner();
358 }
```

```
359
360 void loop() {
361   unsigned long maintenant = millis();
362
363   // 1. Vérification de la connexion Wi-Fi
364   if (WiFi.status() != WL_CONNECTED) {
365     if (etat != ETAT_PERTE_WIFI) {
366       etat = ETAT_PERTE_WIFI;
367       digitalWrite(LED_PIN, LOW);
368       WiFi.begin(ssid, password);
369       derniereTentativeMs = maintenant;
370     } else if (maintenant - derniereTentativeMs > 8000) {
371       // Tentative de reconnexion automatique
372       WiFi.disconnect();
373       WiFi.begin(ssid, password);
374       derniereTentativeMs = maintenant;
375     }
376     if (maintenant - dernierRenduMs >= PERIODE_RENDU_MS) { image++;
          dessiner(); }
377     return; // On attend la reconnexion avant de continuer
378   }
379   if (etat == ETAT_PERTE_WIFI) {
380     etat = ETAT_ATTENTE;
381     dessiner();
382   }
383
384   // 2. Gestion des paquets UDP : on lit tous les paquets en file et on garde
       le plus récent
385   bool nouveau = false;
386   int packetSize;
387   while ((packetSize = udp.parsePacket()) > 0) {
388     int len = udp.read(incomingPacket, sizeof(incomingPacket) - 1);
389     incomingPacket[len > 0 ? len : 0] = 0;
390     nouveau = true;
391   }
392
393   if (nouveau) {
394     dernierPaquetMs = maintenant;
395     Etat precedent = etat;
396     int precedentNb = nbCibles;
397     if (strcmp(incomingPacket, "QUIT") == 0) {
```

```
398       etat = ETAT_FIN;
399       digitalWrite(LED_PIN, LOW);
400     } else {
401       nbCibles = atoi(incomingPacket); // Conversion du texte brut en entier
402       if (nbCibles > 0) { etat = ETAT_INTRUSION; digitalWrite(LED_PIN, HIGH); }
403       else              { etat = ETAT_RAS;    digitalWrite(LED_PIN, LOW); }
404     }
405     // Mise àjour instantanée de l'OLED dès que la situation change
406     if (etat != precedent || nbCibles != precedentNb) dessiner();
407   } else if ((etat == ETAT_RAS || etat == ETAT_INTRUSION) &&
408              maintenant - dernierPaquetMs > DELAI_SANS_FLUX_MS) {
409     // Plus de flux depuis le PC : retour àl'écran d'attente
410     etat = ETAT_ATTENTE;
411     digitalWrite(LED_PIN, LOW);
412     dessiner();
413   }
414
415   // 3. Animations (radar, pulsation, clignotement de l'alerte)
416   if (etat != ETAT_FIN && maintenant - dernierRenduMs >= PERIODE_RENDU_MS) {
417     image++;
418     dessiner();
419   }
420 }
```

ANNEXE C

CONCEPTION ÉLECTRONIQUE DÉTAILLÉE DU NŒUD D'ALERTE

Cette annexe détaille la conception électronique du nœud d'alerte autonome dont l'architecture a été présentée en annexe B. Le schéma a été saisi avec KiCad (KiCad Developers, s. d.) ; il comprend quatre blocs fonctionnels :

1. l'étage de charge et de protection de la batterie Li-ion/LiPo;
2. le convertisseur continu-continu élévateur (*boost*) MT3608;
3. le régulateur linéaire ajustable AMS1117;
4. l'étage de calcul autour du module ESP32-WROOM-32.

Chaque bloc est d'abord isolé du schéma général puis expliqué composant par composant, avec les relations de dimensionnement utilisées. Le schéma complet est ensuite donné à la figure 19. La dernière partie décrit le circuit imprimé final (version 1.4), passé d'une carte double face de 100×50 mm à une carte quatre couches de 58×50 mm, et en présente les vues de routage et les rendus en trois dimensions.

C.1 Vue d'ensemble de la chaîne d'énergie

La carte de développement utilisée pour les essais (chapitre II) était alimentée par le bus USB. Pour rendre le nœud d'alerte autonome, la carte dédiée est alimentée par une batterie lithium-ion polymère (LiPo) d'une cellule, rechargée par le connecteur USB-C. Le parcours de l'énergie est le suivant :

USB-C (5 V) → chargeur MCP73831 → batterie LiPo (3,0 V à 4,2 V) protégée par DW01A + FS8205A → rail `+VBAT` → convertisseur élévateur MT3608 → rail `+OUT` (réglé vers 5 V) → régulateur AMS1117 → rail `+3.3V` → ESP32, écran OLED et voyant.

Le passage par un convertisseur élévateur s'explique par la tension de déchet de l'AMS1117, environ 1,1 V à pleine charge (Advanced Monolithic Systems, s. d.). Le régulateur a donc besoin d'au moins 4,4 V environ en entrée pour délivrer 3,3 V. Or la tension d'une cellule LiPo varie de 4,2 V (pleine charge) à environ 3,0 V (fin de décharge) : elle ne suffit pas à elle seule. Le MT3608 relève donc la tension de la batterie à une valeur fixe et confortable (typiquement 5 V), et l'AMS1117 en dérive un 3,3 V propre et peu bruité pour le module radio. Cette solution à deux étages n'est pas la plus économe en énergie : le rendement global est le produit du rendement du convertisseur (environ 90 %) et de celui du régulateur linéaire ($3{,}3/5 \approx 66\,\%$). En revanche, elle reprend le régulateur AMS1117 déjà validé sur la carte d'essai, filtre le bruit de découpage avant l'ESP32 et laisse disponible un rail `+OUT` réglable, utilisable pour d'autres capteurs.

Les quatre rails de la carte et leurs classes de pistes sont résumés dans le tableau 7.

TABLE 7 – Rails d'alimentation du nœud d'alerte et classes de pistes associées sur le circuit imprimé.

Rail	Source	Plage	Classe (piste / jeu)
`VBUS` (`Net-(D1-A)`)	connecteur USB-C J1	5 V	USB (0,50 mm / 0,20 mm)
`+VBAT`, `-VBAT`	batterie LiPo1	3,0 V à 4,2 V	PWR_BAT (0,80 mm / 0,25 mm)
`+OUT`	convertisseur MT3608	4,2 V à 28 V (réglé à 5 V)	PWR_BAT (0,80 mm / 0,25 mm)
`+3.3V`	régulateur AMS1117	3,3 V	PWR_3V3 (0,60 mm / 0,25 mm)

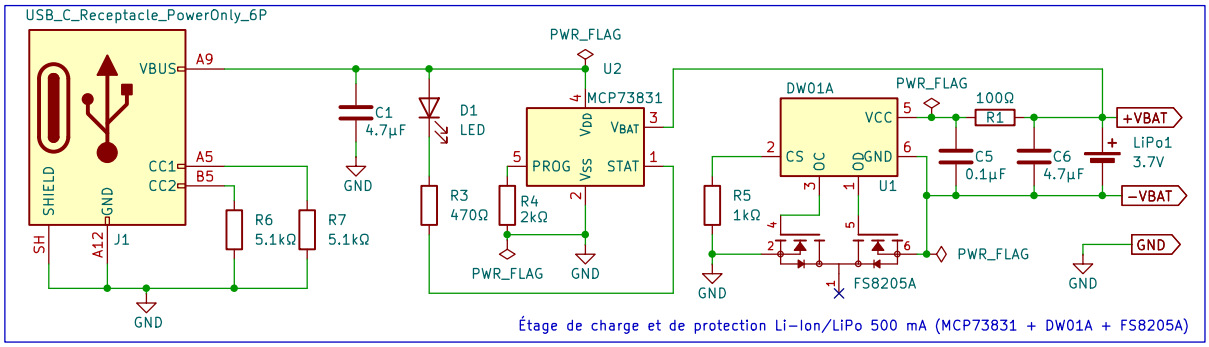


FIGURE 15 – Bloc 1 extrait du schéma KiCad : connecteur USB-C J1 (alimentation seule), chargeur Li-ion MCP73831 (U2) programmé à 500 mA, voyant de charge D1, circuit de protection DW01A (U1) commandant le double transistor FS8205A, et batterie LiPo1 de 3,7 V.

C.2 Bloc 1 : charge et protection de la batterie

Le bloc de la figure 15 recharge la batterie à partir de l'USB-C et la protège contre les surcharges, les décharges profondes, les surintensités et les courts-circuits.

C.2.1 Connecteur USB-C et négociation de la source

J1 est un réceptacle USB-C à six contacts « alimentation seule » : il ne porte que le bus `VBUS` (contacts A9 et B9), la masse (A12, B12 et blindage) et les deux lignes de configuration CC1 et CC2. Une source USB-C ne fournit sa tension que si elle détecte un récepteur. Ce récepteur se signale par une résistance de tirage $R_d = 5{,}1\,\text{k}\Omega$ vers la masse sur chaque ligne CC (USB Implementers Forum, s. d.) ; ce sont les résistances R6 et R7. Sans elles, un chargeur USB-C conforme laisserait `VBUS` à 0 V. Le condensateur C1 (4,7 µF) découple l'entrée du chargeur au plus près du connecteur.

C.2.2 Chargeur linéaire MCP73831

Le MCP73831 (U2) est un chargeur linéaire à courant constant puis tension constante (CC/CV) pour une cellule Li-ion (Microchip Technology, s. d.). Sa tension de fin de charge vaut 4,20 V (version -2). Le courant de charge en phase CC est fixé par la résistance R4 placée entre

la broche PROG et la masse :

$$I_{\text{charge}} = \frac{1000\,\text{V}}{R_{\text{PROG}}} = \frac{1000}{2\,\text{k}\Omega} = 500\,\text{mA}. \tag{C.1}$$

Cette valeur correspond au courant disponible sur un port USB 2.0 standard. Pour une batterie de 1000 mA h, elle correspond à environ 0,5 C, ce qui donne une charge complète en un peu plus de deux heures (phase CV comprise). Le circuit étant linéaire, il dissipe $(V_{\text{BUS}} - V_{\text{BAT}}) \times I_{\text{charge}}$, soit au plus $(5 - 3{,}0) \times 0{,}5 = 1\,\text{W}$ en début de charge; le MCP73831 réduit alors automatiquement son courant grâce à sa régulation thermique.

La broche STAT est une sortie à trois états. Elle est tirée au niveau bas pendant la charge, ce qui allume la diode D1 alimentée depuis `VBUS` à travers R3 (470 Ω). Le courant du voyant vaut environ $(5 - 2{,}0)/470 \approx 6{,}4\,\text{mA}$. En fin de charge, STAT passe au niveau haut et la diode s'éteint.

C.2.3 Protection de la cellule : DW01A et FS8205A

Une cellule LiPo doit être protégée contre la surcharge (au-delà d'environ 4,3 V), la décharge profonde (en dessous d'environ 2,4 V) et les courts-circuits. Le DW01A (U1) (Fortune Semiconductor, s. d.[a]) surveille en permanence la tension de la cellule et commande deux transistors MOS à canal N montés en série dans le retour négatif. Ces deux transistors sont intégrés dans le FS8205A (Fortune Semiconductor, s. d.[b]), avec des drains communs.

— la sortie **OD** (*over-discharge*) bloque le transistor de décharge lorsque la tension de la cellule devient trop faible ou que le courant de décharge est excessif;

— la sortie **OC** (*over-charge*) bloque le transistor de charge lorsque la tension dépasse le seuil de surcharge;

— l'entrée **CS** mesure, à travers R5 (1 kΩ), la chute de tension aux bornes des transistors. Celle-ci est proportionnelle au courant, ce qui sert à la détection de surintensité et de court-circuit.

Le DW01A est alimenté depuis `+VBAT` par un filtre passe-bas R1 (100 Ω) – C5 (0,1 µF), de constante de temps 10 µs. Ce filtre évite les déclenchements intempestifs sur les appels de courant du convertisseur. La borne négative de la cellule est le rail `-VBAT`; la masse du système (`GND`) se trouve de l'autre côté des transistors. C6 (4,7 µF) découple la cellule, et le connecteur JST-PH LiPo1 permet de la remplacer sans soudure.

Point de vigilance. Les deux transistors du FS8205A sont parcourus par la totalité du courant de la batterie. Leur résistance à l'état passant (quelques dizaines de milliohms) fixe le seuil de surintensité. Sur le circuit imprimé, les pistes `-VBAT` et `GND` qui les relient doivent donc être larges : elles sont routées en 0,80 mm (classe PWR_BAT).

Deux références de masse. La borne négative de la batterie (`-VBAT`) n'est pas la masse du système (`GND`) : les deux sont séparées par les transistors du FS8205A. Toute mesure doit donc être référencée à `GND`. Il ne faut jamais relier directement `-VBAT` à `GND`, par exemple avec la pince de masse d'un oscilloscope alimenté par le secteur, car la protection serait alors court-circuitée.

Absence de partage de charge. Le convertisseur élévateur est alimenté directement par la batterie, y compris pendant la recharge. Le chargeur voit donc la somme du courant de la cellule et du courant consommé par la carte. Si cette consommation dépasse le seuil de fin de charge du MCP73831, la phase à tension constante ne se termine jamais et la cellule reste maintenue à 4,2 V. Ce fonctionnement reste sûr, mais il accélère le vieillissement de la batterie. Une prochaine version pourrait ajouter un circuit de partage de charge (*load sharing*) : un transistor MOS à canal P et une diode Schottky alimenteraient la carte directement depuis `VBUS` lorsqu'un câble est branché.

C.3 Bloc 2 : convertisseur élévateur MT3608

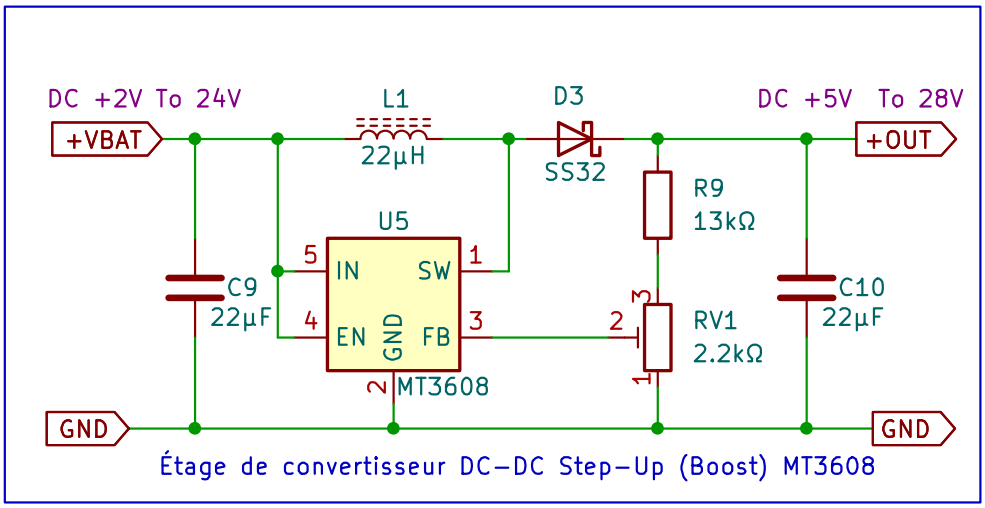


FIGURE 16 – Bloc 2 extrait du schéma KiCad : convertisseur élévateur MT3608 (U5) avec l'inductance L1 de 22 µH, la diode Schottky D3 (SS32), les condensateurs d'entrée C9 et de sortie C10 (22 µF) et le pont de contre-réaction R9 + RV1 qui fixe la tension du rail `+OUT`.

C.3.1 Principe de fonctionnement

Le MT3608 (U5) est un convertisseur élévateur à découpage. Il fonctionne à une fréquence fixe de 1,2 MHz et intègre un interrupteur de puissance; sa tension de référence est $V_{FB} = 0{,}6\,\text{V}$ (Aerosemi Technology, s. d.).

— Lorsque l'interrupteur interne est fermé, la broche SW est reliée à la masse et l'inductance L1 stocke de l'énergie depuis `+VBAT`.

— Lorsqu'il s'ouvre, la tension de SW monte jusqu'à ce que la diode D3 conduise, et l'énergie de L1 est transférée vers le condensateur de sortie C10.

En conduction continue et en négligeant les pertes, le rapport cyclique D vaut :

$$\frac{V_{\mathrm{OUT}}}{V_{\mathrm{IN}}} = \frac{1}{1-D} \quad \Longrightarrow \quad D = 1 - \frac{V_{\mathrm{IN}}}{V_{\mathrm{OUT}}}. \tag{C.2}$$

Les calculs de ce paragraphe reposent sur les hypothèses suivantes : $V_{\mathrm{IN}} = 3{,}7\,\mathrm{V}$ (tension nominale de la cellule), $V_{\mathrm{OUT}} = 5\,\mathrm{V}$, convertisseur idéal (tension de la diode négligée), et, pour le bilan de courant, une sortie de 500 mA avec un rendement $\eta = 0{,}9$. On obtient $D \approx 0{,}26$. Si l'on tient compte de la chute de tension d'environ 0,4 V dans la diode, $D = 1 - V_{\mathrm{IN}}/(V_{\mathrm{OUT}} + V_{\mathrm{D}})$ vaut 0,31 ; il atteint 0,44 en fin de décharge ($V_{\mathrm{IN}} = 3{,}0\,\mathrm{V}$). L'ondulation de courant crête à crête dans l'inductance vaut alors :

$$\Delta I_L = \frac{V_{\mathrm{IN}}\, D}{L\, f_{\mathrm{sw}}} = \frac{3{,}7 \times 0{,}26}{22\,\mu\mathrm{H} \times 1{,}2\,\mathrm{MHz}} \approx 36\,\mathrm{mA}, \tag{C.3}$$

soit environ 44 mA avec $D = 0{,}31$. Cette valeur reste faible devant le courant moyen d'entrée. Pour une puissance utile de 2,5 W (5 V, 500 mA) et un rendement de 90 %, le courant moyen absorbé sur la batterie vaut :

$$I_{\mathrm{IN}} = \frac{V_{\mathrm{OUT}}\, I_{\mathrm{OUT}}}{\eta\, V_{\mathrm{IN}}}, \tag{C.4}$$

soit environ 0,75 A à 3,7 V et 0,93 A à 3,0 V. C'est ce qui justifie des pistes `+VBAT` de 0,80 mm. Ce cas de 500 mA est volontairement pessimiste : en fonctionnement normal, la carte consomme plutôt 150 mA à 300 mA sur `+OUT`.

C.3.2 Composants de puissance

— **L1** (22 µH, Bourns SRN4018) : inductance blindée, dont le faible rayonnement est utile à proximité de l'antenne Wi-Fi de l'ESP32.

— **D3** (SS32) : diode Schottky de 3 A, 20 V. Sa faible tension de seuil (environ 0,4 V) et son temps de recouvrement quasi nul limitent les pertes à 1,2 MHz.

— **C9** et **C10** (22 µF) : condensateurs d'entrée et de sortie. La note portée sur le schéma recommande des condensateurs céramiques multicouches X5R/X7R pour obtenir une résistance série équivalente très faible, même si le symbole utilisé est celui d'un condensateur polarisé.

— La broche EN est reliée à IN : le convertisseur démarre dès que la batterie est connectée.

C.3.3 Réglage de la tension de sortie

La tension de sortie est fixée par le pont diviseur formé de R9 (13 kΩ) et du potentiomètre RV1 (2,2 kΩ), dont le curseur est relié à FB. En notant R_x la résistance entre le curseur et la

masse ($0 \le R_x \le 2{,}2\,\mathrm{k\Omega}$), la régulation impose $V_{\mathrm{FB}} = 0{,}6\,\mathrm{V}$, d'où :

$$V_{\mathrm{OUT}} = V_{\mathrm{FB}}\,\frac{R_9 + R_{\mathrm{RV1}}}{R_x} = 0{,}6 \times \frac{15{,}2\,\mathrm{k\Omega}}{R_x}. \tag{C.5}$$

Le tableau 8 donne quelques points de réglage.

TABLE 8 – Tension de sortie du MT3608 selon la position du curseur de RV1.

R_x (curseur–masse)	V_{OUT}	Commentaire
2,20 kΩ	4,15 V	butée : tension minimale
1,82 kΩ	5,0 V	point de fonctionnement retenu
1,00 kΩ	9,1 V	au-delà de la plage utile
0,33 kΩ	27,6 V	limite du MT3608 (28 V)

Risque de surtension. La tension de sortie croît très vite quand le curseur se rapproche de la masse. Un mauvais réglage de RV1 peut donc porter `+OUT` au-delà de la tension d'entrée maximale de l'AMS1117 (15 V) et de la tension nominale des condensateurs. Il faut régler RV1 à vide, avec un voltmètre sur `+OUT`, *avant* de raccorder le reste de la carte. Pour une version de série, une résistance fixe de 1,8 kΩ (série E12) entre FB et la masse, avec R9 (13 kΩ) seule comme résistance haute, donnerait $V_{\mathrm{OUT}} = 0{,}6 \times (1 + 13/1{,}8) \approx 4{,}93\,\mathrm{V}$ et supprimerait ce risque.

C.3.4 Contraintes de placement

La boucle parcourue par le courant découpé (C9 → L1 → SW → D3 → C10 → masse) rayonne d'autant plus que sa surface est grande. Sur le circuit imprimé final, l'ensemble des pastilles de cette boucle tient dans un rectangle de $5{,}95 \times 6{,}67$ mm. Le nœud SW ne comporte que 3,0 mm de cuivre. Le nœud FB, très sensible, est routé en face avant sans via et mesure 1,1 mm.

C.4 Bloc 3 : régulateur linéaire ajustable AMS1117

C.4.1 Calcul de la tension de sortie

L'AMS1117 (U4) en version ajustable maintient une tension de référence $V_{\mathrm{REF}} = 1{,}25\,\mathrm{V}$ entre sa sortie et sa broche ADJ (Advanced Monolithic Systems, s. d.). Avec R10 (240 Ω) entre la sortie et ADJ, et la résistance R_{RV2} du potentiomètre RV2 (câblé en résistance variable) entre ADJ et la masse :

$$V_{\mathrm{OUT}} = V_{\mathrm{REF}}\left(1 + \frac{R_{\mathrm{RV2}}}{R_{10}}\right) + I_{\mathrm{ADJ}}\,R_{\mathrm{RV2}} \approx 1{,}25\left(1 + \frac{R_{\mathrm{RV2}}}{240\,\Omega}\right). \tag{C.6}$$

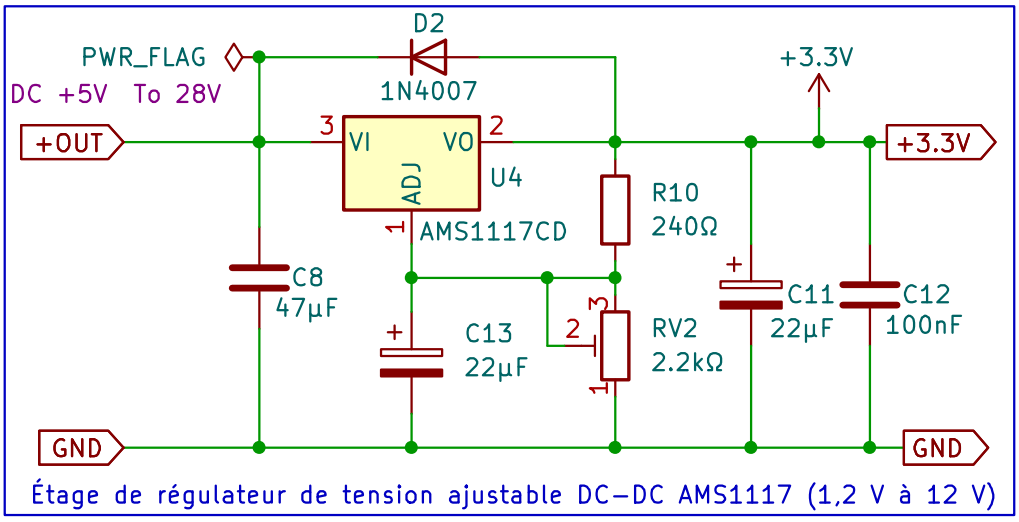


FIGURE 17 – Bloc 3 extrait du schéma KiCad : régulateur AMS1117 ajustable (U4) alimenté par le rail +OUT, pont de réglage R10 (240 Ω) + RV2 (2,2 kΩ), condensateur de broche ADJ C13, condensateurs d'entrée C8 et de sortie C11/C12, et diode de protection D2 (1N4007).

Le courant I_{ADJ} (quelques dizaines de microampères) est négligeable. La plage de réglage s'étend de 1,25 V ($R_{\mathrm{RV2}} = 0$) à environ 12,7 V ($R_{\mathrm{RV2}} = 2{,}2\,\mathrm{k\Omega}$), ce qui correspond à la mention « 1,2 V à 12 V » du schéma. Pour le rail de 3,3 V, il faut :

$$R_{\mathrm{RV2}} = R_{10}\left(\frac{V_{\mathrm{OUT}}}{V_{\mathrm{REF}}} - 1\right) = 240 \times \left(\frac{3{,}3}{1{,}25} - 1\right) \approx 394\,\Omega. \tag{C.7}$$

Le rail `+3.3V` alimente directement l'ESP32, dont la tension maximale est de 3,6 V (Espressif Systems, s. d.). D'après l'équation précédente, cette limite est atteinte pour $R_{\mathrm{RV2}} = 240 \times (3{,}6/1{,}25 - 1) \approx 450\,\Omega$, soit à peine 15 % au-dessus du réglage nominal. Le réglage de RV2 doit donc être fait avant la mise en place du module, ou avec une grande prudence. Une résistance fixe de 390 Ω (série E12) supprimerait ce risque dans une version de série : elle donne 3,28 V, ou 3,30 V en tenant compte de $I_{\mathrm{ADJ}} \approx 50\,\mu\mathrm{A}$.

C.4.2 Condensateurs et diode de protection

— **C8** (47 µF) découple l'entrée et absorbe les appels de courant de l'ESP32 vus à travers le régulateur.

— **C11** (22 µF) et **C12** (100 nF) assurent la stabilité de la boucle de régulation et le découplage haute fréquence de la sortie. La fiche technique de l'AMS1117 spécifie une capacité de sortie minimale de 22 µF.

— **C13** (22 µF) sur la broche ADJ améliore la réjection de l'ondulation résiduelle du convertisseur, car elle évite que cette ondulation soit amplifiée par le pont de réglage.

— **D2** (1N4007), anode sur la sortie et cathode sur l'entrée, décharge la sortie vers l'entrée si l'entrée est court-circuitée ou retombe plus vite que la sortie. Elle protège ainsi le

régulateur contre une polarisation inverse.

C.4.3 Dissipation thermique

Avec 5 V en entrée et une pointe de 250 mA (émission Wi-Fi), la puissance dissipée vaut $(5 - 3{,}3) \times 0{,}25 \approx 0{,}43$ W. Sur le circuit imprimé, le boîtier TO-252 dissipe cette puissance par son tab, relié à la sortie `+3.3V` :

— une zone de cuivre `+3.3V` de 112 mm^2 sur la face avant;

— neuf vias thermiques de 0,5 mm vers le plan `+3.3V` de la couche interne In2.

C.4.4 Bilan énergétique et autonomie

Le rendement global de la chaîne batterie $\rightarrow$ 5 V $\rightarrow$ 3,3 V est le produit du rendement du convertisseur élévateur et de celui du régulateur linéaire :

$$\eta_{\text{global}} = \eta_{\text{boost}} \times \frac{V_{3\text{V3}}}{V_{\text{OUT}}} \approx 0{,}9 \times \frac{3{,}3}{5{,}0} \approx 0{,}59. \tag{C.8}$$

Environ 40 % de l'énergie de la batterie est donc perdue en chaleur.

Prenons, à titre d'exemple, une consommation moyenne de 140 mA sur le rail 3,3 V (ESP32 connecté en Wi-Fi et écran allumé) et une batterie de 1000 mA h. Cette capacité n'est qu'un exemple : elle dépend de la cellule effectivement choisie.

— La puissance utile vaut $3{,}3 \times 0{,}14 \approx 0{,}46$ W.

— La puissance prélevée sur la batterie vaut $0{,}46/0{,}59 \approx 0{,}78$ W.

— L'énergie stockée vaut $3{,}7\ \text{V} \times 1\ \text{A h} = 3{,}7\ \text{W h}$.

— L'autonomie vaut donc $3{,}7/0{,}78 \approx 4{,}7$ h.

Un convertisseur abaisseur-élévateur (*buck-boost*) produisant directement 3,3 V, avec un rendement proche de 90 %, porterait cette autonomie à environ 7 h. Ces valeurs sont des estimations par le calcul : elles n'ont pas été mesurées sur la carte.

C.5 Bloc 4 : étage de calcul ESP32-WROOM-32

C.5.1 Module ESP32-WROOM-32 et découplage

Le module ESP32-WROOM-32 (U3) intègre le microcontrôleur double cœur, la mémoire flash, le quartz et l'antenne imprimée Wi-Fi/Bluetooth (Espressif Systems, s. d.). Il est alimenté par le rail `+3.3V` sur sa broche VDD (broche 2). Lors des émissions Wi-Fi, il appelle des pointes de courant de plusieurs centaines de milliampères pendant quelques microsecondes. Trois condensateurs de 100 nF (C2, C3, C4) découplent donc la broche d'alimentation; ils

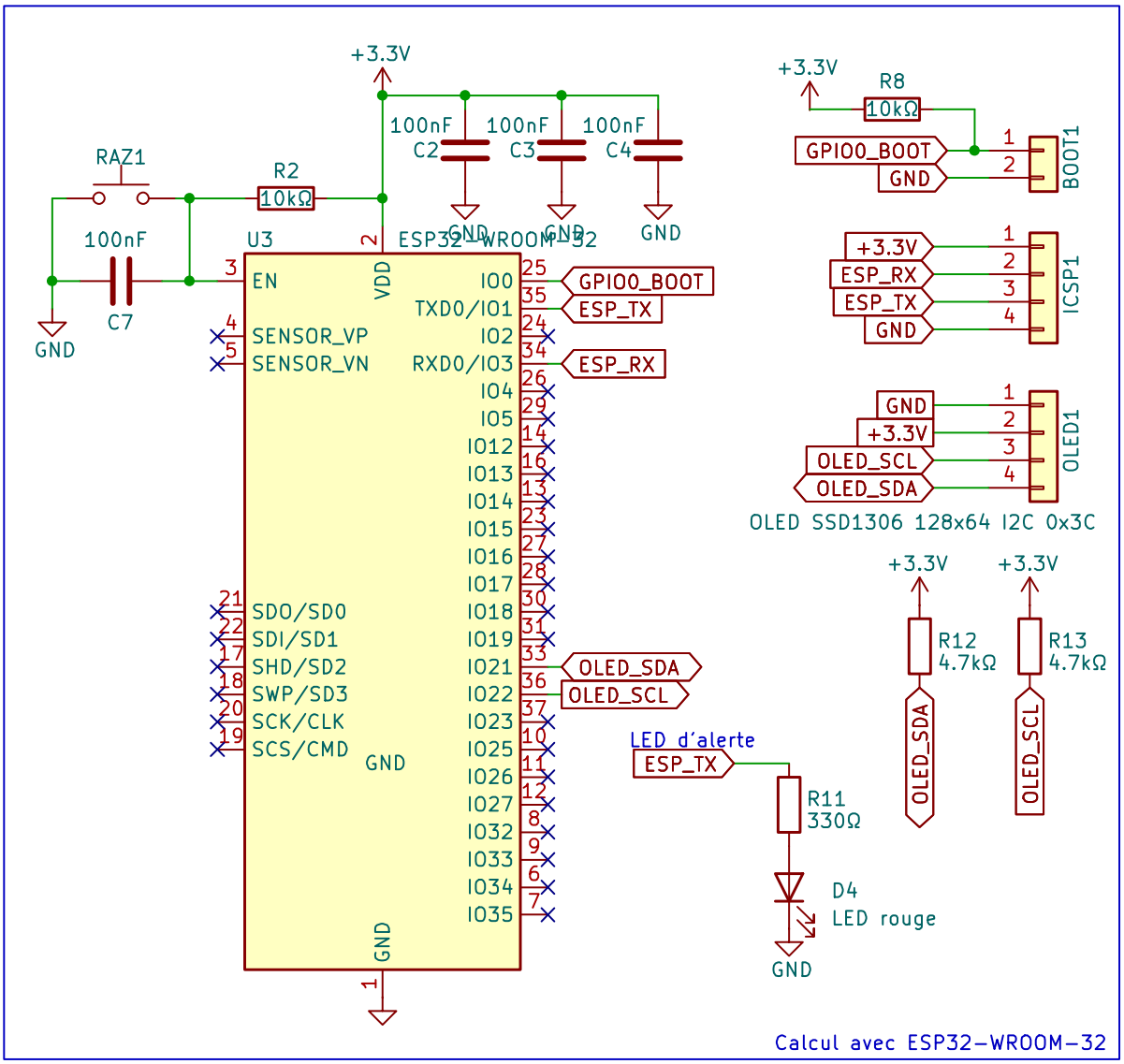


FIGURE 18 – Bloc 4 extrait du schéma KiCad : module ESP32-WROOM-32 (U3) et son découplage (C2, C3, C4), circuit de réinitialisation R2/C7/RAZ1, cavalier de démarrage BOOT1 avec R8, connecteur de programmation série ICSP1, connecteur de l'écran OLED SSD1306 (OLED1) avec les résistances de rappel I^2C R12 et R13, et voyant d'alerte D4 commandé par la ligne ESP_TX.

s'ajoutent aux condensateurs de sortie du régulateur. Sur le circuit imprimé, ces condensateurs sont placés à moins de 2,5 mm de la broche 2, sur les deux faces. Ils sont reliés aux plans d'alimentation par des vias courts.

C.5.2 Réinitialisation et mode de programmation

— **EN** (réinitialisation, actif au niveau bas) : la résistance R2 (10 kΩ) vers `+3.3V` et le condensateur C7 (100 nF) vers la masse forment un circuit RC de constante de temps $\tau = R_2 C_7 = 1$ ms. Ce délai retarde le démarrage jusqu'à ce que l'alimentation soit stable. Le bouton RAZ1 force EN à la masse pour redémarrer manuellement le module.

— **GPIO0** (sélection du mode de démarrage) : R8 (10 kΩ) maintient GPIO0 au niveau haut, ce qui correspond à un démarrage normal depuis la flash. Lorsque le cavalier BOOT1 est posé, GPIO0 est relié à la masse et l'ESP32 démarre en mode de téléchargement série au prochain appui sur RAZ1.

— **ICSP1** (connecteur de programmation à quatre broches : `+3.3V`, `ESP_RX`, `ESP_TX`, masse) : il reçoit un adaptateur USB-UART externe à niveaux logiques de 3,3 V. La ligne TX de l'adaptateur se connecte à `ESP_RX` (RXD0, GPIO3) et sa ligne RX à `ESP_TX` (TXD0, GPIO1). La carte n'embarque pas de convertisseur USB-UART : la programmation se fait par ce connecteur. Deux précautions s'imposent :

  — l'adaptateur doit fonctionner en 3,3 V, car un adaptateur en 5 V détruirait l'entrée RXD0;

  — la broche 1 d'ICSP1 est reliée directement au rail `+3.3V`. Si l'adaptateur alimente la carte par cette broche alors que la batterie est branchée, deux sources se retrouvent en parallèle, et la diode D2 conduit du rail 3,3 V vers `+OUT`. Lorsque la carte est alimentée par sa batterie, il ne faut relier que RX, TX et la masse.

  La procédure de programmation est la suivante : poser BOOT1, appuyer sur RAZ1, lancer le téléversement (PlatformIO ou `esptool`), retirer BOOT1, puis appuyer de nouveau sur RAZ1.

C.5.3 Écran OLED et bus I²C

L'écran OLED SSD1306 de 128 × 64 points (Solomon Systech, s. d.) est raccordé par le connecteur OLED1 (masse, `+3.3V`, SCL, SDA) au bus I²C de l'ESP32 : SDA sur GPIO21 (broche 33) et SCL sur GPIO22 (broche 36). Son adresse est `0x3C`. Les lignes I²C sont à drain ouvert (NXP Semiconductors, s. d.) ; les résistances de rappel R12 et R13 (4,7 kΩ) fixent le niveau haut. Le temps de montée t_r entre 30 % et 70 % du niveau haut vaut :

$$t_r \approx 0{,}847\, R_p\, C_b = 0{,}847 \times 4{,}7\,\text{k}\Omega \times 50\,\text{pF} \approx 200\,\text{ns}, \tag{C.9}$$

pour une capacité de bus estimée à 50 pF. Cette valeur est inférieure au maximum de 300 ns du mode rapide à 400 kHz. Sur le circuit imprimé, les pistes SCL et SDA mesurent respectivement 14,6 mm et 7,0 mm; elles sont routées en face avant sans via.

C.5.4 Voyant d'alerte

Le voyant rouge D4, en série avec R11 (330 Ω), est commandé par la ligne `ESP_TX` (GPIO1). Le courant vaut environ $(3{,}3 - 1{,}9)/330 \approx 4{,}2$ mA, bien en deçà du courant admissible par une sortie de l'ESP32. GPIO1 est aussi la sortie TX0 de l'UART0. Au repos, une ligne UART est au niveau haut : tant que le micrologiciel n'utilise pas GPIO1 comme sortie ordinaire, le voyant reste allumé et clignote au rythme des données transmises (messages de démarrage, programmation). À l'inverse, lorsque le micrologiciel configure GPIO1 comme sortie ordinaire pour piloter le voyant, la liaison série devient inutilisable pendant le fonctionnement. Comme le signale le tableau 6, une version ultérieure devrait déplacer le voyant vers une broche libre, par exemple GPIO2.

C.5.5 Broches non utilisées

Les broches non utilisées du module sont marquées « non connectées » sur le schéma : entrées SENSOR_VP/VN, GPIO4 à GPIO35 non affectés, et broches SD0 à SD3, CLK et CMD de la flash interne. Les broches de la flash interne ne doivent en aucun cas être utilisées. Avec les quatre broches non connectées du chargeur, du convertisseur et du transistor de protection, on dénombre 32 pastilles volontairement libres sur le circuit imprimé.

C.6 Schéma complet

La figure 19 rassemble les quatre blocs. Les liaisons entre blocs passent par des étiquettes globales (`+VBAT`, `-VBAT`, `+OUT`, `+3.3V`, `GND`) plutôt que par des fils, ce qui garde chaque bloc lisible isolément. Les symboles `PWR_FLAG` indiquent au vérificateur électrique de KiCad quels rails sont effectivement alimentés.

C.7 Conception du circuit imprimé

C.7.1 Évolution de la carte

Le circuit imprimé a connu trois versions principales, résumées dans le tableau 9. La première version (V1.1), double face, reprenait le schéma ci-dessus sur une carte de 100×50 mm. La version 1.3 l'a convertie en quatre couches et entièrement reroutée en conservant le même contour. La version 1.4, présentée ici, réduit la surface de 42 % en réorganisant complètement le placement autour des dimensions réelles de l'écran OLED, mesurées sur son modèle 3D.

TABLE 9 – Évolution du circuit imprimé du nœud d'alerte.

	V1.1	**V1.3**	**V1.4 (finale)**
Contour	100×50 mm	100×50 mm	58×50 mm, angles arrondis (R 1 mm)
Surface	5000 mm^2	5000 mm^2	2900 mm^2 (-42 %)
Couches	2	4 (GND et `+3.3V` internes)	4 (GND et `+3.3V` internes)
Fixation	—	—	2 trous M3
Segments / vias	453 / 37	203 / 125	150 / 114
Erreurs DRC	—	0	0

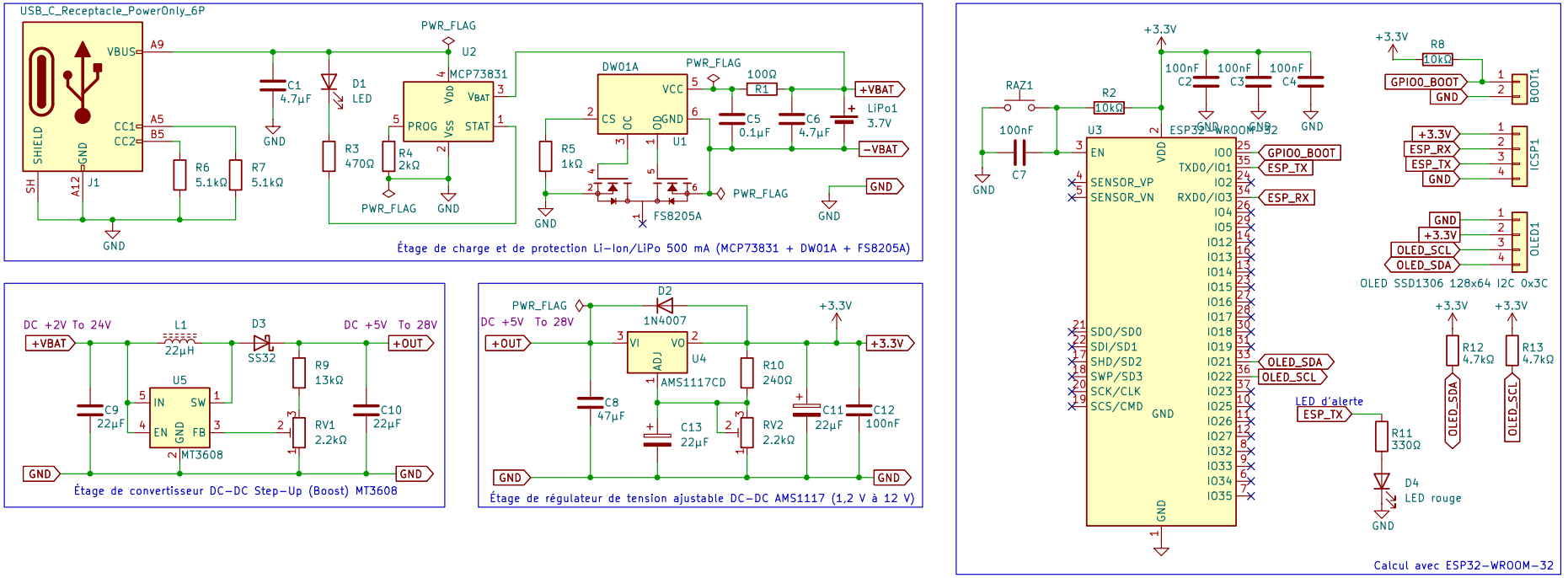


FIGURE 19 – Schéma électrique complet du nœud d'alerte autonome (KiCad, bordures retirées). En haut à gauche : charge USB-C et protection de la batterie LiPo (bloc 1). En bas à gauche : convertisseur élévateur MT3608 (bloc 2) et régulateur ajustable AMS1117 (bloc 3). À droite : module ESP32-WROOM-32, réinitialisation, programmation, écran OLED et voyant d'alerte (bloc 4).

C.7.2 Empilement des couches et règles de conception

La carte de 1,6 mm comporte quatre couches de cuivre de 35 µm (tableau 10). Les deux couches internes sont des plans pleins : la masse sur In1, juste sous la face avant, et le rail `+3.3V` sur In2. Toute piste y est interdite par une règle de conception. Cet empilement offre trois avantages :

— un chemin de retour de faible inductance sous chaque piste de la face avant, à seulement 0,1 mm de distance;

— une distribution du 3,3 V à faible impédance;

— un blindage entre le convertisseur à découpage et le module radio.

Les faces avant et arrière sont, en outre, remplies de cuivre relié à la masse.

TABLE 10 – Empilement des couches de la carte quatre couches (épaisseur totale 1,6 mm).

Couche	Rôle	Épaisseur
F.Cu	composants, signaux, masse de remplissage	35 µm
préimprégné	diélectrique FR4	0,10 mm
In1.Cu (GND)	plan de masse plein	35 µm
âme	diélectrique FR4	1,24 mm
In2.Cu (+3V3)	plan 3,3 V plein	35 µm
préimprégné	diélectrique FR4	0,10 mm
B.Cu	passifs CMS, liaisons, masse de remplissage	35 µm

Les pistes sont regroupées en classes : puissance batterie (PWR_BAT, 0,80 mm), 3,3 V (PWR_3V3, 0,60 mm), USB, I^2C et mesure (SENSE, 0,25 mm). Le fichier de règles personnalisées de KiCad impose en outre les contraintes suivantes :

— aucun via sur les nœuds de mesure (FB, ADJ et point milieu du pont R9/RV1) ni sur le bus I^2C;

— une longueur maximale de 70 mm pour l'I^2C;

— aucune piste sur les plans internes;

— un perçage minimal de 0,30 mm, sauf pour les trous de 0,2 mm intégrés à la pastille thermique de l'ESP32;

— une zone interdite de quatre couches sous l'antenne du module, au bord haut de la carte.

C.7.3 Prise en compte du modèle 3D de l'écran OLED

Le modèle STEP du module OLED a été analysé pour connaître son encombrement réel :

— circuit du module : 26×26 mm, verre d'environ $21{,}4 \times 24{,}7$ mm;

— connecteur à quatre broches sur le côté gauche, trous de fixation à 22×22 mm d'entraxe;

— une fois monté, entretoise plastique de 3,0 mm, verre culminant à 5,7 mm, soit 2,9 mm de hauteur libre sous le module.

Cette analyse a révélé que le modèle 3D associé à l'empreinte était retourné et décalé. Il a été corrigé (rotation et hauteur) afin que les rendus présentés plus loin soient fidèles. Les règles de KiCad interdisant tout chevauchement d'empreintes, aucun composant n'a été placé sous le module : seules des pistes de signal (SDA, SCL, EN) y passent.

C.7.4 Placement

Le placement a été obtenu par une optimisation sous contraintes (recuit simulé). Elle minimise la longueur totale des connexions, avec un poids plus fort sur les rails de puissance, sous les contraintes suivantes :

— antenne de l'ESP32 hors de la carte au bord haut;

— connecteurs USB-C et batterie sur les bords;

— connecteurs de programmation et bouton accessibles;

— boucle du convertisseur inférieure à 10 mm;

— nœuds de mesure courts;

— découplage de l'ESP32 à moins de 3 mm;

— aucun chevauchement d'empreintes sur chaque face.

Les résistances et condensateurs CMS ont été répartis sur les deux faces. Les distances obtenues sont rassemblées dans le tableau 11.

TABLE 11 – Distances caractéristiques du placement final (centre à centre des pastilles).

Contrainte	**Exigence**	**Obtenu**
Emprise de la boucle du convertisseur	< 10 mm	$5{,}95 \times 6{,}67$ mm
Diode D3 – condensateur de sortie C10	collés	2,86 mm
Broche FB (U5.3) – potentiomètre RV1	≤ 6 mm	2,89 mm
Broche ADJ (U4.1) – R10 / C13 / RV2	≤ 8 mm	6,79 / 6,12 / 5,74 mm
Broche VDD de l'ESP32 – découplage	≤ 3 mm	de 0,47 à 2,51 mm
Broche EN – C7 / R2	proche	2,52 / 3,75 mm
Broches I^2C – R12 / R13	≤ 10 mm	5,05 / 4,86 mm

C.7.5 Routage et vérification

Le routage a été réalisé par script dans l'environnement Python de KiCad, en sept étapes :

1. vias de masse, couture du bord, ceinture autour du convertisseur et vias thermiques;
2. rails de puissance;

3. nœuds de mesure;
4. bus I^2C;
5. liaisons série, démarrage et réinitialisation;
6. USB, charge et protection;
7. voyant.

Une vérification complète des règles de conception (DRC) a suivi chaque étape. Tous les segments sont orientés à 0, 45 ou 90°, aucune piste de puissance ne passe sous l'ESP32 ni sous l'écran, et la carte finale ne présente aucune erreur ni aucune connexion manquante. Le tableau 12 donne les principaux résultats.

TABLE 12 – Résultats du routage de la version 1.4.

Critère	**Résultat**
Erreurs DRC / connexions manquantes	0 / 0 (hors 32 pastilles volontairement libres)
Segments / vias	150 / 114
Vias de couture de masse	36 au bord, 9 autour du convertisseur
Vias thermiques du régulateur	9 (0,5 mm) vers le plan `+3.3V`
Cuivre du nœud de commutation SW	3,0 mm, face avant, 0,8 mm à 1,0 mm de large
Nœuds FB / ADJ	1,1 / 12,3 mm, face avant, sans via
Bus I^2C (SCL / SDA)	14,6 / 7,0 mm, sans via, écart ≥ 1,0 mm
Pistes sur plans internes	aucune

C.7.6 Vues du circuit imprimé et rendus 3D

Les figures 20 et 21 montrent le routage des deux faces. Les figures 22, 23 et 24 présentent les rendus 3D de la carte équipée, écran OLED compris.

C.7.7 Limites et évolutions envisagées

La version 1.4 satisfait toutes les règles de conception, mais plusieurs points restent à améliorer avant une fabrication en série :

— la ligne de réinitialisation EN est longue (environ 67 mm, surtout en face arrière) car le bouton est placé en bas à droite, à côté de l'écran. Le condensateur C7, placé à 2,5 mm de l'ESP32, limite la sensibilité aux perturbations, mais rapprocher le bouton du module est souhaitable;

— le voyant d'alerte partage la broche TX0 de l'UART0 et devrait être déplacé vers GPIO2;

— les potentiomètres RV1 et RV2 peuvent être remplacés par des résistances fixes une fois les tensions validées (environ 1,8 kΩ pour `+OUT` = 5 V et 390 Ω pour 3,3 V). Cela supprime le risque de dérégler le rail qui alimente l'ESP32;

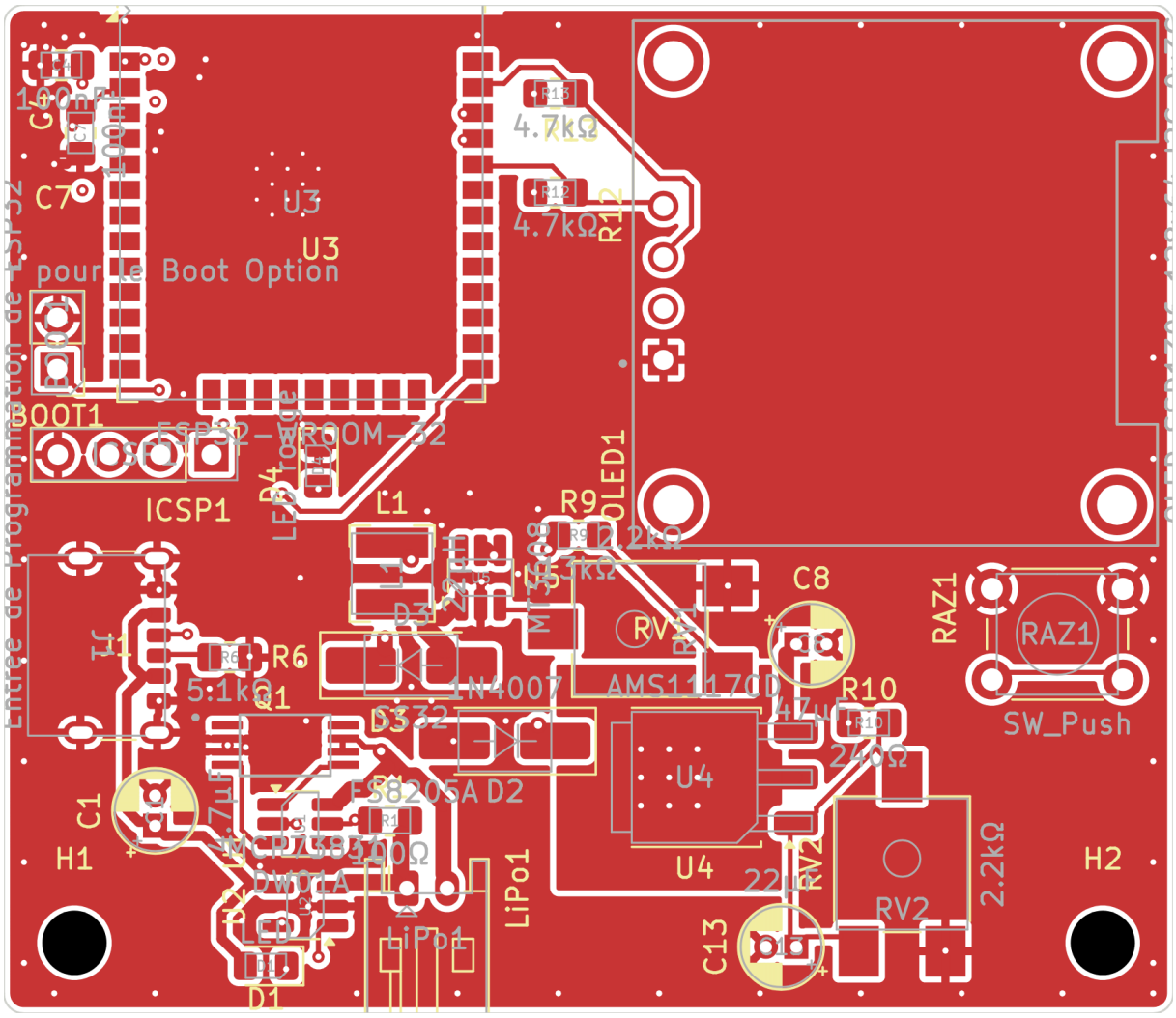


FIGURE 20 – Routage de la face avant (F.Cu) de la carte V1.4 (58 × 50 mm, quatre couches). Le rouge correspond au cuivre de la face avant, rempli de masse; les pistes sont détourées par leur garde d'isolement. En haut à gauche, le module ESP32-WROOM-32, dont l'antenne dépasse du bord supérieur; à gauche du module, la colonne de découplage (C4, C7). En haut à droite, l'empreinte de l'écran OLED, dont les quatre broches font face à l'ESP32; les résistances de rappel I^2C R12 et R13 sont placées entre les deux. Au centre, le convertisseur MT3608 (U5) avec L1 et D3, puis le régulateur AMS1117 (U4) et sa zone de cuivre `+3.3V` munie de vias thermiques. En bas à gauche, le connecteur USB-C J1, le chargeur MCP73831 (U2), le circuit de protection DW01A (U1) et le FS8205A (Q1); en bas, le connecteur de batterie LiPo1. Le potentiomètre RV1 est au centre, à côté du convertisseur, et RV2 en bas à droite, près du régulateur. Le bouton de réinitialisation RAZ1 est à droite, sous l'écran. Les deux trous de fixation M3 occupent les coins inférieurs.

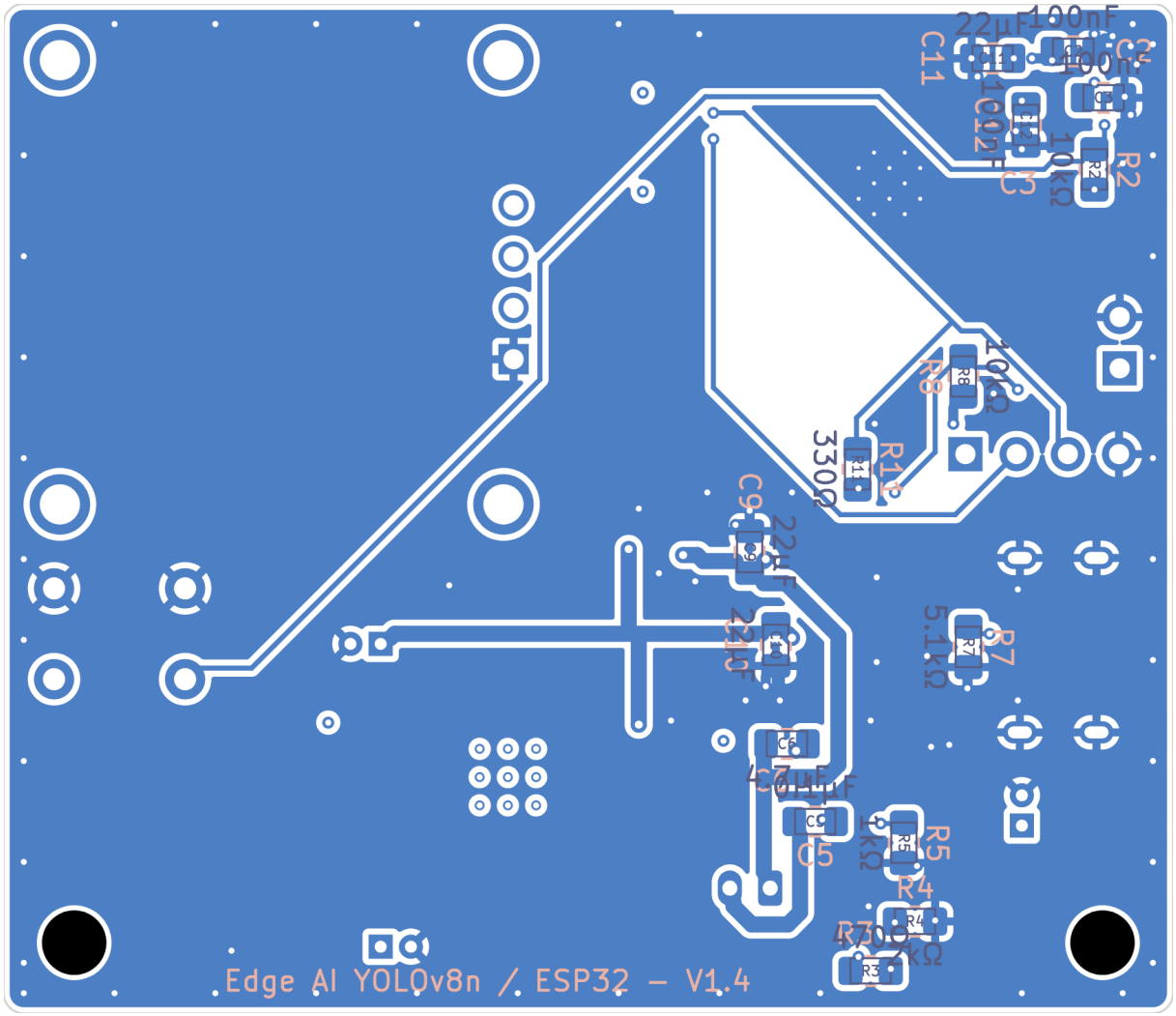


FIGURE 21 – Routage de la face arrière (B.Cu) de la carte V1.4, vue en miroir (comme vue par-dessous). Le bleu correspond au cuivre de la face arrière, rempli de masse. En haut à droite, les condensateurs de découplage de l'ESP32 (C2, C3, C11, C12) et R2, montés sous le bord du module. On y voit aussi les liaisons en face arrière : série TX/RX vers le connecteur de programmation, réinitialisation EN vers RAZ1, rails +OUT et +VBAT. Les petites pastilles réparties sur le pourtour sont les vias de couture de masse, au pas de 5 mm. Au centre, la grille de 3×3 vias thermiques est située sous le régulateur. Les zones blanches sont des îlots de cuivre isolés par les pistes, retirés automatiquement; la référence de masse reste assurée par le plan interne In1.

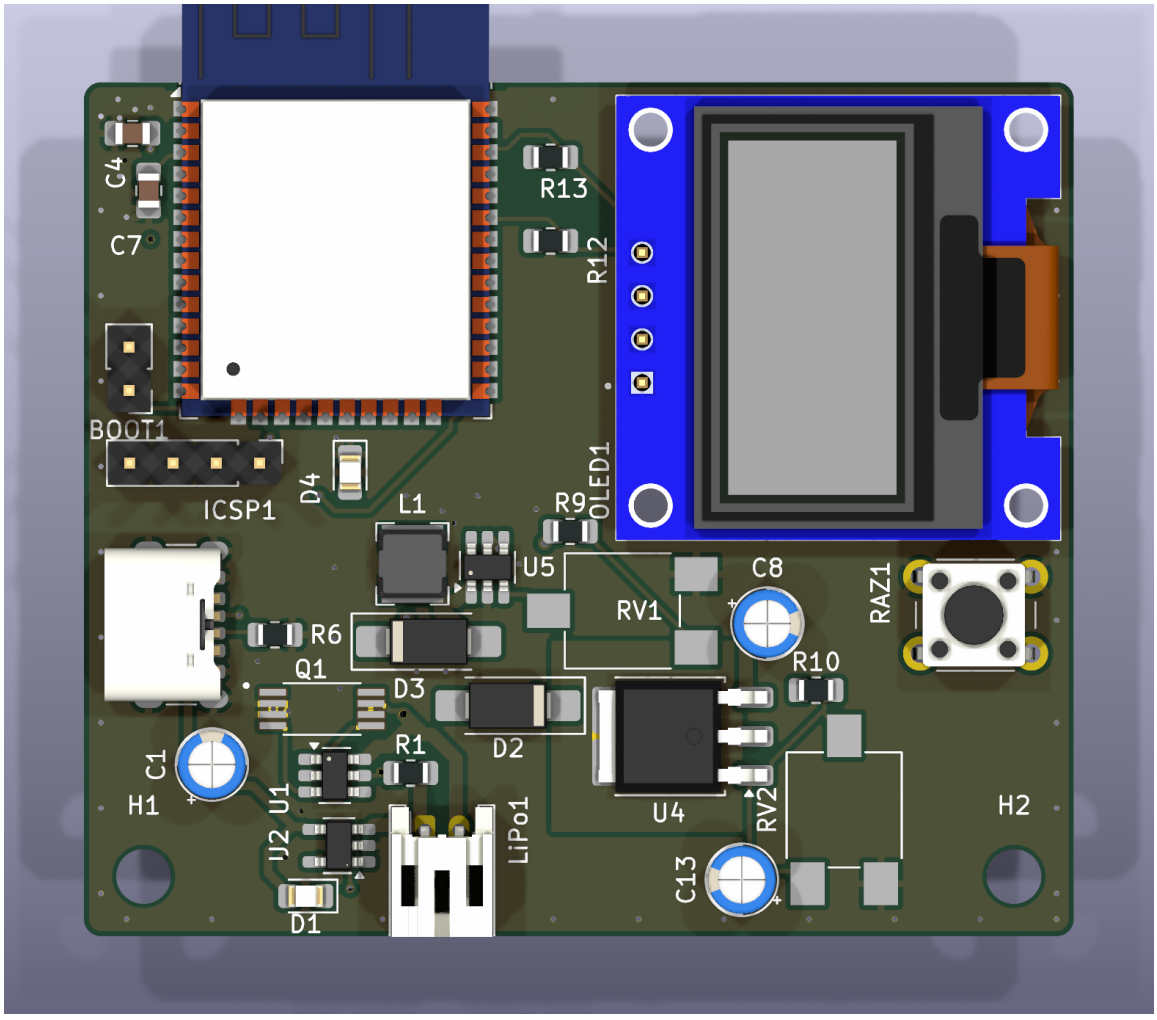


FIGURE 22 – Rendu 3D de la carte V1.4 vue de dessus (KiCad). L'antenne du module ESP32-WROOM-32 (en bleu foncé, en haut à gauche) dépasse du bord supérieur pour ne pas être perturbée par le cuivre de la carte. L'écran OLED de 0,96 pouce occupe le quart supérieur droit, connecteur côté ESP32. On distingue également les connecteurs de programmation ICSP1 et de démarrage BOOT1 sous et à gauche du module, le connecteur USB-C sur le bord gauche et le connecteur de batterie JST-PH sur le bord inférieur. Le bouton de réinitialisation est à droite, sous l'écran. Les condensateurs électrolytiques C1, C8 et C13 et le régulateur AMS1117 en boîtier TO-252 sont aussi visibles.

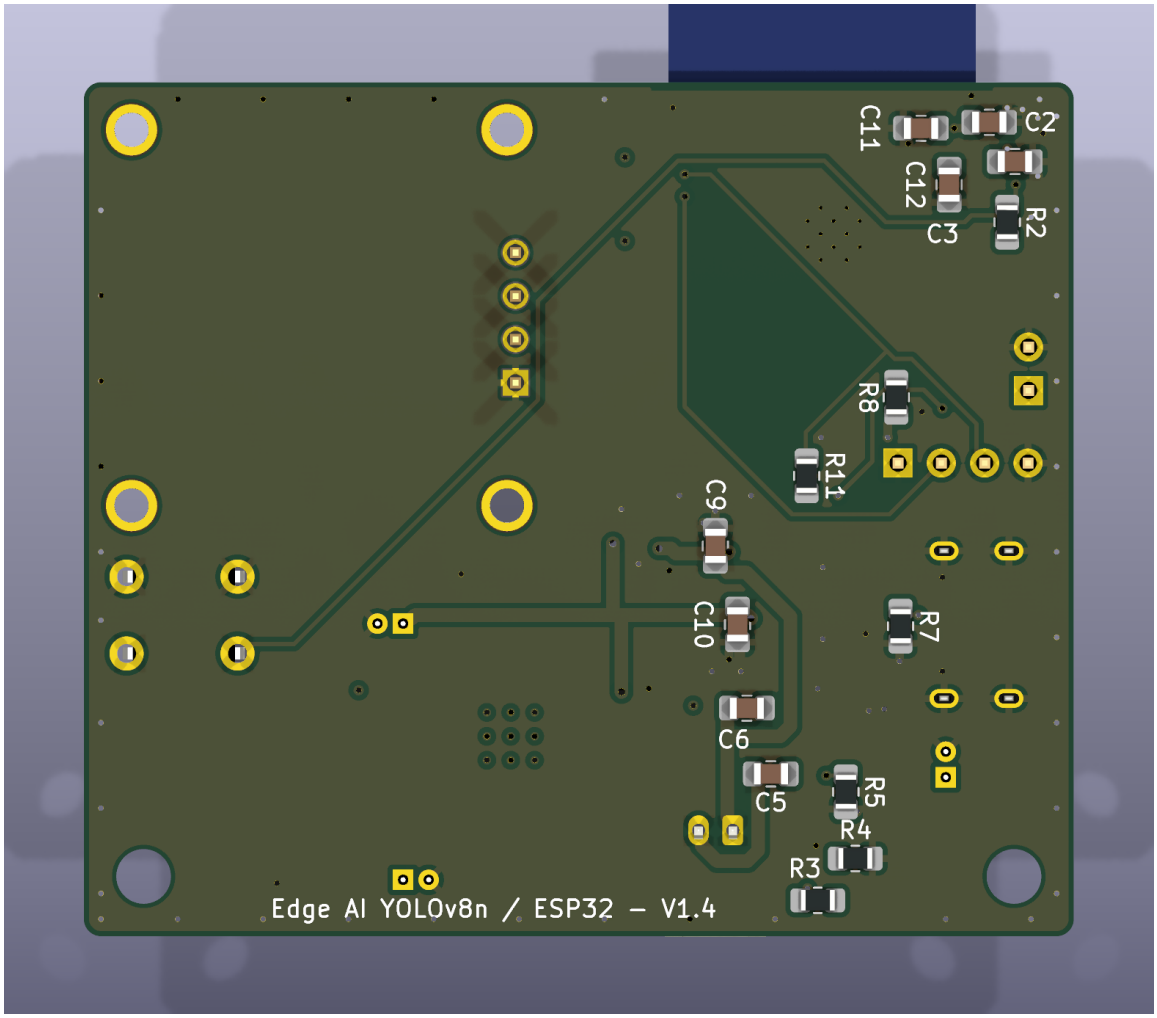


FIGURE 23 – Rendu 3D de la carte V1.4 vue de dessous. La face arrière ne porte que des composants CMS de faible hauteur (résistances et condensateurs 0805), ce qui permet de poser la carte à plat ou de la fixer par les deux trous M3. Les broches traversantes des connecteurs, du bouton, des condensateurs électrolytiques et de l'écran OLED dépassent sous la carte. Les broches de l'écran, d'environ 4 mm, sont à couper après soudure.

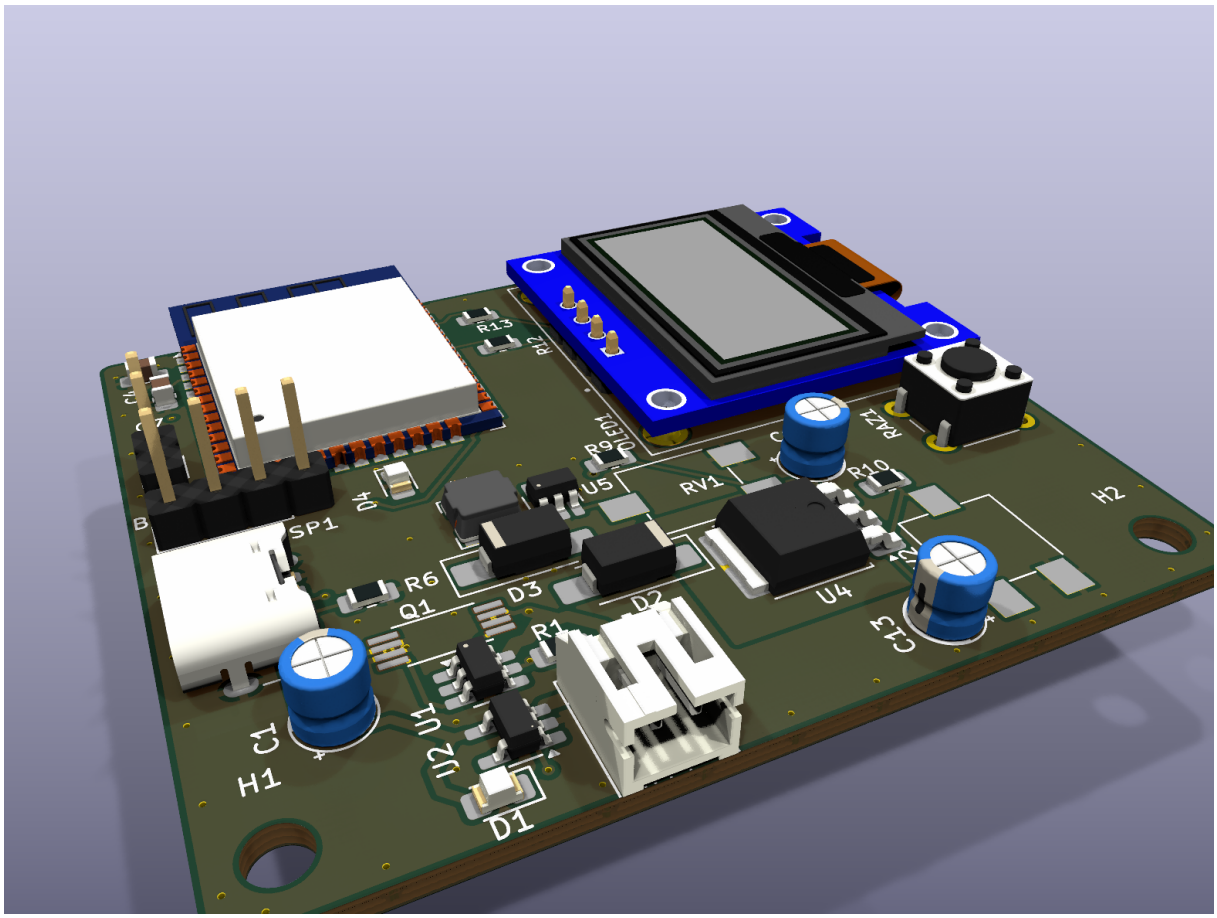


FIGURE 24 – Rendu 3D en perspective de la carte V1.4 équipée. L'écran OLED est monté sur son connecteur à quatre broches : son circuit se trouve à 3,0 mm au-dessus de la carte et le verre culmine à 5,7 mm. La nappe souple de l'afficheur est repliée côté droit. Au premier plan, le connecteur USB-C de recharge, le connecteur de batterie, le chargeur et la protection de la batterie, le convertisseur élévateur (inductance blindée et diodes SS32/1N4007) et le régulateur AMS1117. À l'arrière-plan, le module ESP32 avec les connecteurs de programmation, l'écran et le bouton de réinitialisation.

— une architecture à convertisseur abaisseur-élévateur (*buck-boost*) unique vers 3,3 V améliorerait l'autonomie d'environ 50 % (section C.4.4) : elle supprimerait les pertes du régulateur linéaire, de l'ordre d'un tiers de la puissance consommée;

— un circuit de partage de charge permettrait d'alimenter la carte depuis l'USB pendant la recharge, pour que le chargeur termine correctement son cycle;

— un pont diviseur sur la tension de la batterie, relié à une entrée analogique de l'ESP32, permettrait d'afficher le niveau de charge sur l'écran OLED;

— la carte n'a pas encore été fabriquée ni mesurée : les valeurs thermiques et l'autonomie données dans cette annexe restent des estimations par le calcul.